

Enhancing WAF Performance Using SIEM Alerts

A Graduation Project Report (Graduation 2)-submitted in partial fulfillment of the requirements for the degree of Bachelor of Science in Artificial Intelligence Engineering - Intelligent Information Security Systems Engineering

Prepared by:

Yamen Abdul Nasser Al-Hajeen
Abdul Aziz Al-Mjuel

Supervised by:

Dr. Wassem Al-Junaidi
Eng. Mohammad Yamen Hallak

Academic Year

2026/2027

Dedication

Acknowledgment

Abstract

The increasing reliance on web applications has significantly expanded the attack surface available to cybercriminals, making web application security a critical concern for organizations. Although security solutions such as Web Application Firewalls (WAFs), Security Information and Event Management (SIEM) systems, and Intrusion Detection Systems (IDSs) provide valuable protection, they often operate independently and generate large volumes of security alerts without providing intelligent decision-making or adaptive response capabilities. Consequently, security administrators are required to manually analyze alerts and determine appropriate mitigation actions, which increases response time and may reduce the effectiveness of incident handling.

This research proposes an integrated cybersecurity framework that combines ModSecurity, Wazuh, a Rule-Based Expert Decision Support System (DSS), and the pfSense firewall to provide real-time detection, intelligent decision-making, and adaptive automated response for web application attacks. The proposed framework employs customized Wazuh decoders, custom detection rules, and correlation rules to process security events generated by ModSecurity and identify multiple attack scenarios, including SQL Injection (SQLi), Cross-Site Scripting (XSS), Command Injection, repeated attacks, and multi-vector attacks. Based on predefined cybersecurity knowledge, the Rule-Based Expert Decision Support System evaluates the severity and context of detected events before selecting the most appropriate response strategy, ranging from security monitoring to administrator approval or automatic firewall blocking.

The proposed framework was implemented and evaluated within a controlled virtual laboratory environment using realistic web attack scenarios. Experimental evaluation demonstrated that the framework successfully processed and classified all generated security events while providing near real-time performance. The average attack detection time was approximately 0.52 seconds, the average decision-making time was 0.26 seconds, and the average automated response time was 0.46 seconds, resulting in an overall end-to-end response time of approximately 1.24 seconds for automatically mitigated attacks. Furthermore, the framework achieved 100% event processing and attack classification accuracy and produced no false positive alerts during legitimate user activities under the experimental environment.

The obtained results demonstrate that integrating customized attack detection, event correlation, rule-based expert decision support, and adaptive automated response into a unified cybersecurity architecture enhances the effectiveness of web application protection while reducing response time and supporting more reliable security operations. The proposed framework provides a practical foundation for future research on intelligent and adaptive cybersecurity systems for modern web applications.

Keywords: Cybersecurity, Web Application Security, Wazuh, ModSecurity, Security Information and Event Management (SIEM), Rule-Based Expert Decision Support System, SQL Injection, Cross-Site Scripting, Command Injection, Attack Correlation, Automated Incident Response, pfSense.

Table of Contents

Chapter 1	
Introduction	
1.1 Chapter Overview	1
1.2 Background	1
1.3 Scientific Problem and Project Justifications	2
1.4 Project Aim	2
1.5 Project Objectives	3
1.6 Related Work	4
1.7 State of the art.....	10
1.8 Challenges	11
1.9 Practical applications.....	11
Chapter 2	13
Theoretical Background.....	13
2.1 Cybersecurity	14
2.2 web applications.....	14
2.2.1 Web Application Security	15
2.2.2 OWASP Top 10	16
2.2.3 Web Application Firewall (WAF).....	17
2.2.4 ModSecurity Web Application Firewall	17
2.3 Security Information and Event Management (SIEM)	18
2.3.1 Wazuh SIEM	19
2.3.2 Log Collection	19
2.3.3 Log Analysys and Event Correlation	20
2.3.4 Alert Generation	20
2.3.5 Active Response	21
2.4 pfSense Firewall	22
2.4.1 Firewall Rules	22
2.4.2 Firewall Logging.....	23
2.5 Rule-Based Expert Decision Support System	24
2.5.1 Decision Support Systems (DSS)	24
2.5.2 Rule-Based Expert Systems.....	24
2.5.3 Knowledge Base	25
2.5.4 Inference Engine	25

2.5.5 Decision Rules (IF–THEN Rules)	26
2.5.6 Decision Documentation	27
2.5.7 False Positive Reduction	27
Chapter 3	29
Work Environment Setup	29
3.1 Chapter Overview	30
3.2 Development Environment	30
3.2.1 Hardware Environment	31
3.2.2 Virtualization Environment	32
3.2.3 Software Environment	34
3.3 Network Topology	45
3.4 System Configuration	49
3.5 Experimental Setup	55
3.5.1 Testing Environment	55
3.5.2 Attack Scenarios	55
3.5.3 Evaluation Procedure	56
3.6 Chapter Summary	56
Chapter 4	57
Proposed solution	57
4.1 Chapter Overview	58
4.2 Overall System Architecture	58
4.3 Development Methodology	59
4.4 Wazuh Detection Layer	60
4.4.1 Custom Decoders	60
4.4.2 Custom Detection Rules	61
4.5 Rule-Based Expert Decision Support System	64
4.5.1 System Overview	64
4.5.2 Alert Monitoring Module	65
4.5.3 Decision Engine	66
4.5.4 Automated Firewall Response	68
4.5.5 Decision Logging	69
4.6 Implementation and Attack Scenarios	70
4.6.1 SQL Injection Attack Scenario	70
4.6.2 Cross-Site Scripting (XSS) Attack Scenario	72
4.6.3 Command Injection Attack Scenario	73
4.6.4 Multi-Vector Attack Scenario	75

4.6.5 Repeated Attack Scenario	77
4.7 Chapter Summary	78
Chapter 5	79
Results and Discussion.....	79
5.1 Chapter Overview	80
5.2 Experimental Results	80
5.3 Performance Evaluation.....	81
5.3.1 Attack Detection Time Evaluation	81
5.3.2 Decision Support System Processing Time Evaluation	81
5.3.3 Automated Firewall Response Time Evaluation	82
5.4.3 Overall End-to-End System Performance.....	83
5.4 Detection Accuracy and False Positive Analysis	84
5.5 Comparative Analysis	85
Chapter 6	89
Conclusion and Future Work	89
6.1 Chapter Overview	90
6.2 Research Summary.....	90
6.3 Research Contributions.....	91
6.4 Discussion of Results	92
6.5 Limitations of the Proposed Framework.....	92
6.6 Future Work	93
6.7 Final Remarks	94
References	96

List of Figures

Figure 1 . OWASP Top 10 Vulnerabilities	16
Figure2 SIEM Components	18
Figure 3 Types of Firewalls	23
Figure 4 Decision Support Systems Achitecture	28
Figure5 DSS Types.....	28
Figure 6 Host Machine Hardware Specifications	32
Figure7 VirtualBox Environment	33
Figure 8 Ubuntu Home Screen.....	35
Figure 9 Apache Web Page.....	36
Figure10 DVWA Deployed on Apache.....	37
Figure 11 ModSecurity Integrated with Apache and Cunfigured using OWASP CRS.....	39
Figure 12 Wazuh Agent Connected to Manager Successfully.....	41
Figure13 Pfsense Dashboard	42
Figure 14 . Parrot Home Screen	44
Figure 15 Overall Architecture of the Proposed Framewark	46
Figure16 Data Flow of the Proposed System	48
Figure 17 Overall System Architecture.....	59
Figure 18 Development Methodology	60
Figure 19 Custom Wazuh Decoders.....	61
Figure 20 Custom Rules for Individual Attacks	62
Figure 21 Custom Wazuh Correlation Rules for Multi-Vector attack	63
Figure 22 Custom Rules for Repeated Attacks	63
Figure 23 Rule-Based Expert DSS Architecture	65
Figure 24 Alert Monitoring Initialization	65
Figure 25 Extraction of Security Attributes.....	66
Figure 26 Rule-Based Decision Logic.....	67
Figure 27 Automated Firewall Response Using Paramiko and pfSense	68
Figure 28 Generation and Logging of Decision Records	69
Figure 29 SQL Injection Executed Against DVWA	70
Figure 30 SQL Injection Alert by Wazuh	71
Figure 31 Decision Generated by DSS for SQL Injection	71
Figure 32 XSS Executed Against DVWA	72
Figure 33 . XSS Alert by Wazuh	72
Figure 34 Decision Generated by DSS for XSS	72
Figure35 Command Injection Executed Against DVWA	73
Figure 36 Command Injection Alert by Wazuh.....	74
Figure 37 Decision Generated by DSS for Command Injection	74
Figure 38 Automatically Generated Rule for Blocking the Attacker IP	74
Figure 39 Multi-Vector Attack Executed Against DVWA	75
Figure 40 Multi-Vector Attack Alert by Wazuh.....	76
Figure 41 Decision Generated by DSS for Multi-Vector Attack	76
Figure 42 Automatically Generated Rule for Blocking the Attacker IP.....	76
Figure 43 Repeated SQL Injection Executed Against DVWA.....	77
Figure 44 Repeated Attack Alert by Wazuh	77
Figure 45 . Decision Generated by DSS for Command Injection	77
Figure 46 Overall System Performance	84

List of Tables

Table 1 Summary of Previous Studies	7
Table 2 Comparison to the proposed System	9
Table 3 Hardware specification	31
Table 4 Comparison of Virtualization Platforms	33
Table 5 Comparison of Linux Server OS	34
Table 6 Comparison of Web Servers	36
Table 7 Comparison of Vulnerable Web Applications	37
Table 8 Comparison of Web Application Firewalls	38
Table 9 Comparison of SIEM Platforms	40
Table 10 Comparison of Firewall Platforms	42
Table 11 Comparison of Penetration Testing OS	43
Table 12 Comparison of Decision Support Approaches	45
Table 13 . Virtual Machines in Proposed Framework	47
Table 14 The Role of each Virtual Machine	47
Table 15 Ubuntu Server Configuration	49
Table 16 Apache Configuration	50
Table 17 DVWA configuration	51
Table 18 ModSecurity Configuration	51
Table 19 Wazuh Manager Configuration	52
Table 20 Wazuh Agent Configuration	53
Table 21 Pfsense Configuration	53
Table 22 DSS Configuration	54
Table 23 Summary of Custom Wazuh Detection Rules	64
Table 24 Decision Logic in the DSS	68
Table 25 Summary of SQL Injection Scenario	71
Table 26 Summary of XSS Scenario	73
Table 27 Summary of Command Injection Scenario	75
Table 28 Summary of Multi-Vector Attack Scenario	76
Table 29 Summary of Repeated Attack Scenario	78
Table 30 Overall Experimental Results	80
Table 31 Attack Detection Performance	81
Table 32 DSS Processing Performance	82
Table 33 Firewall Response Performance	82
Table 34 Overall System Performance	83
Table 35 Detection Accuracy Results	85
Table 36 False Positive Evaluation	85
Table 37 Comparison with Related Studies	87

List of Abbreviations and Acronyms

Abbreviation	Full Term
WAF	Web Application Firewall
SIEM	System Information and Event Management
AI	Artificial Intelligence
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
NIST	National Institute of Standards and Technology
OWASP	Open Worldwide Application Security Project
SQL	Structured Query Language
SQLI	SQL injection
XSS	Cross-Site Scripting
NGFW	Next Generation Firewall
WTA	Weighted Time Automata
IDS	Intrusion Detection System
IPS	Intrusion Prevention System
SOC	Security Operation Centre
RAG	Retrieval-Augmented Generation
LLM	Large Language Model
DoS	Denial of Service
SOAR	Security Orchestration, Automation, and Response
EDSS	Expert Decision Support System
LFI	Local File Inclusion
RFI	Remote File Inclusion
SSRF	Server-Side Request Forgery
NAT	Network Address Translation
VPN	Virtual Private Network
DSS	Decision Support System
DVWA	Damn Vulnerable Web Application
RAM	Random Access Memory
GPU	Graphical Processing Unit
CPU	Central Processing Unit
PHP	Hypertext Preprocessor
JSON	JavaScript Object Notation
OS	Operating System
SSH	Secure Shell Protocol
CRS	Core Rule Set
DB	DataBase

Chapter 1

Introduction

1.1 Chapter Overview

This chapter introduces the fundamental structural framework of the graduation project. It establishes the baseline security context for modern web application infrastructures. The comprehensive historical background of network logging and threat analysis is presented. Furthermore, the primary operational objectives of this integration research are defined. The core scientific problem and specific project justifications are formally outlined. Recent state-of-the-art technological advancements are explored alongside existing academic literature. Finally, technical engineering challenges are detailed, and practical real-world deployment applications are demonstrated.

1.2 Background

The rapid expansion of Internet technologies and web-based applications has significantly transformed the way organizations deliver services and exchange information. However, this evolution has also increased the number and sophistication of cyberattacks targeting web applications. Common attacks such as **SQL Injection (SQLi)**, Cross-Site Scripting (**XSS**), and Command Injection exploit vulnerabilities in web applications to gain unauthorized access, manipulate sensitive data, or disrupt system availability [1], [2].

To defend against these threats, organizations deploy multiple security technologies, each designed to protect a specific layer of the infrastructure. Firewalls provide network-level protection by filtering incoming and outgoing traffic based on predefined security policies, whereas Web Application Firewalls (**WAFs**) inspect **HTTP** and **HTTPS** traffic to identify and block malicious requests targeting web applications [1]. Although WAFs are effective in preventing many application-layer attacks, they primarily focus on web traffic and do not provide comprehensive visibility into events occurring across the entire network infrastructure.

Security Information and Event Management (**SIEM**) systems address this limitation by collecting, normalizing, and correlating security events generated by multiple devices, including firewalls, servers, intrusion detection systems, and web application firewalls. Through centralized log management and event correlation, SIEM platforms enable security administrators to monitor security incidents more efficiently and improve threat detection and incident response capabilities [3].

Despite the capabilities of SIEM platforms, the increasing volume of generated alerts often makes manual analysis slow and inefficient. Security analysts must continuously examine alerts, identify attack patterns, determine threat severity, and decide on appropriate mitigation actions. Consequently, organizations increasingly rely on automated decision support mechanisms to assist administrators in analyzing security events and responding to threats in a timely manner [4].

1.3 Scientific Problem and Project Justifications

Although modern organizations deploy multiple security solutions such as firewalls, Web Application Firewalls (WAFs), and Security Information and Event Management (SIEM) systems, these components often operate independently with limited coordination. While a WAF can successfully detect and block many web application attacks, it lacks the ability to correlate security events with other network devices and security systems. Similarly, SIEM platforms collect and analyze logs from multiple sources but generally rely on security analysts to manually interpret alerts and decide on appropriate response actions [5].

As the number of security events increases, manual analysis becomes time-consuming and may delay incident response. This delay provides attackers with additional opportunities to continue malicious activities before effective countermeasures are applied. Furthermore, administrators may overlook critical alerts due to alert fatigue or the overwhelming volume of generated security events [6].

Another limitation is that many existing security environments do not provide automated cooperation between the SIEM platform and the network firewall. Consequently, even after detecting a malicious activity, the response process often depends on manual intervention to identify the attacker, verify the threat, and block the malicious IP address. This increases response time and reduces the overall effectiveness of the security infrastructure.

Therefore, there is a need for an integrated solution capable of analyzing SIEM alerts, applying predefined decision rules, and automatically initiating appropriate response actions. Such integration can significantly reduce response time, improve attack mitigation, and enhance the overall security posture of web applications.

1.4 Project Aim

The primary aim of this project is to enhance the performance of Web Application Firewalls (WAFs) by integrating them with a Security Information and Event Management (SIEM) platform and an AI-based rule-based Decision Support System. The proposed framework is designed to collect and analyze security alerts generated by ModSecurity and pfSense through the Wazuh SIEM platform. Using a predefined knowledge base and inference rules, the Decision Support System evaluates security events, identifies potential threats, and determines the appropriate response. Based on the generated decision, the system automatically sends blocking commands to the pfSense firewall to prevent further malicious activities. This approach aims to reduce incident response time, minimize manual intervention, and improve the overall security and resilience of web applications against common cyberattacks.

1.5 Project Objectives

The main objectives of this project are:

- To design and implement a secure virtual network environment for evaluating web application security.
- To deploy and configure **ModSecurity** as a Web Application Firewall (WAF) for detecting and preventing common web-based attacks.
- To configure the **Wazuh** SIEM platform to collect, normalize, and centralize security logs from multiple data sources, including ModSecurity and **pfSense**.
- To integrate the security components into a unified architecture that enables seamless communication and information exchange between the WAF, SIEM, Decision Support System, and firewall.
- To develop a rule-based Expert Decision Support System that analyzes SIEM alerts using a predefined knowledge base and inference rules.
- To design and implement custom correlation rules capable of identifying complex and multi-stage cyberattacks by correlating security events from different sources.
- To reduce false positive alerts by applying intelligent rule-based decision-making before initiating automated response actions.
- To automate incident response by sending blocking commands to the pfSense firewall in order to prevent further malicious activities.
- To reduce the workload of security analysts by automating alert analysis, decision-making, and response processes.
- To evaluate the effectiveness of the proposed framework against common cyberattacks, including SQL Injection (**SQLi**), Cross-Site Scripting (**XSS**), Command Injection, and network reconnaissance attacks.
- To reduce incident response time by enabling automated security analysis, decision-making, and response execution.
- To document the decisions generated by the Expert Decision Support System, providing security analysts with clear records of detected threats, applied inference rules, and executed response actions for auditing and future analysis.

1.6 Related Work

Several studies have explored the integration of Security Information and Event Management (SIEM) platforms with Web Application Firewalls (WAFs) and other security technologies to improve cyberattack detection and incident response. **Sake and Sahni [7]** proposed a **collaborative architecture that integrates Wazuh SIEM, Wazuh Agents, and a Next Generation Firewall (NGFW)** to detect SQL Injection attacks through centralized log collection and event analysis. Their framework demonstrated that combining multiple security components significantly improves attack detection accuracy, enhances event correlation, and reduces the time required to identify security incidents. However, the proposed solution focused primarily on SQL Injection detection and did not provide automated decision-making or dynamic firewall response. In contrast, the proposed project extends this approach by integrating ModSecurity, pfSense Firewall, a rule-based Expert Decision Support System, and automated Active Response to block malicious IP addresses automatically.

Sreekanth V. R., Jasmeen Kaur, and Rohit Sansiya [8] presented a framework for **optimizing ModSecurity Web Application Firewall configurations while integrating generated security logs with a SIEM platform**. Their work improved web attack detection efficiency, reduced unnecessary alerts, and enhanced the visibility of web security events through centralized monitoring. Although the proposed framework strengthened web application protection, it primarily focused on WAF optimization without incorporating network-level monitoring or automated incident response. The proposed project expands this architecture by integrating pfSense firewall logs, custom correlation rules, a rule-based Expert Decision Support System, and automated response mechanisms within a unified security framework.

Fahmi, Hartono, and Anwar [9] proposed an integrated security framework **combining Wazuh SIEM, ModSecurity, and VirusTotal based on the NIST Cybersecurity Framework to improve website attack detection**. Their system successfully detected 10 out of 17 **OWASP** attack scenarios, including SQL Injection, Cross-Site Scripting (XSS), Command Injection, File Inclusion, File Upload, Open Redirect, and Brute Force attacks. Furthermore, the integration improved centralized monitoring and supported faster incident response. Since this architecture closely resembles the proposed project, it serves as an important reference. Nevertheless, the proposed framework does not implement a rule-based Expert Decision Support System or automatic firewall blocking. The proposed project addresses these limitations by integrating pfSense firewall logs, custom event correlation, intelligent rule-based decision support, and automated Active Response to dynamically block malicious IP addresses.

Overall, previous studies have demonstrated the effectiveness of integrating SIEM platforms with WAFs and other security solutions for improving attack detection and centralized monitoring. However, most existing approaches emphasize detection and monitoring while providing limited support for automated decision-making and response. The proposed system addresses these limitations by combining centralized log collection, custom correlation rules, a rule-based Expert Decision Support System, and automated firewall response within a single integrated security architecture.

Modeling Wazuh Rules with Weighted Timed Automata. M. M. Alhawari, J. Krallmann, et al. (2024) [10]

This study proposes a formal approach for modeling and validating Wazuh detection rules using Weighted Timed Automata (WTA). The objective is to verify the correctness and efficiency of Wazuh rule execution while improving the reliability of security event detection. The proposed model helps identify inconsistencies within detection rules and enhances the accuracy of security monitoring.

This research is closely related to our project because both systems rely on custom Wazuh rules to analyze security events. However, while the study focuses on validating and optimizing Wazuh rules, our project extends this concept by developing a rule-based Expert Decision Support System that analyzes correlated SIEM alerts and automatically triggers Active Response through the pfSense firewall.

The experimental results demonstrated that formal rule modeling improves the consistency and reliability of Wazuh rule execution while reducing rule conflicts and enhancing event detection accuracy.

Hybrid Intrusion Detection System and Network Infrastructure Vulnerability Mitigation Using Wazuh and Suricata Active Response. A. Y. Suryana et al. (2024) [11]

This study presents a hybrid intrusion detection architecture integrating Wazuh SIEM, Suricata IDS, and Active Response to automatically mitigate network attacks. The framework combines log correlation with automated response mechanisms to improve incident handling and reduce response time.

This work is related to our project because both systems implement Active Response after detecting malicious activities through Wazuh. However, our project differs by integrating ModSecurity WAF, pfSense Firewall, and a rule-based Expert Decision Support System that evaluates correlated security alerts before initiating automated firewall blocking.

The proposed framework successfully improved attack detection, accelerated incident response, and enhanced the protection of network infrastructure through automated mitigation techniques.

Enhancing Security Operations Center: Wazuh Security Event Response with Retrieval-Augmented Generation-Driven Copilot. A. Aljedaibi et al. (2025) [12]

This study introduces an intelligent security assistant that integrates with Wazuh SIEM to support Security Operations Center (SOC) analysts during incident investigation and response. The proposed system utilizes a Retrieval-Augmented Generation (RAG) architecture to analyze security alerts, provide contextual explanations, and assist analysts in making faster security decisions.

This research is closely related to our project because both systems aim to support security analysts by improving security decision-making based on SIEM alerts. Unlike the proposed Copilot, which depends on Large Language Models (LLMs), our project implements a rule-based Expert Decision Support System that performs explainable decision-making using predefined inference rules and automatically blocks malicious IP addresses through pfSense Active Response.

The proposed solution significantly reduced investigation time, improved analyst productivity, and enhanced the overall efficiency of Security Operations Center operations.

Exploration of Open Source SIEM Tools and Deployment of an Appropriate Wazuh-Based Solution for Strengthening Cyberdefense. Amami et al. (2024) [13] explored several open-source SIEM platforms and proposed the deployment of a Wazuh-based solution to strengthen organizational cyber defense capabilities. The study focused on centralized log collection, threat detection, and security monitoring while comparing different open-source SIEM solutions. This work is related to our project because both systems employ Wazuh as the primary SIEM platform for centralized security monitoring. However, our project extends the proposed framework by integrating ModSecurity WAF, pfSense Firewall, a rule-based Expert Decision Support System, and automated Active Response for blocking malicious IP addresses. The results demonstrated that Wazuh provides an effective open-source SIEM solution capable of improving centralized monitoring, security event correlation, and overall cyber defense capabilities.

Implementation of Security Information & Event Management (SIEM) Wazuh with Active Response and Telegram Notification for Mitigating Brute Force Attacks. Farrel et al. (2024) [14]

proposed a Wazuh-based SIEM solution integrated with Active Response and Telegram notifications to automatically mitigate brute force attacks. The architecture combines centralized log collection with automated response mechanisms to reduce incident response time and improve attack mitigation. This study is closely related to our project because both systems rely on Wazuh SIEM and automated response after detecting malicious activities. However, our project extends this approach by integrating ModSecurity WAF, pfSense Firewall, custom correlation rules, and a rule-based Expert Decision Support System capable of analyzing multiple attack scenarios before initiating automated firewall blocking.

The experimental results showed that the proposed solution successfully detected brute force attacks, automatically mitigated malicious activities, and reduced response time through Active Response automation.

Mitigation of Multi Target Denial of Service (DoS) Attacks Using Wazuh Active Response. Paramaputra et al. (2025) [15]

investigated the effectiveness of Wazuh Active Response in mitigating multi-target Denial of Service (DoS) attacks through automated firewall blocking. The proposed framework automatically detects malicious activities and executes predefined response actions to reduce the impact of network attacks. This work is related to our project because both systems utilize Wazuh SIEM and Active Response to automate incident handling. However, our proposed framework extends this concept by integrating ModSecurity WAF, pfSense Firewall, centralized log correlation, and a rule-based Expert Decision Support System capable of handling multiple web application attack scenarios rather than focusing solely on DoS attacks.

The experimental results demonstrated that Wazuh Active Response effectively mitigated multi-target DoS attacks, shortened response time, and improved the overall resilience of the protected network infrastructure.

Exploring and Improving the Usability of ModSecurity Web Application Firewall. Alagoz, Tok, and Bicakci. (2021) [16].

investigated the usability and configuration challenges of the ModSecurity Web Application Firewall (WAF). The study proposed a rule validation and feedback mechanism to reduce configuration errors and improve the effectiveness of ModSecurity. The authors demonstrated that proper WAF rule management significantly enhances the security and availability of protected web applications. This study is closely related to our project because both systems rely on ModSecurity WAF as the primary web application protection mechanism. However, our proposed framework extends this approach by integrating Wazuh SIEM, pfSense Firewall, custom correlation rules, a rule-based Expert Decision Support System, and automated Active Response to provide centralized monitoring and automatic mitigation of cyberattacks.

The experimental results showed that the proposed rule validation mechanism significantly reduced configuration errors, minimized web server downtime caused by incorrect rule syntax, and improved the overall effectiveness and usability of ModSecurity.

Table 1 Summary of Previous Studies

Ref.	Authors&years	Main contribution	Main Results
[7]	Sake & Sahni (2024)	Integrated Wazuh SIEM, Wazuh Agents, and NGFW for SQL Injection detection.	Improved SQL Injection detection accuracy, enhanced event correlation, and reduced incident response time.
[8]	Sreekanth V. R. et al. (2024)	Optimized ModSecurity WAF configuration with SIEM integration.	Increased web attack detection efficiency, reduced unnecessary alerts, and improved centralized monitoring
[9]	Fahmi et al. (2024)	Integrated Wazuh, ModSecurity, and VirusTotal based on the NIST framework.	Detected 10 of 17 OWASP attack scenarios and improved centralized monitoring and incident response.
[10]	Alhawari et al. (2024)	Modeled and validated Wazuh rules using Weighted Timed Automata.	Improved rule consistency, reduced rule conflicts, and

			enhanced detection accuracy.
[11]	Suryana et al. (2024)	Integrated Wazuh, Suricata, and Active Response for automated intrusion mitigation.	Improved attack detection and accelerated automated incident response.
[12]	Aljedaibi et al. (2025)	Developed an intelligent SOC assistant integrated with Wazuh using Retrieval-Augmented Generation (RAG).	Reduced investigation time and improved analyst productivity.
[13]	Amami et al. (2024)	Evaluated open-source SIEM solutions and deployed a Wazuh-based cyber defense framework.	Improved centralized monitoring and event correlation using Wazuh SIEM.
[14]	Farrel et al. (2024)	Implemented Wazuh SIEM with Active Response and Telegram notifications.	Successfully mitigated brute-force attacks and reduced response time.
[15]	Paramaputra et al. (2025)	Applied Wazuh Active Response for mitigating multi-target DoS attacks.	Reduced attack impact and improved automated network protection.
[16]	Alagoz et al. (2021)	Improved the usability and configuration of ModSecurity WAF.	Reduced configuration errors and enhanced ModSecurity effectiveness.

Table 2 Comparision to the proposed System

Feature	[7]	[8]	[9]	[10]	[11]	[12]	[13]	[14]	[15]	[16]	Proposed system
Wazuh SIEM	✓	✓	✓	✓	✓	✓	✓	✓	✓	✗	✓
ModSecurity WAF	✗	✓	✓	✗	✗	✗	✗	✗	✗	✓	✓
Pfsense Firewall	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓
Centralized Log Collection	✓	✓	✓	✓	✓	✓	✓	✓	✓	✗	✓
Multi-Source Log Collection	(partial)	✗	✓	✗	✓	✓	✓	✗	✗	✗	✓
Custome Correlation Rules	✗	(partial)	✓	✓	✓	✗	✗	✗	✗	✗	✓
Rule-based Decision Support	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓
AI-Assisted Decision Support	✗	✗	✗	✗	✗	✓	✗	✗	✗	✗	✓ expert system
Active Response	✗	✗	✗	✗	✓	✗	✗	✓	✓	✗	✓
Automatic IP Blocking	✗	✗	✗	✗	✓	✗	✗	✓	✓	✗	✓
Reduced Analyst Workload	✗	✗	✗	✗	(partial)	✓	✗	(partial)	✗	✗	✓
Reduced Response Time	✓	✗	✓	✗	✓	✓	✗	✓	✓	✗	✓
False Positive Reduction	✗	✓	(partial)	✓	(partial)	(partial)	✗	✗	✗	✓	✓

Table 2 demonstrates that previous studies mainly focused on attack detection, SIEM integration, WAF optimization, or automated response as separate components. In contrast, the proposed system integrates ModSecurity WAF, Wazuh SIEM, pfSense Firewall, centralized log collection, custom correlation rules, a rule-based Expert Decision Support System, and automated Active Response within a unified security architecture. This comprehensive integration enhances detection accuracy, reduces false positives, decreases response time, and minimizes the workload of security analysts.

1.7 State of the art

Cybersecurity has evolved significantly over the last decade due to the rapid increase in sophisticated cyber threats targeting web applications, enterprise networks, and cloud infrastructures. Modern security architectures no longer rely on isolated security devices; instead, they integrate multiple security technologies to provide comprehensive visibility, threat detection, and automated incident response. International cybersecurity standards emphasize the importance of continuous monitoring, centralized security management, and risk-based decision-making to improve organizational resilience against cyberattacks [17], [18].

One of the most significant developments in web application security is the widespread adoption of Web Application Firewalls (WAFs). Unlike traditional firewalls, WAFs inspect HTTP and HTTPS traffic to detect and block attacks targeting web applications, including SQL Injection, Cross-Site Scripting (XSS), Command Injection, Local and Remote File Inclusion, and other vulnerabilities identified in the OWASP Top 10. Among the available open-source WAF solutions, ModSecurity has become one of the most widely adopted platforms because of its flexibility, customizable rule sets, and compatibility with major web servers such as Apache and Nginx [17], [19]. At the same time, organizations increasingly depend on Security Information and Event Management (SIEM) platforms to collect, normalize, correlate, and analyze security events generated by multiple devices. SIEM solutions provide centralized log management, real-time event correlation, threat detection, and security visualization, enabling security analysts to investigate incidents more efficiently. Modern SIEM platforms such as Wazuh support agent-based monitoring, custom detection rules, file integrity monitoring, vulnerability detection, and Active Response capabilities, making them suitable for Security Operations Center (SOC) environments [20], [21]. Another important trend in modern cybersecurity is the implementation of Security Orchestration, Automation, and Response (SOAR) technologies. These solutions automate repetitive security tasks by executing predefined response actions immediately after threat detection, reducing the time between detection and mitigation. Automated actions may include blocking malicious IP addresses, updating firewall rules, isolating compromised systems, or generating incident reports. Security automation has become essential for reducing analyst workload, improving operational efficiency, and minimizing the impact of cyberattacks [23].

In addition to automation, Expert Decision Support Systems (EDSS) have gained increasing attention as a practical approach for assisting cybersecurity professionals during incident analysis. Unlike machine learning models that require large training datasets, rule-based expert systems use predefined knowledge bases and inference rules to provide transparent, explainable, and consistent security decisions. Such systems are particularly suitable for environments where explainability, reliability, and deterministic behavior are essential requirements [22].

Furthermore, modern threat intelligence frameworks such as the MITRE ATT&CK Framework have become widely adopted for understanding attacker behavior and mapping adversarial techniques across different stages of the cyber kill chain. Integrating MITRE ATT&CK with SIEM platforms enables more accurate event correlation, attack classification, and prioritization of incident response activities, thereby enhancing the effectiveness of Security Operations Centers [24].

Although significant progress has been achieved in SIEM platforms, Web Application Firewalls, security automation, and decision support technologies, many existing

security solutions still focus on individual security components rather than providing a fully integrated architecture.

The proposed project addresses this limitation by combining ModSecurity Web Application Firewall, Wazuh SIEM, pfSense Firewall, centralized log collection, custom correlation rules, and a rule-based Expert Decision Support System within a single cybersecurity framework. This integration improves attack detection accuracy, reduces false positive alerts, shortens response time, supports explainable security decisions, and automatically initiates Active Response to block malicious IP addresses.

1.8 Challenges

Developing an integrated cybersecurity framework that combines multiple security technologies presents several technical and operational challenges. One of the primary challenges is the collection and normalization of security logs generated by different sources, including ModSecurity WAF, pfSense Firewall, and Wazuh agents. These components generate heterogeneous log formats that must be processed and correlated accurately to provide meaningful security insights [20].

Another challenge is reducing false positive alerts while maintaining a high detection rate. Large volumes of security events may overwhelm security analysts if every alert is treated as a critical incident. Therefore, designing effective custom correlation rules and a rule-based Expert Decision Support System is essential for distinguishing real attacks from benign activities [22].

Automating incident response also presents challenges because response actions must be executed accurately without affecting legitimate users. The system must evaluate correlated security events before initiating firewall blocking to avoid unnecessary service disruption. In addition, ensuring compatibility and seamless communication between Wazuh SIEM, ModSecurity WAF, pfSense Firewall, and the Expert Decision Support System requires careful system integration and configuration [22], [23].

Finally, maintaining scalability and reliability is another important challenge. As the number of monitored systems and security events increases, the proposed framework should continue to provide efficient log processing, timely decision-making, and rapid automated response without degrading overall system performance [6].

1.9 Practical applications

Developing an integrated cybersecurity framework that combines multiple security technologies presents several technical and operational challenges. One of the primary challenges is the collection and normalization of security logs generated by different sources, including ModSecurity WAF, pfSense Firewall, and Wazuh agents. These components generate heterogeneous log formats that must be processed and correlated accurately to provide meaningful security insights.

Another challenge is reducing false positive alerts while maintaining a high detection rate. Large volumes of security events may overwhelm security analysts if every alert is treated as a critical incident. Therefore, designing effective custom correlation rules and a rule-based Expert Decision Support System is essential for distinguishing real attacks from benign activities.

Automating incident response also presents challenges because response actions must be executed accurately without affecting legitimate users. The system must evaluate

correlated security events before initiating firewall blocking to avoid unnecessary service disruption. In addition, ensuring compatibility and seamless communication between Wazuh SIEM, ModSecurity WAF, pfSense Firewall, and the Expert Decision Support System requires careful system integration and configuration.

Finally, maintaining scalability and reliability is another important challenge. As the number of monitored systems and security events increases, the proposed framework should continue to provide efficient log processing, timely decision-making, and rapid automated response without degrading overall system performance.

Chapter 2

Theoretical Background

2.1 Cybersecurity

Cybersecurity is the practice of protecting computer systems, networks, applications, and digital information from unauthorized access, cyberattacks, data breaches, and other malicious activities. As organizations increasingly rely on digital technologies and Internet-connected services, cybersecurity has become a fundamental requirement for ensuring the confidentiality, integrity, and availability of information assets. Effective cybersecurity strategies combine technical controls, organizational policies, and continuous monitoring to defend against both internal and external threats [1], [18]. The primary objective of cybersecurity is to safeguard information systems while maintaining the confidentiality, integrity, and availability of data, commonly known as the CIA Triad. Confidentiality ensures that sensitive information is accessible only to authorized users, integrity guarantees that information remains accurate and unaltered, and availability ensures that systems and services remain accessible whenever they are needed. These three principles serve as the foundation for designing secure information systems and cybersecurity solutions [1], [18].

Cyber threats continue to evolve in complexity and frequency, targeting web applications, enterprise networks, cloud infrastructures, and critical services. Common attack techniques include SQL Injection, Cross-Site Scripting (XSS), Distributed Denial-of-Service (DDoS) attacks, brute-force attacks, malware, phishing, and privilege escalation. Many of these attacks exploit software vulnerabilities or misconfigurations to gain unauthorized access to systems or compromise sensitive information [17], [18], [24].

To address these challenges, modern organizations deploy multiple layers of security technologies, including firewalls, Web Application Firewalls (WAFs), Intrusion Detection and Prevention Systems (IDS/IPS), antivirus solutions, endpoint protection, and Security Information and Event Management (SIEM) platforms. These technologies work together to detect malicious activities, monitor security events, and respond to incidents before they cause significant damage. Integrating these security mechanisms within a centralized monitoring framework has become one of the most effective approaches for improving cyber resilience and reducing incident response time [18], [20].

Because cyber threats continuously evolve, cybersecurity is no longer limited to preventing attacks but also includes rapid detection, incident response, recovery, and continuous improvement. International standards such as the NIST Cybersecurity Framework (CSF 2.0) emphasize a comprehensive security lifecycle that includes governance, risk management, continuous monitoring, threat detection, response, and recovery. This lifecycle enables organizations to maintain a proactive security posture and effectively manage emerging cyber risks [18].

2.2 web applications

Web applications are software applications that are accessed through web browsers over the Internet or private networks without requiring installation on users' devices. Unlike traditional desktop applications, web applications are hosted on web servers and allow users to interact with services remotely using standard web technologies such as HTTP and HTTPS protocols. Today, web applications are widely used in various sectors, including e-commerce, online banking, healthcare, education, social media, cloud computing, and government services, making them an essential part of modern digital infrastructures.

The rapid growth of web-based services has significantly increased the importance of web applications for both organizations and individuals. They provide flexible access to information and services from any location while simplifying software deployment and maintenance. However, because they are publicly accessible through the Internet, web applications are continuously exposed to cyber threats that target application vulnerabilities and sensitive data. Consequently, ensuring the reliability, availability, and security of web applications has become a critical objective for organizations that depend on online services [25].

2.2.1 Web Application Security

Web application security is a specialized field of cybersecurity that focuses on protecting web applications from attacks that exploit vulnerabilities in application logic, software implementation, and communication protocols. Since web applications are publicly accessible through the Internet, they have become one of the primary targets for cybercriminals seeking unauthorized access to sensitive information or system resources. Effective web application security requires implementing multiple security controls throughout the application lifecycle, including secure software development practices, vulnerability assessment, access control mechanisms, input validation, encryption, and continuous security monitoring. These measures help organizations reduce the risk of data breaches, service disruption, and unauthorized system access while maintaining the confidentiality, integrity, and availability of web services [17].

Modern web applications are exposed to a wide variety of cyber threats that exploit weaknesses in application code or server configuration. According to the OWASP Top 10, the most critical web application security risks include Broken Access Control, Cryptographic Failures, Injection attacks such as SQL Injection, Insecure Design, Security Misconfiguration, Vulnerable and Outdated Components, Identification and Authentication Failures, Software and Data Integrity Failures, Security Logging and Monitoring Failures, and Server-Side Request Forgery (SSRF). These vulnerabilities can allow attackers to manipulate databases, execute malicious code, steal confidential information, or gain unauthorized access to web systems. Consequently, protecting web applications has become a fundamental requirement for organizations that provide online services [17].

To mitigate these threats, organizations commonly deploy multiple defensive mechanisms such as secure coding practices, vulnerability scanning tools, penetration testing, Web Application Firewalls (WAFs), and continuous monitoring solutions. Among these technologies, WAFs play a critical role by inspecting HTTP and HTTPS requests before they reach the web server, enabling organizations to detect and block malicious requests in real time. Integrating WAFs with centralized monitoring platforms further enhances the ability to detect, analyze, and respond to security incidents affecting web applications [25].

2.2.2 OWASP Top 10

The OWASP Top 10 is one of the most widely recognized security awareness standards for web application security. Published by the Open Worldwide Application Security Project (OWASP), it identifies the ten most critical security risks affecting modern web applications based on industry research and real-world vulnerability data. The primary objective of the OWASP Top 10 is to help developers, security professionals, and organizations understand common web application vulnerabilities and implement effective security controls during the software development lifecycle.

The latest edition, OWASP Top 10:2021, classifies the most critical web application security risks into ten categories: Broken Access Control, Cryptographic Failures, Injection, Insecure Design, Security Misconfiguration, Vulnerable and Outdated Components, Identification and Authentication Failures, Software and Data Integrity Failures, Security Logging and Monitoring Failures, and Server-Side Request Forgery (SSRF). These vulnerabilities represent the most common attack vectors exploited by cybercriminals to compromise web applications, steal sensitive information, execute malicious commands, or disrupt online services.

The OWASP Top 10 serves as an international reference for designing secure web applications and evaluating web application security. Many security solutions, including Web Application Firewalls (WAFs), vulnerability scanners, and penetration testing tools, utilize OWASP guidelines to detect and mitigate web-based attacks. In the proposed project, the deployed ModSecurity Web Application Firewall is configured to detect and prevent several attack types identified in the OWASP Top 10, particularly SQL Injection and Cross-Site Scripting (XSS), before forwarding security events to the Wazuh SIEM platform for centralized monitoring and further analysis [17].

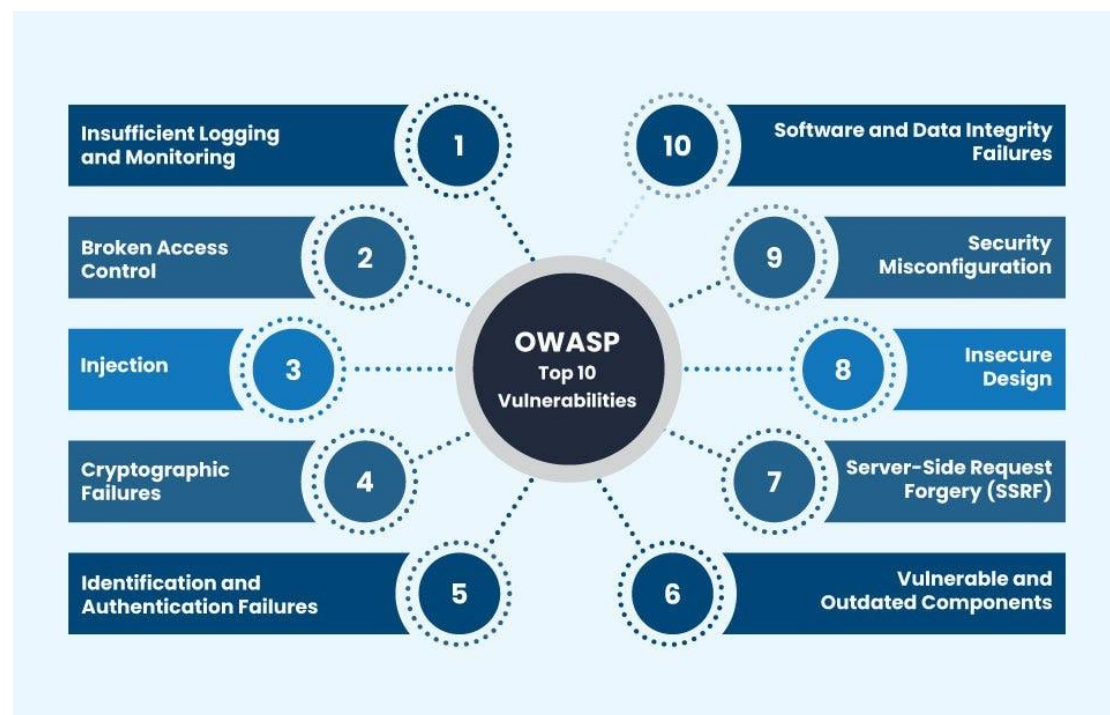


Figure 1 . OWASP Top 10 Vulnerabilities

2.2.3 Web Application Firewall (WAF)

A Web Application Firewall (WAF) is a specialized security solution designed to protect web applications by monitoring, filtering, and analyzing HTTP and HTTPS traffic exchanged between clients and web servers. Unlike traditional network firewalls, which primarily inspect network packets based on IP addresses, ports, and protocols, a WAF operates at the application layer (Layer 7 of the OSI model), allowing it to detect attacks that specifically target web applications. By inspecting the content of web requests and responses, a WAF can identify malicious patterns and enforce predefined security policies before requests reach the protected application.

A WAF protects web applications against a wide range of attacks, including SQL Injection (SQLi), Cross-Site Scripting (XSS), Command Injection, File Inclusion attacks, Cookie Manipulation, and other application-layer threats. It achieves this by comparing incoming requests with predefined security rules and signatures, allowing legitimate traffic while blocking or logging suspicious requests. Depending on the deployment configuration, a WAF may operate in detection mode, where malicious activities are only logged, or in prevention mode, where malicious requests are actively blocked before reaching the web server.

Modern organizations increasingly deploy WAF solutions as an essential component of a layered cybersecurity strategy because they provide real-time protection against web-based attacks while generating detailed security logs that can be integrated with centralized monitoring platforms. These logs support incident investigation, attack analysis, and security event correlation, enabling organizations to improve both web application security and overall cybersecurity visibility [26].

2.2.4 ModSecurity Web Application Firewall

ModSecurity is an open-source Web Application Firewall (WAF) that provides real-time protection for web applications by inspecting HTTP and HTTPS requests before they reach the web server. It operates at the application layer and applies predefined security rules to identify malicious requests, making it capable of detecting and preventing a wide range of web-based attacks. Because of its flexibility and compatibility with web servers such as Apache, Nginx, and IIS, ModSecurity has become one of the most widely adopted open-source WAF solutions for protecting web applications.

One of the main features of ModSecurity is its rule-based detection engine, which analyzes incoming requests against customizable security policies. It supports the OWASP Core Rule Set (CRS), a collection of predefined security rules designed to detect common attacks such as SQL Injection, Cross-Site Scripting (XSS), Command Injection, Local File Inclusion (LFI), Remote File Inclusion (RFI), Directory Traversal, and other application-layer threats. Organizations can also create custom rules to address application-specific security requirements.

ModSecurity can operate in two different modes. In Detection Only mode, malicious requests are identified and logged without blocking user traffic, allowing administrators to evaluate rule performance and reduce false positive alerts. In Prevention mode,

requests matching security rules are automatically blocked before reaching the protected web application. Additionally, ModSecurity generates detailed audit logs containing information about detected attacks, request headers, payloads, timestamps, and triggered security rules. These logs can be forwarded to Security Information and Event Management (SIEM) platforms for centralized monitoring, event correlation, and incident investigation.

In the proposed project, ModSecurity is deployed on the Apache web server hosting the DVWA application. It detects malicious HTTP requests generated during attack simulations and records detailed security events. These logs are collected by the Wazuh Agent and forwarded to the Wazuh Manager, where they are correlated with firewall events and analyzed by the rule-based Expert Decision Support System before initiating automated Active Response through the pfSense firewall [27].

2.3 Security Information and Event Management (SIEM)

Security Information and Event Management (SIEM) is a cybersecurity technology that combines Security Information Management (SIM) and Security Event Management (SEM) into a unified platform for collecting, storing, analyzing, and monitoring security events generated by multiple systems and devices. SIEM solutions provide centralized visibility of an organization's security posture by aggregating logs from servers, firewalls, intrusion detection systems, web application firewalls, operating systems, and network devices into a single platform. This centralized approach enables security teams to detect suspicious activities more efficiently and respond to security incidents in a timely manner.

A SIEM platform continuously collects security logs from different data sources, normalizes the collected information into a standardized format, and analyzes events using predefined rules and correlation mechanisms. By examining relationships between events occurring across multiple systems, SIEM platforms can identify attack patterns that might remain undetected when analyzing individual logs separately. This capability significantly improves threat detection accuracy while reducing the complexity of monitoring large and distributed computing environments. Modern SIEM platforms also provide real-time alerting, event visualization, compliance reporting, forensic investigation capabilities, and integration with automated response mechanisms. These features enable Security Operations Centers (SOCs) to improve situational awareness, reduce incident response time, and enhance overall cybersecurity operations. Because of these capabilities, SIEM has become a fundamental component of modern enterprise cybersecurity architectures and is widely adopted across organizations that require continuous monitoring and effective incident management [28].

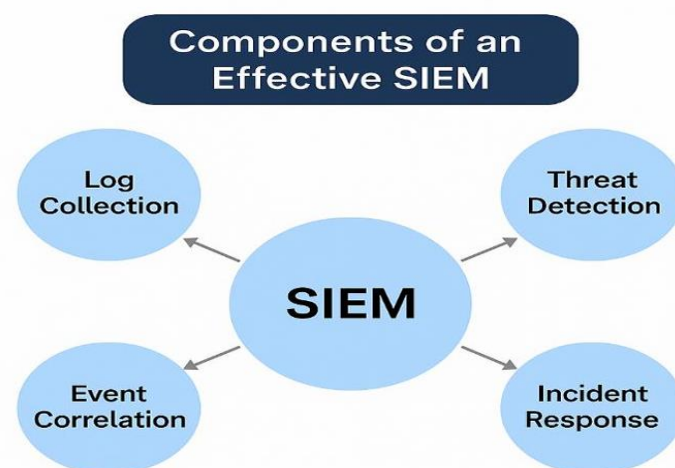


Figure2 SIEM Components

2.3.1 Wazuh SIEM

Wazuh is an open-source Security Information and Event Management (SIEM) and Extended Detection and Response (XDR) platform designed to provide centralized security monitoring, threat detection, log analysis, and incident response. It enables organizations to collect and analyze security events generated by endpoints, servers, network devices, firewalls, and web applications from a single platform. By combining log collection, event correlation, vulnerability detection, file integrity monitoring, and automated response capabilities, Wazuh helps security analysts identify and investigate cyber threats more efficiently.

The Wazuh architecture consists of four primary components: the Wazuh Agent, Wazuh Server (Manager), Wazuh Indexer, and Wazuh Dashboard. The Wazuh Agent is installed on monitored endpoints to collect security events and system information. The Wazuh Server receives these events, decodes and analyzes them using built-in and custom detection rules, and generates security alerts. Processed events are then forwarded to the Wazuh Indexer, where they are indexed and stored for efficient searching and analysis. Finally, the Wazuh Dashboard provides a web-based interface that allows security analysts to visualize alerts, monitor security events, investigate incidents, and manage the Wazuh environment through centralized dashboards. One of the key advantages of Wazuh is its flexibility and extensibility. It supports integration with various security technologies, including firewalls, intrusion detection systems, Web Application Firewalls (WAFs), cloud platforms, and external threat intelligence services. In addition, Wazuh allows administrators to create custom rules, decoders, and Active Response actions, enabling organizations to tailor the platform to their specific security requirements.

In the proposed project, Wazuh serves as the central SIEM platform responsible for collecting logs from ModSecurity and pfSense Firewall, analyzing security events using custom correlation rules, generating alerts, and providing the security information required by the rule-based Expert Decision Support System before initiating automated firewall blocking through Active Response [20].

2.3.2 Log Collection

Log collection is one of the fundamental functions of a Security Information and Event Management (SIEM) system. It involves gathering security-related events and system logs from multiple data sources, including operating systems, web servers, firewalls, Web Application Firewalls (WAFs), network devices, and security applications. Centralizing these logs enables security analysts to monitor all security events from a single platform instead of examining each device individually.

Within the Wazuh platform, log collection is performed by Wazuh Agents installed on monitored systems or through agentless integrations with supported network devices. The collected logs are securely transmitted to the Wazuh Manager, where they are decoded, normalized, and prepared for further analysis. Wazuh supports numerous log formats, allowing organizations to integrate heterogeneous security devices into one centralized monitoring environment.

Centralized log collection provides several operational advantages, including improved visibility of security events, simplified incident investigation, efficient compliance reporting, and faster threat detection. By collecting logs from different security

components, analysts can reconstruct attack timelines and identify relationships between security events occurring across multiple systems.

In the proposed project, log collection represents the first stage of the security monitoring process. Security logs generated by the ModSecurity Web Application Firewall and the pfSense Firewall are forwarded to the Wazuh Manager, where they are processed and stored for subsequent event correlation, rule evaluation, and decision support. This centralized collection mechanism provides the security information required for generating accurate alerts and initiating automated incident response [20].

2.3.3 Log Analysis and Event Correlation

Log analysis is the process of examining collected security logs to identify suspicious activities, security incidents, and indicators of compromise. After logs are collected from multiple security devices, a SIEM platform analyzes their content by extracting relevant information such as timestamps, source and destination IP addresses, event types, user activities, and triggered security rules. This analysis transforms raw log data into meaningful security information that enables security analysts to detect malicious behavior and investigate potential cyberattacks. Effective log analysis improves the visibility of security events and helps organizations identify threats before they cause significant damage.

Event correlation is an advanced SIEM capability that associates related security events originating from different data sources to identify attack patterns that cannot be recognized from individual events alone. Instead of analyzing each log independently, correlation rules examine relationships between multiple events occurring within a specific time period or involving the same source, destination, or attack behavior. This approach enables security teams to detect complex or multi-stage attacks, reduce duplicate alerts, and improve the overall accuracy of threat detection.

One of the major advantages of event correlation is the reduction of false positive alerts. Individual security devices may generate numerous alerts that are not necessarily malicious; however, correlating events from multiple sources increases confidence in identifying genuine attacks. This process reduces unnecessary investigations, minimizes analyst workload, and enables Security Operations Centers (SOCs) to focus on high-priority incidents requiring immediate attention.

In the proposed project, security events generated by ModSecurity and pfSense Firewall are analyzed and correlated using custom correlation rules implemented within the Wazuh platform. These correlated events are then evaluated by the rule-based Expert Decision Support System, which determines whether the detected activity represents a real cyberattack before recommending or initiating automated firewall blocking. This approach improves detection accuracy, reduces false positive alerts, and accelerates the overall incident response process [18].

2.3.4 Alert Generation

Alert generation is the process by which a Security Information and Event Management (SIEM) platform identifies suspicious or malicious activities and notifies security analysts of potential security incidents. After security logs have been collected, normalized, and analyzed, the SIEM evaluates each event against predefined detection

rules and correlation policies. When an event or a group of correlated events satisfies specific detection criteria, the system automatically generates a security alert containing relevant information such as the event description, source and destination IP addresses, severity level, timestamp, and the rule responsible for triggering the alert.

The effectiveness of alert generation depends largely on the quality of detection and correlation rules. Well-designed rules help distinguish genuine attacks from normal system activities, reducing false positive alerts while ensuring that significant security incidents are not overlooked. Modern SIEM platforms also assign severity levels to alerts, allowing security analysts to prioritize incident investigation and response according to the potential impact of each detected threat.

Within the Wazuh platform, alerts are generated after events are processed by decoders and evaluated against built-in or custom detection rules. Each generated alert is assigned a unique identifier and severity level before being stored in the Wazuh Indexer and displayed on the Wazuh Dashboard. These alerts provide detailed information about detected security events and support further investigation, reporting, and incident response activities.

In the proposed project, the generated Wazuh alerts are forwarded to the rule-based Expert Decision Support System, where they are evaluated using predefined inference rules. Based on this evaluation, the system determines the appropriate response, documents the decision, and initiates automated firewall blocking through Active Response when malicious activity is confirmed [20].

2.3.5 Active Response

Active Response is an automated security mechanism that enables a Security Information and Event Management (SIEM) platform to execute predefined actions immediately after detecting malicious activities. Instead of relying solely on manual intervention by security analysts, Active Response automatically performs defensive actions based on detection rules and predefined security policies. These actions may include blocking malicious IP addresses, terminating suspicious processes, disabling compromised user accounts, isolating infected systems, or updating firewall rules to prevent further attacks.

Within the Wazuh platform, Active Response is triggered when a security event satisfies specific detection criteria defined by built-in or custom rules. After an alert is generated, Wazuh executes predefined response scripts either locally on monitored endpoints or remotely on integrated security devices. This automated approach significantly reduces the time required to contain cyber threats while minimizing the impact of ongoing attacks.

One of the primary advantages of Active Response is its ability to improve incident response efficiency and reduce the workload of security analysts. By automatically handling repetitive and well-defined security incidents, organizations can shorten response times, improve operational consistency, and minimize the risk of human error during incident handling. However, effective Active Response depends on accurate detection and carefully designed response rules to avoid unnecessary actions resulting from false positive alerts.

In the proposed project, Active Response is initiated only after the rule-based Expert Decision Support System evaluates the generated Wazuh alerts and confirms that the detected activity represents a genuine cyberattack. Once the decision is approved, the system automatically sends a blocking command to the pfSense Firewall, preventing

the attacker from accessing the protected web application. This approach combines automated incident response with explainable rule-based decision-making, improving both security effectiveness and response reliability [20].

2.4 pfSense Firewall

pfSense is an open-source firewall and routing platform based on the FreeBSD operating system. It provides comprehensive network security services by controlling incoming and outgoing network traffic according to predefined security policies. Due to its flexibility, reliability, and extensive security features, pfSense is widely deployed in enterprise networks, educational institutions, data centers, and small to medium-sized organizations to protect network infrastructures against unauthorized access and cyber threats.

The platform supports a wide range of network security functions, including packet filtering, Network Address Translation (NAT), Virtual Private Networks (VPNs), traffic shaping, intrusion prevention integration, and firewall rule management. Its web-based graphical interface simplifies firewall administration, allowing network administrators to configure security policies, monitor network activities, and manage firewall rules without requiring advanced command-line operations.

One of the major advantages of pfSense is its ability to generate detailed firewall logs that record network connections, blocked packets, and security events. These logs provide valuable information for detecting suspicious activities, investigating security incidents, and supporting centralized security monitoring. When integrated with Security Information and Event Management (SIEM) platforms, pfSense firewall logs contribute to improving attack detection accuracy through event correlation and comprehensive security analysis.

In the proposed project, pfSense acts as the primary network firewall positioned between the attacker and the protected web server. It monitors network traffic, generates firewall logs, and receives automated blocking commands from the rule-based Expert Decision Support System through Wazuh Active Response. This integration enables the system to automatically block malicious IP addresses after confirming attack activities, thereby reducing response time and strengthening network security [29].

2.4.1 Firewall Rules

Firewall rules are predefined security policies that determine whether network traffic should be permitted or denied based on specific criteria such as source and destination IP addresses, ports, protocols, and network interfaces. These rules represent the core mechanism of firewall operation, enabling organizations to control network communications and protect internal systems from unauthorized access. Firewall rules are evaluated sequentially, and the first matching rule determines the action that will be applied to the network packet.

Properly configured firewall rules help organizations reduce the attack surface by restricting unnecessary network access and allowing only legitimate communications. Administrators can define rules to permit trusted traffic, deny suspicious connections, or restrict access to specific services and network segments. This rule-based approach provides flexibility while ensuring that network security policies are consistently

enforced.

In the proposed project, firewall rules play a crucial role in the automated incident response process. After the rule-based Expert Decision Support System confirms that a detected event represents a malicious attack, a new blocking rule is automatically applied to the pfSense Firewall through Wazuh Active Response. This mechanism immediately prevents the attacker's IP address from communicating with the protected web server, thereby reducing the impact of the attack and minimizing response time [29].

2.4.2 Firewall Logging

Firewall logging is the process of recording network events and traffic activities that pass through the firewall. These logs contain valuable information, including timestamps, source and destination IP addresses, communication ports, protocols, rule identifiers, and the actions taken for each network connection. Firewall logs provide security analysts with detailed visibility into network activities and play an essential role in incident investigation, forensic analysis, and security monitoring.

Centralized analysis of firewall logs enables organizations to identify unauthorized access attempts, reconnaissance activities, repeated connection failures, and other suspicious behaviors. When integrated with Security Information and Event Management (SIEM) platforms, firewall logs can be correlated with events generated by other security devices, improving attack detection accuracy and reducing false positive alerts.

In the proposed project, pfSense Firewall continuously generates firewall logs describing both permitted and blocked network connections. These logs are forwarded to the Wazuh SIEM platform, where they are correlated with ModSecurity alerts using custom correlation rules. The resulting security alerts are then evaluated by the rule-based Expert Decision Support System before initiating automated firewall blocking through Active Response [29].



Figure 3 Types of Firewalls

2.5 Rule-Based Expert Decision Support System

2.5.1 Decision Support Systems (DSS)

A Decision Support System (DSS) is a computer-based information system designed to assist decision-makers in analyzing complex problems and selecting appropriate actions based on available information. Rather than replacing human expertise, a DSS supports the decision-making process by organizing data, applying predefined decision logic, and presenting recommendations that help improve the quality, consistency, and speed of decisions. Decision Support Systems are widely used in domains such as healthcare, finance, manufacturing, transportation, and cybersecurity, where rapid and accurate decisions are essential.

A typical Decision Support System consists of three main components: a data management component that provides the required information, a knowledge or rule base containing the decision logic, and an inference mechanism that evaluates available information against predefined rules to produce recommendations. Depending on the application domain, a DSS may rely on mathematical models, statistical analysis, optimization techniques, or rule-based reasoning to support decision-making.

In cybersecurity, Decision Support Systems assist security analysts by evaluating security events, prioritizing incidents, reducing false positive alerts, and recommending appropriate response actions. By analyzing multiple security indicators simultaneously, a DSS improves situational awareness and enables analysts to respond more efficiently to security incidents while maintaining consistent decision-making procedures. In the proposed project, the Decision Support System receives security alerts generated by the Wazuh SIEM platform after event correlation. It evaluates these alerts according to predefined cybersecurity rules, determines the appropriate response based on the detected threat level, documents the reasoning behind each decision, and recommends or initiates automated firewall blocking when necessary. This approach improves decision consistency, reduces response time, and minimizes unnecessary security actions [22].

2.5.2 Rule-Based Expert Systems

A Rule-Based Expert System is a type of artificial intelligence system that emulates the reasoning process of human experts by applying predefined knowledge in the form of logical rules. Instead of learning from historical data through machine learning algorithms, a rule-based expert system relies on explicit IF-THEN rules created by domain experts to analyze input information and produce consistent, explainable decisions. This approach makes rule-based systems particularly suitable for applications where transparency, reliability, and deterministic behavior are essential.

A rule-based expert system consists of three fundamental components: a Knowledge Base, which stores expert knowledge as predefined rules; an Inference Engine, which evaluates input data against these rules; and a Working Memory, which temporarily stores the facts and information required during the reasoning process. By systematically applying logical inference, the system determines the most appropriate

decision based on the available evidence while providing a clear explanation for the selected action.

Rule-based expert systems are widely applied in cybersecurity because security policies and incident response procedures are commonly defined through deterministic rules. They are used to classify security events, prioritize alerts, recommend mitigation actions, and automate repetitive decision-making tasks while maintaining full explainability. Unlike data-driven approaches, rule-based systems do not require training datasets, making them easier to validate, maintain, and adapt to organizational security policies.

The proposed project implements a Rule-Based Expert Decision Support System that evaluates Wazuh security alerts using predefined cybersecurity rules. Based on the alert severity, attack type, event correlation, and whitelist verification, the system determines the appropriate response, records the justification for each decision, and initiates automated firewall blocking only when the defined security conditions are satisfied [30].

2.5.3 Knowledge Base

The Knowledge Base is the core component of a rule-based expert system, as it stores the expert knowledge required to support decision-making. Instead of relying on historical datasets or statistical models, the knowledge base represents domain expertise using a collection of predefined rules, facts, and decision criteria. These rules are developed by domain experts according to established standards, best practices, and organizational security policies, enabling the system to make consistent and explainable decisions.

In a cybersecurity expert system, the knowledge base typically contains rules describing different attack types, threat severity levels, security policies, response procedures, and trusted entities. During the decision-making process, these rules are evaluated to determine the most appropriate action for each detected security event. Because the knowledge is explicitly represented, administrators can easily modify, update, or extend the rules to address newly discovered threats or changes in organizational security requirements.

In the proposed project, the Knowledge Base consists of predefined cybersecurity rules implemented within the Decision Support System. These rules evaluate multiple security indicators, including attack type, Wazuh rule identifiers, alert severity level, event correlation status, and whitelist verification. Based on these conditions, the system classifies each security event into predefined threat levels and determines the corresponding response action. This rule-based representation ensures transparent, consistent, and explainable security decisions while allowing future expansion by adding new cybersecurity rules without modifying the overall system architecture [30].

2.5.4 Inference Engine

The Inference Engine is the reasoning component of a rule-based expert system. Its primary function is to evaluate input data against the rules stored in the Knowledge Base and determine the appropriate decision based on logical reasoning. Rather than relying on statistical analysis or machine learning models, the inference engine systematically applies predefined IF–THEN rules to the available facts, ensuring that every decision follows the same logical process and remains transparent and

reproducible.

The inference process begins by receiving information from external sources, such as security alerts generated by a SIEM platform. The inference engine extracts relevant attributes from each alert, including the attack type, severity level, source IP address, rule identifier, and other contextual information. It then compares these attributes with the predefined rules stored in the Knowledge Base. When a rule's conditions are satisfied, the corresponding action and justification are produced as the final decision. One of the main advantages of a rule-based inference engine is its explainability. Since every decision is derived from explicit logical rules, the system can provide a clear justification describing why a particular action was selected. This capability improves analyst confidence, simplifies security auditing, and facilitates future modification of security policies without changing the overall system architecture.

In the proposed project, the Inference Engine receives security alerts generated by Wazuh SIEM after log analysis and event correlation. It evaluates multiple decision criteria, including attack severity, attack type, event correlation status, and whitelist verification, before assigning an appropriate threat level and response action. The generated decision includes both the recommended action and its scientific justification, enabling transparent decision-making prior to initiating automated firewall blocking through pfSense [30].

2.5.5 Decision Rules (IF–THEN Rules)

Decision Rules are the logical statements that define how a rule-based expert system evaluates input information and determines the appropriate response. These rules are commonly represented using the IF–THEN structure, where a specific condition (IF) is evaluated and, if satisfied, a corresponding action (THEN) is executed. This approach enables the system to apply expert knowledge consistently and transparently while ensuring that identical situations always produce identical decisions.

In cybersecurity applications, decision rules are used to classify security events according to predefined criteria such as attack type, severity level, source identity, event correlation, and organizational security policies. Each rule represents a specific security scenario together with its corresponding response. As new threats emerge or security requirements change, additional rules can be incorporated into the Knowledge Base without modifying the overall architecture of the expert system.

The effectiveness of a rule-based decision system depends on the quality and organization of its decision rules. Well-designed rules reduce ambiguity, improve decision consistency, and minimize false positive alerts by evaluating multiple security indicators instead of relying on a single event. Furthermore, assigning different response actions to different threat levels enables the system to apply proportional security measures while avoiding unnecessary disruption to legitimate users. In the proposed project, the Decision Support System implements a set of predefined IF–THEN rules that evaluate several security indicators, including whitelist verification, attack type, Wazuh rule identifiers, alert severity, and event correlation status. According to these conditions, the system assigns each detected event to one of several predefined decision levels, including Informational, Medium Threat, High Threat, and Critical Threat. Based on the resulting classification, the system recommends monitoring, event logging, administrative review, or immediate automated firewall blocking, while providing a scientific justification for every generated decision [30].

2.5.6 Decision Documentation

Decision documentation is an essential feature of modern Decision Support Systems because it provides a complete record of every decision generated by the system together with the reasoning behind it. Recording decision outcomes enables security analysts to review previous incidents, verify that organizational security policies have been correctly applied, and perform post-incident investigations. Furthermore, maintaining documented decisions improves system transparency, facilitates security auditing, and supports continuous refinement of security rules based on operational experience.

In cybersecurity environments, decision documentation also contributes to accountability by preserving important information such as timestamps, detected attack types, source IP addresses, applied security rules, selected response actions, and the justification for each decision. This information enables analysts to reconstruct security incidents, evaluate response effectiveness, and demonstrate compliance with organizational security policies and cybersecurity standards.

In the proposed project, every decision generated by the Rule-Based Expert Decision Support System is automatically documented in a dedicated log file. The recorded information includes the timestamp, attacker IP address, Wazuh rule identifier, detected attack type, assigned decision level, recommended response action, and the scientific justification used to reach the decision. This documented information supports security auditing, incident investigation, dashboard visualization, and future evaluation of the decision-making process [30].

2.5.7 False Positive Reduction

False positive alerts occur when legitimate activities are incorrectly classified as malicious events. Excessive false positives increase the workload of security analysts, consume valuable investigation time, and may reduce confidence in automated security systems. Consequently, minimizing false positive alerts is considered one of the primary objectives of Security Information and Event Management (SIEM) platforms and Decision Support Systems.

Rule-based expert systems reduce false positives by evaluating multiple security indicators instead of relying on a single event. Combining factors such as attack severity, event correlation, trusted source verification, and predefined security policies enables the system to distinguish genuine attacks from normal operational activities more accurately. This multi-criteria evaluation improves decision quality while reducing unnecessary automated responses that could disrupt legitimate users. In the proposed project, false positive reduction is achieved through several complementary mechanisms. The Decision Support System first verifies whether the source IP address belongs to a predefined whitelist before applying any defensive action. It then evaluates the attack severity, Wazuh rule identifier, event correlation status, and attack type before determining the appropriate response. Instead of automatically blocking every detected event, the system classifies alerts into multiple decision levels ranging from informational monitoring to immediate automated firewall blocking. This layered decision-making approach improves detection accuracy, reduces unnecessary firewall blocks, and supports more reliable cybersecurity operations [30].

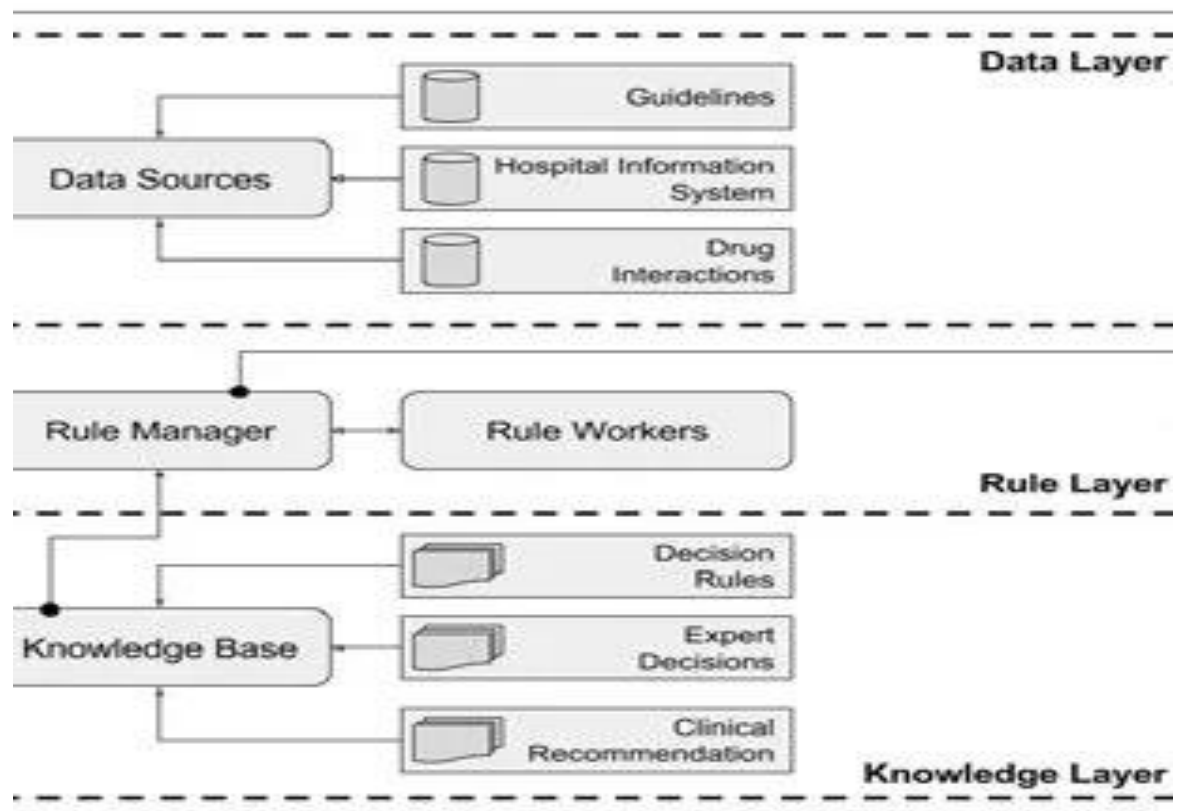


Figure 4 Decision Support Systems Achitecture

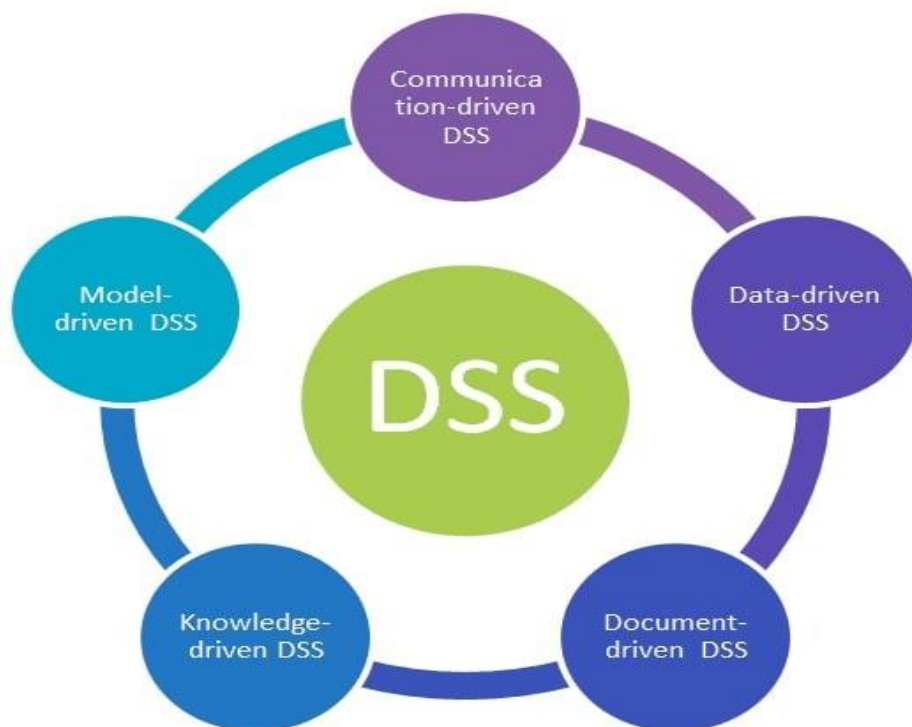


Figure5 DSS Types

Chapter 3

Work Environment Setup

3.1 Chapter Overview

This chapter presents the practical environment used to develop and implement the proposed cybersecurity system. It describes the hardware and software platforms, virtualization environment, network topology, and security components employed throughout the project. In addition, the chapter explains the rationale behind selecting each tool and compares it with alternative solutions to justify the design decisions adopted during system implementation.

The proposed environment was designed to simulate a realistic enterprise network capable of detecting, analyzing, and responding to web application attacks. The implementation integrates several open-source security technologies, including pfSense Firewall, ModSecurity Web Application Firewall, Apache Web Server, DVWA, Wazuh SIEM, and a Rule-Based Expert Decision Support System developed in Python. These components operate together to provide centralized log collection, event correlation, intelligent decision support, and automated incident response.

To ensure flexibility, isolation, and repeatability during testing, all system components were deployed as virtual machines within a virtualization platform. This approach enables different operating systems and security services to communicate through a controlled virtual network while accurately simulating real-world attack scenarios. The environment also simplifies system configuration, testing, troubleshooting, and future expansion without requiring dedicated physical hardware.

The following sections describe the hardware specifications, software environment, virtualization platform, network topology, system components, configuration procedures, and the reasons for selecting each technology over alternative solutions. These descriptions provide the technical foundation required for implementing and evaluating the proposed cybersecurity framework.

3.2 Development Environment

The development environment represents the infrastructure used to design, implement, configure, and evaluate the proposed cybersecurity framework. Since the project integrates multiple security technologies that operate on different operating systems, a virtualized environment was selected to provide flexibility, isolation, and efficient resource utilization. The complete implementation consists of several virtual machines that communicate through a controlled virtual network, allowing realistic simulation of cyberattacks while maintaining a safe testing environment.

The development environment includes both hardware and software resources required to support the deployment of the proposed architecture. The hardware platform provides the computational resources needed to execute multiple virtual machines simultaneously, while the software environment consists of the operating systems, security tools, web services, and development utilities used throughout the implementation. This configuration enables centralized monitoring, event analysis, automated decision support, and active response within a single integrated cybersecurity platform.

The following subsections describe the hardware specifications, software components, virtualization platform, and network topology adopted during the implementation of the proposed system.

3.2.1 Hardware Environment

The proposed cybersecurity environment was implemented on a personal workstation with sufficient computational resources to support multiple virtual machines operating simultaneously. Since the project integrates several security components, including a SIEM platform, firewall, web server, web application firewall, and an expert decision support system, adequate processing power and memory were required to ensure stable performance during attack simulation, log analysis, and automated response. The selected hardware platform successfully provided the necessary computational capacity throughout the implementation and testing phases.

Table 3 Hardware specification

Component	Specification
Processor	Intel® Core™ i7-11800H (11th Generation, 2.30 GHz)
Memory (RAM)	16 GB DDR4
Storage	954 GB SSD
Graphics Processing Unit (GPU)	Graphics Processing Unit (GPU) NVIDIA Graphics Card – 6 GB VRAM
Host Operating System	Windows 11 Pro (64-bit)
Virtualization Platform	Oracle VirtualBox

The selected hardware configuration provides sufficient computing resources to execute multiple virtual machines concurrently while maintaining acceptable system performance. This capability is essential because the proposed cybersecurity framework relies on several interconnected security services operating simultaneously, including Wazuh SIEM, ModSecurity WAF, pfSense Firewall, Apache Web Server, and the Rule-Based Expert Decision Support System.

About

The screenshot displays the Windows 'About' page for a desktop PC. At the top, four summary cards provide key hardware details: Installed RAM (16.0 GB, Speed: 3200 MHz), Processor (11th Gen Intel(R) Core(TM) i7-11800H @ 2.30GHz, 2.30 GHz), Graphics Card (6 GB, Multiple GPUs installed), and Storage (954 GB, 125 GB of 954 GB used). Below these, the PC name 'DESKTOP-PKV27KI' and model 'ASUS TUF Gaming F15 FX506HM_FX506HM' are shown, along with a 'Rename this PC' button. The 'Device Specifications' section, which can be expanded or copied, lists detailed information for each component: Device Name (DESKTOP-PKV27KI), Processor (11th Gen Intel(R) Core(TM) i7-11800H @ 2.30GHz, 2.30 GHz), Installed RAM (16.0 GB (15.7 GB usable)), Graphics Card (NVIDIA GeForce RTX 3060 Laptop GPU (6 GB), Intel(R) UHD Graphics (128 MB)), Storage (954 GB SSD SAMSUNG MZVLQ1T0HBLB-00B00), Device ID (F14ACB10-3FF3-4402-B81E-9568142B695D), Product ID (00331-10000-00001-AA147), System Type (64-bit operating system, x64-based processor), and Pen and touch (No pen or touch input is available for this display).

Component	Specification
Installed RAM	16.0 GB (Speed: 3200 MHz)
Processor	11th Gen Intel(R) Core(TM) i7-11800H @ 2.30GHz (2.30 GHz)
Graphics Card	6 GB (Multiple GPUs installed)
Storage	954 GB (125 GB of 954 GB used)

Device Name	DESKTOP-PKV27KI
Processor	11th Gen Intel(R) Core(TM) i7-11800H @ 2.30GHz 2.30 GHz
Installed RAM	16.0 GB (15.7 GB usable)
Graphics Card	NVIDIA GeForce RTX 3060 Laptop GPU (6 GB), Intel(R) UHD Graphics (128 MB)
Storage	954 GB SSD SAMSUNG MZVLQ1T0HBLB-00B00
Device ID	F14ACB10-3FF3-4402-B81E-9568142B695D
Product ID	00331-10000-00001-AA147
System Type	64-bit operating system, x64-based processor
Pen and touch	No pen or touch input is available for this display

Figure 6 Host Machine Hardware Specifications

3.2.2 Virtualization Environment

The proposed cybersecurity framework was deployed using Oracle VirtualBox, an open-source virtualization platform that enables multiple virtual machines to operate simultaneously on a single physical computer. Virtualization provides an isolated and controlled environment where different operating systems and security components can communicate through virtual networks while remaining independent from the host operating system.

Oracle VirtualBox was selected because it offers a free and reliable virtualization solution with comprehensive networking capabilities, including NAT, Bridged Adapter, Host-Only Adapter, and Internal Network modes. These networking options are essential for simulating realistic enterprise environments, allowing the attacker machine, firewall, web server, SIEM platform, and other security components to exchange traffic under controlled conditions. In addition, VirtualBox supports snapshots, making it possible to restore virtual machines quickly after testing cyberattack scenarios.

Compared with commercial virtualization solutions, VirtualBox provides sufficient performance and functionality for cybersecurity research and educational projects without requiring additional licensing costs. This makes it a practical choice for developing and evaluating the proposed integrated cybersecurity framework.

Table 4 Comparison of Virtualization Platforms

Feature	Oracle VirtualBox	VMware Workstation Pro
License	Open Source	Commercial
Cost	Free	Paid
Multiple Virtual Machines	✓	✓
Suitable for the Proposed System	✓	✓

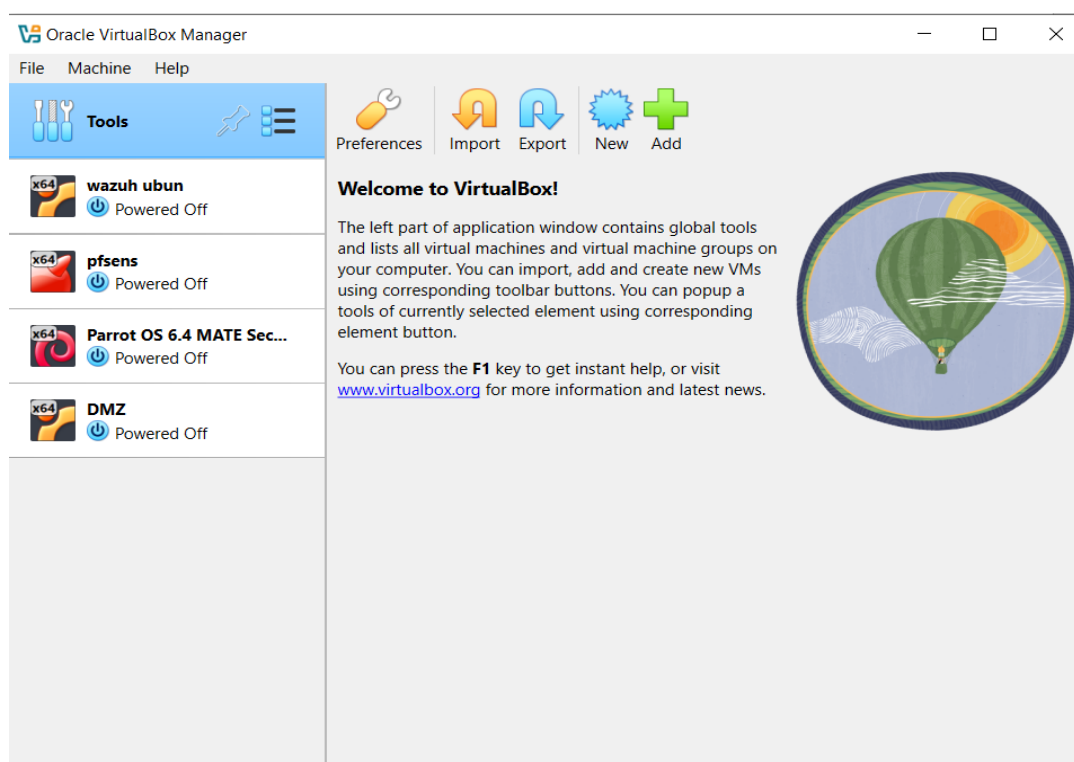


Figure 7 VirtualBox Environment

3.2.3 Software Environment

The proposed cybersecurity framework relies on several software components that operate together to provide web application hosting, security monitoring, attack detection, decision support, and automated response. Each software component was selected according to technical criteria such as compatibility, stability, community support, ease of integration, and suitability for cybersecurity research. The following subsections describe the primary software components used during the implementation of the proposed system, together with the reasons for selecting each tool over available alternatives.

3.2.3.1 Ubuntu Server

Ubuntu Server is a Linux-based operating system designed for server environments and enterprise applications. It provides a stable, secure, and efficient platform for hosting web services, databases, and cybersecurity applications. Owing to its lightweight architecture, long-term support (LTS) releases, and extensive software repositories, Ubuntu Server has become one of the most widely adopted operating systems for deploying web servers and security infrastructures.

In the proposed project, Ubuntu Server was selected as the operating system for hosting the Apache Web Server, DVWA, ModSecurity Web Application Firewall, Wazuh Agent, and the Rule-Based Expert Decision Support System. Its compatibility with these security components simplified system integration and ensured reliable operation throughout the implementation process. Furthermore, the availability of comprehensive documentation and strong community support facilitated installation, configuration, troubleshooting, and future maintenance.

Compared with alternative Linux server distributions such as Debian and CentOS, Ubuntu Server provides an excellent balance between stability, software availability, hardware compatibility, and ease of administration. In addition, the Long-Term Support (LTS) releases receive security updates and maintenance for extended periods, making Ubuntu Server particularly suitable for cybersecurity environments that require reliable and continuously supported platforms.

Table 5 Comparison of Linux Server OS

Feature	Ubuntu	Debian	CentOS
Community support	High	Medium	Normal
Ease of Configuration	High	Medium	Medium
Compatibility with Security Tools	High	High	Medium

Reason for Selection

Ubuntu Server was selected because it provides excellent compatibility with Apache, ModSecurity, Python, and Wazuh Agent while offering long-term security updates, extensive documentation, and straight forward administration. These characteristics make it a reliable operating system for implementing the proposed cybersecurity framework.



Figure 8 Ubuntu Home Screen

3.2.3.2 Apache HTTP Server

Apache HTTP Server is one of the most widely used open-source web server software solutions for hosting websites and web applications. It is developed and maintained by the Apache Software Foundation and supports multiple operating systems, including Linux, Windows, and macOS. Apache is recognized for its stability, flexibility, extensive module support, and compatibility with numerous web technologies, making it a popular choice for both academic research and enterprise environments.

In the proposed project, Apache HTTP Server was selected to host the Damn Vulnerable Web Application (DVWA), providing the web service through which different cyberattacks are launched and analyzed. Apache also serves as the integration point for ModSecurity Web Application Firewall, enabling HTTP requests to be inspected before reaching the hosted web application. This integration allows the system to detect and prevent malicious requests while simultaneously generating security logs that are forwarded to the Wazuh SIEM platform for centralized monitoring and analysis.

Compared with alternative web servers such as Nginx and Microsoft Internet Information Services (IIS), Apache offers greater flexibility through its modular architecture and native compatibility with ModSecurity. Since the proposed framework relies heavily on ModSecurity to inspect web traffic and generate security events, Apache provides a more suitable deployment environment while maintaining high reliability and ease of configuration. Furthermore, Apache benefits from extensive

documentation and a large open-source community, simplifying deployment, maintenance, and troubleshooting [31].

Table 6 Comparison of Web Servers

Feature	Apache HTTP Server	Nginx	Microsoft IIS
Open Source	✓	✓	X
Native ModSecurity Support	X	Limited	X
Cross-Platform Support	✓	✓	Limited

Reason for Selection

Apache HTTP Server was selected because it provides seamless integration with ModSecurity, stable web hosting capabilities, extensive module support, and straightforward configuration. These features make it an ideal platform for hosting DVWA and generating the security events required by the proposed cybersecurity framework.



Figure 9 Apache Web Page

3.2.3.3 Damn Vulnerable Web Application (DVWA)

Damn Vulnerable Web Application (DVWA) is an open-source PHP/MySQL web application intentionally designed with multiple security vulnerabilities for educational purposes, cybersecurity training, and penetration testing. It provides a safe environment in which researchers and security professionals can study common web application attacks without affecting production systems. DVWA includes a wide range of intentionally vulnerable modules that simulate real-world security flaws, making it one of the most widely used platforms for evaluating web application security mechanisms. In the proposed project, DVWA was deployed on the Apache web server to serve as

the target application for various cyberattacks, including SQL Injection, Cross-Site Scripting (XSS), Command Injection, File Inclusion, Brute Force, and other OWASP Top 10 attack scenarios. These attacks generate realistic HTTP requests that are inspected by ModSecurity, recorded by Wazuh, analyzed by the Rule-Based Expert Decision Support System, and, when necessary, mitigated through automated firewall blocking. Consequently, DVWA plays a central role in evaluating the effectiveness of the proposed cybersecurity framework.

Compared with other vulnerable web application platforms such as OWASP Juice Shop and WebGoat, DVWA provides a lightweight architecture, straightforward deployment, and excellent compatibility with Apache and ModSecurity. Moreover, it offers multiple security levels ranging from Low to Impossible, allowing different attack scenarios to be evaluated under controlled conditions. These characteristics make DVWA particularly suitable for validating the detection, correlation, and automated response capabilities of the proposed system [32].

Table 7 Comparison of Vulnerable Web Applications

Feature	DVWA	OWASP Juice Shop	.WebGoat
Open source	✓	✓	✓
Easy Deployment	Easy	Hard	Hard
Multiple Security Levels	✓	×	Limited
Compatible with Apache & ModSecurity	✓	Partial	partial

Reason for Selection

DVWA was selected because it provides a controlled and reproducible environment for launching realistic web application attacks while maintaining compatibility with Apache HTTP Server and ModSecurity. Its multiple security levels, broad coverage of OWASP attack scenarios, and ease of deployment make it an ideal platform for evaluating the proposed cybersecurity framework.



Figure 10 DVWA Deployed on Apache

3.2.3.4 ModSecurity Web Application Firewall

ModSecurity is an open-source Web Application Firewall (WAF) designed to monitor, filter, and protect web applications by inspecting HTTP requests and responses before they reach the web server. It operates as a module integrated with web servers such as Apache and Nginx, enabling real-time analysis of web traffic based on predefined security rules. ModSecurity is capable of detecting and preventing a wide range of web application attacks, including SQL Injection, Cross-Site Scripting (XSS), Command Injection, File Inclusion, Directory Traversal, and other vulnerabilities identified in the OWASP Top 10.

In the proposed cybersecurity framework, ModSecurity was integrated with the Apache HTTP Server to serve as the first layer of defense against malicious web requests. Every incoming HTTP request is inspected using the OWASP Core Rule Set (CRS) before reaching the hosted DVWA application. When an attack is detected, ModSecurity generates detailed security logs containing information such as the attack type, source IP address, triggered security rule, severity level, and request details. These logs are subsequently collected by the Wazuh Agent and forwarded to the Wazuh SIEM platform for centralized monitoring, event correlation, and further analysis by the Rule-Based Expert Decision Support System.

Compared with alternative web application firewalls such as NAXSI and commercial cloud-based WAF solutions, ModSecurity offers greater flexibility, extensive customization capabilities, and native compatibility with Apache HTTP Server. In addition, its seamless integration with the OWASP Core Rule Set enables continuous protection against emerging web application threats while allowing administrators to customize detection rules according to organizational security requirements. These characteristics make ModSecurity particularly suitable for research environments that require detailed attack analysis and integration with SIEM platforms [33].

Table 8 Comparison of Web Application Firewalls

Feature	ModSecurity	NAXSI	Cloudflare WAF
Open Source	✓	✓	✗
Apache Integration	✓	Limited	✗
OWASP CRS Support	✓	✓	✓

Reason for Selection

ModSecurity was selected because it integrates seamlessly with Apache HTTP Server, supports the OWASP Core Rule Set, generates detailed security logs for SIEM analysis, and allows extensive customization of detection rules. These capabilities make it an ideal web application firewall for detecting attacks, supporting centralized monitoring, and enabling automated incident response within the proposed cybersecurity framework.

```
GNU nano 4.8 /etc/modsecurity/modsecurity.conf Modified
-- Rule engine initialization -----
Enable ModSecurity, attaching it to every transaction. Use detection
only to start with, because that minimises the chances of post-installation
disruption.

SecRuleEngine On

-- Request body handling -----
Allow ModSecurity to access request bodies. If you don't, ModSecurity
won't be able to see any POST parameters, which opens a large security
hole for attackers to exploit.

SecRequestBodyAccess On

Enable XML request body parser.
Initiate XML Processor in case of xml content-type

SecRule REQUEST_HEADERS:Content-Type "(?:application(?:/soap\+|/)|text/)xml" \
    "id:'200000',phase:1,t:none,t:lowercase,pass,nolog,ctl:requestBodyProcess>
Enable JSON request body parser.
```

Figure 11 ModSecurity Integrated with Apache and Configured using OWASP CRS

3.2.3.5 Wazuh Security Information and Event Management (SIEM)

Wazuh is an open-source Security Information and Event Management (SIEM) platform that provides centralized security monitoring, log management, threat detection, file integrity monitoring, vulnerability assessment, and automated incident response. It enables organizations to collect and analyze security events from multiple data sources through a centralized architecture, allowing security analysts to monitor network activities and detect malicious behavior in real time.

In the proposed cybersecurity framework, Wazuh serves as the central monitoring platform responsible for collecting security logs generated by ModSecurity, the Apache Web Server, and the operating system through the Wazuh Agent. After receiving these logs, Wazuh applies predefined detection rules and custom correlation rules to identify attack patterns and generate security alerts. These alerts are then forwarded to the Rule-Based Expert Decision Support System, which evaluates their severity and recommends appropriate response actions. For critical attacks, the decision support system automatically initiates Active Response by sending a command to the pfSense firewall to block the attacker's IP address.

Compared with alternative SIEM platforms such as Splunk Enterprise Security and the Elastic Stack (ELK), Wazuh offers a fully open-source solution with integrated security capabilities, including endpoint monitoring, log analysis, event correlation, and Active Response. Its native support for security agents, flexible rule customization, and seamless integration with ModSecurity make it particularly suitable for implementing

the proposed cybersecurity framework while avoiding the licensing costs associated with commercial SIEM platforms [3].

Table 9 Comparison of SIEM Platforms

Feature	Wazuh	Splunk	Elastic SIEM
Open Source	✓	✗	✓
Event Correlation	✓	✓	✓
Active Response	✓	Limited	Requires Additional configurations
Licensing Cost	Free	Commercial	Free

Reason for Selection

Wazuh was selected because it provides centralized log collection, customizable detection rules, event correlation, and built-in Active Response capabilities within a single open-source platform. Furthermore, its seamless integration with ModSecurity and Python-based automation simplifies the implementation of the proposed cybersecurity framework while providing comprehensive visibility into security events.

3.2.3.6 Wazuh Agent

The Wazuh Agent is a lightweight software component installed on monitored endpoints to collect security-related information and transmit it securely to the Wazuh Manager. It continuously monitors operating system logs, application logs, security events, file integrity changes, and system activities, enabling centralized security monitoring across multiple devices. The agent performs local event collection while minimizing system resource consumption, making it suitable for continuous monitoring in both enterprise and research environments.

In the proposed cybersecurity framework, the Wazuh Agent was installed on the Ubuntu Server hosting Apache HTTP Server, DVWA, and ModSecurity. Its primary responsibility is to collect security logs generated by the operating system, Apache access and error logs, and ModSecurity alerts. After collecting these events, the agent securely forwards them to the Wazuh Manager, where they are processed using detection rules and custom correlation rules. This centralized log collection mechanism enables the proposed system to correlate security events from different sources and provide the Rule-Based Expert Decision Support System with the information required for threat assessment and automated response.

Compared with traditional log forwarding mechanisms such as Syslog and Filebeat, the Wazuh Agent provides native integration with the Wazuh SIEM platform while offering additional endpoint security capabilities, including file integrity monitoring, vulnerability detection, configuration assessment, and secure communication with the manager. These features reduce deployment complexity and improve the overall efficiency of the proposed cybersecurity architecture.

Reason for Selection

The Wazuh Agent was selected because it provides secure and centralized log collection while integrating seamlessly with the Wazuh SIEM platform. In addition to forwarding logs, it supports endpoint monitoring, file integrity monitoring, and vulnerability detection, making it an efficient solution for collecting the security events required by the proposed cybersecurity framework.

```
jad@DMZ:~$ sudo systemctl status wazuh-agent.service
[sudo] password for jad:
● wazuh-agent.service - Wazuh agent
   Loaded: loaded (/lib/systemd/system/wazuh-agent.service; enabled; vendor
   Active: active (running) since Thu 2026-07-16 12:13:57 +03; 45min ago
     Tasks: 24 (limit: 3470)
    Memory: 132.9M
    CGroup: /system.slice/wazuh-agent.service
            └─2013 /var/ossec/bin/wazuh-execd
              └─2024 /var/ossec/bin/wazuh-agentd
                └─2040 /var/ossec/bin/wazuh-logcollector
                  └─2054 /var/ossec/bin/wazuh-modulesd

12:13:52 16 يوليو DMZ env[1968]: Deleting PID file '/var/ossec/var/run/wazuh-l>
12:13:52 16 يوليو DMZ env[1968]: Deleting PID file '/var/ossec/var/run/wazuh-a>
12:13:52 16 يوليو DMZ env[1968]: Deleting PID file '/var/ossec/var/run/wazuh-e>
12:13:53 16 يوليو DMZ env[1968]: Started wazuh-execd...
12:13:54 16 يوليو DMZ env[1968]: Started wazuh-agentd...
12:13:54 16 يوليو DMZ env[1968]: wazuh-syscheckd already running...
12:13:54 16 يوليو DMZ env[1968]: Started wazuh-logcollector...
12:13:55 16 يوليو DMZ env[1968]: Started wazuh-modulesd...
12:13:57 16 يوليو DMZ env[1968]: Completed.
12:13:57 16 يوليو DMZ systemd[1]: Started Wazuh agent.
lines 1-21/21 (END)
```

Figure 12 Wazuh Agent Connected to Manager Successfully

3.2.3.7 pfSense Firewall

pfSense is an open-source firewall and routing platform based on the FreeBSD operating system. It provides advanced network security services, including packet filtering, stateful firewall inspection, Network Address Translation (NAT), Virtual Private Network (VPN) support, traffic monitoring, and access control. Due to its flexibility, stability, and extensive management capabilities, pfSense is widely adopted by educational institutions, research laboratories, and enterprise networks as a reliable perimeter security solution.

In the proposed cybersecurity framework, pfSense functions as the network perimeter firewall responsible for enforcing security policies and blocking malicious network traffic. Unlike conventional firewall deployments that rely solely on manually configured rules, pfSense is integrated with the Rule-Based Expert Decision Support System developed in Python. After Wazuh SIEM detects and correlates security events, the decision support system evaluates the severity of the detected attack. If the attack is classified as a critical threat, the system establishes a secure SSH connection to the pfSense firewall using the Paramiko Python library and automatically executes the EasyRule command to create a firewall rule that blocks the attacker's IP address. This automated workflow significantly reduces response time and minimizes the need for manual intervention during cyber incidents.

Compared with alternative firewall platforms such as OPNsense and IPFire, pfSense provides a mature and stable platform with extensive documentation, a large user community, and straightforward firewall rule management. Furthermore, its support for remote administration and command-line automation simplifies integration with external security applications, making it particularly suitable for implementing the automated incident response mechanism proposed in this project [29].

Table 10 Comparison of Firewall Platforms

Feature	PfSense	OPNsense	IPFire
Open Source	✓	✓	✓
SSH Administration	✓	✓	Limited
Rule Automation	✓	Moderate	Limited

Reason for Selection

pfSense was selected because it provides a stable and highly configurable firewall platform that supports secure remote administration and automated rule management. These capabilities enable seamless integration with the Rule-Based Expert Decision Support System, allowing malicious IP addresses to be blocked automatically after critical attacks are detected by Wazuh SIEM.

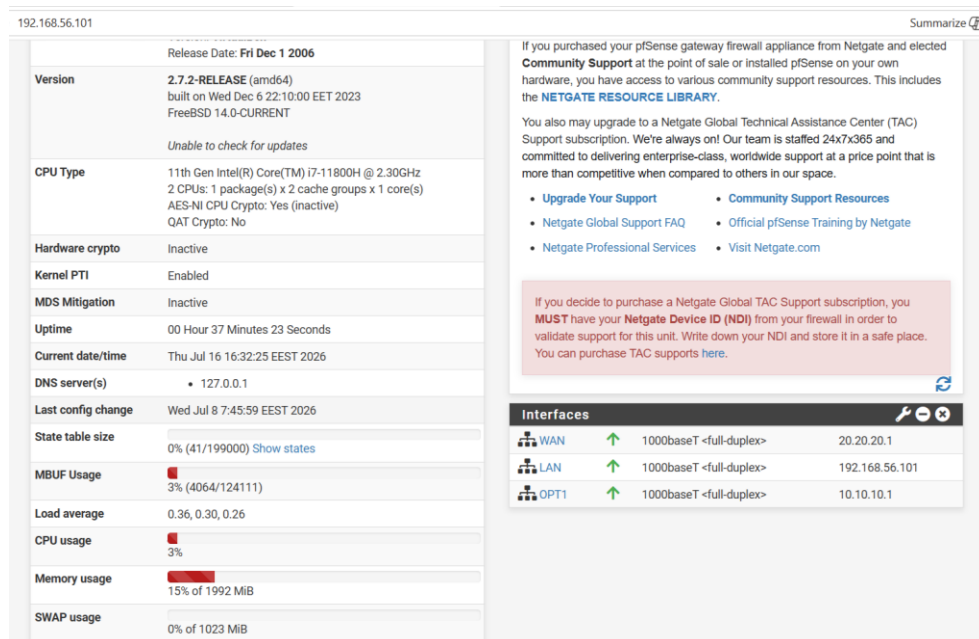


Figure 13 Pfsense Dashboard

3.2.3.8 Parrot Security Operating System

Parrot Security Operating System (Parrot OS) is an open-source Linux distribution specifically developed for cybersecurity professionals, penetration testers, digital forensics analysts, and security researchers. It is based on Debian Linux and includes a

comprehensive collection of security assessment tools that support vulnerability analysis, penetration testing, network reconnaissance, web application testing, and digital investigations. The operating system provides a stable and lightweight platform while maintaining compatibility with a wide range of offensive security tools. In the proposed cybersecurity framework, Parrot Security OS was used as the attacker machine to simulate realistic cyberattacks against the target web application. It was employed to execute various web application attacks, including SQL Injection, Cross-Site Scripting (XSS), Command Injection, Directory Traversal, and Brute Force attacks against DVWA. These attack scenarios generated realistic malicious traffic that was inspected by ModSecurity, logged by the Wazuh Agent, analyzed by Wazuh SIEM, evaluated by the Rule-Based Expert Decision Support System, and ultimately mitigated through automated firewall blocking on the pfSense firewall.

Compared with other penetration testing distributions such as Kali Linux and BlackArch Linux, Parrot Security OS provides a lightweight environment with lower hardware requirements while offering an extensive collection of penetration testing and forensic tools. Its efficient resource utilization and comprehensive security toolkit made it an appropriate choice for implementing repeated attack simulations throughout the experimental evaluation of the proposed cybersecurity framework [34].

Table 11 Comparison of Penetration Testing OS

Feature	Parrot Security	Kali Linux	BlackArch Linux
Open source	✓	✓	✓
Debian-Based	✓	✓	✗
Lightweight Performance	✓	Moderate	Moderate

Reason for Selection

Parrot Security OS was selected because it provides a lightweight penetration testing environment with a comprehensive collection of cybersecurity tools while requiring fewer system resources than many alternative security distributions. These characteristics allowed repeated execution of attack scenarios against the proposed cybersecurity framework without affecting the stability of the experimental environment.

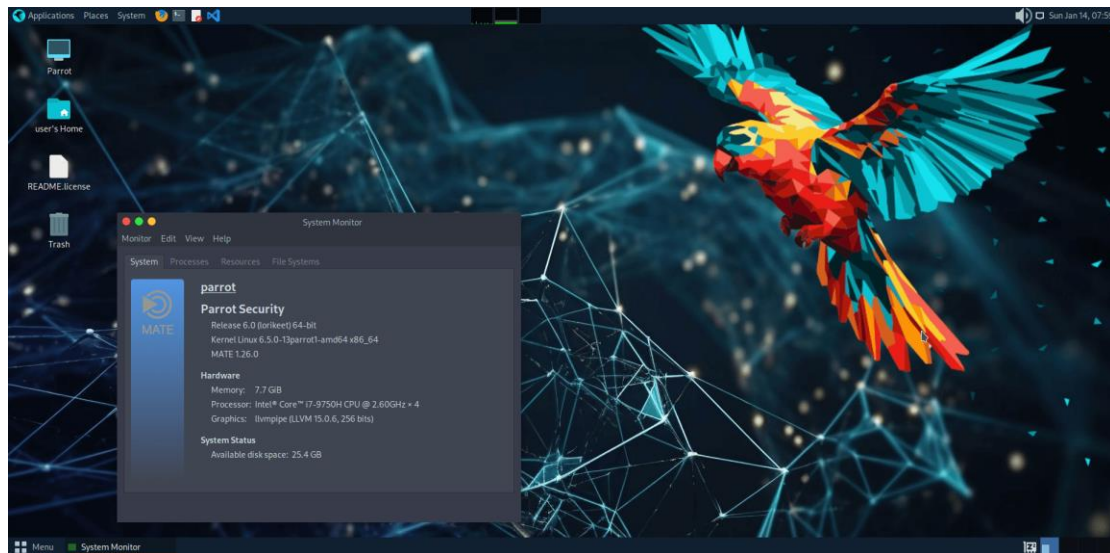


Figure 14 . Parrot Home Screen

3.2.3.9 Rule-Based Expert Decision Support System

The Rule-Based Expert Decision Support System (DSS) is a software component developed in Python to assist security analysts in evaluating security alerts generated by the Wazuh SIEM platform and recommending appropriate response actions. Unlike machine learning-based approaches that require large training datasets, the proposed system relies on predefined security rules and expert knowledge to perform deterministic and explainable decision-making. This approach ensures that every generated decision is supported by explicit security rules and well-defined reasoning, making the system suitable for cybersecurity environments where transparency and predictability are essential.

In the proposed cybersecurity framework, the Decision Support System continuously monitors the Wazuh alerts.json file for newly generated security alerts. Each alert is parsed to extract relevant information, including the Wazuh rule identifier, severity level, attack description, source IP address, and rule groups. Based on these attributes, the inference engine applies predefined decision rules to classify the detected event into one of several threat levels, including Informational, Medium Threat, High Threat, and Critical Threat. Each classification is associated with a corresponding response action, ranging from simple event logging and continuous monitoring to automated firewall blocking for critical attacks.

The implemented decision logic evaluates different attack scenarios according to predefined security policies. For example, attacks identified as Command Injection or correlated multi-vector attacks are immediately classified as critical threats and trigger automatic firewall blocking. Repetitive web application attacks are classified as high threats and require administrator approval before mitigation, while isolated low-severity events are logged for continuous monitoring without interrupting legitimate services. Furthermore, the system maintains a whitelist of trusted IP addresses to prevent false positive responses and preserve normal business operations. To support automated incident response, the Decision Support System establishes a secure SSH connection to the pfSense firewall using the Paramiko Python library whenever a critical threat is detected. It then executes the EasyRule command to

dynamically create a firewall rule that blocks the attacker's IP address. In addition, every generated decision is recorded in a dedicated JSON log file together with its corresponding justification, enabling security analysts to review the reasoning behind each response action and supporting future auditing and forensic investigations. Compared with intelligent assistants based on Large Language Models (LLMs), the proposed Rule-Based Expert Decision Support System provides deterministic behavior, transparent decision-making, and direct integration with the implemented Active Response mechanism. These characteristics ensure that the generated decisions remain fully explainable while supporting rapid and automated incident response within the proposed cybersecurity framework.

Reason for Selection

The Rule-Based Expert Decision Support System was selected because it provides transparent, deterministic, and easily maintainable decision-making without requiring model training or large datasets. Its rule-based inference engine integrates directly with Wazuh SIEM and the pfSense firewall, enabling rapid threat assessment and automated incident response while ensuring that every generated decision can be fully explained and audited.

Table 12 Comparison of Decision Support Approaches

Feature	Rule-Based DSS	LLM-Based Assistant	Machine Learning
Explainable Decision	✓	Limited	Limited
Dataset Required	✗	✓	✓
Deterministic Output	✓	✗	✗
Automatic Firewall Integration	✓	Limited	Requires Development
Easy Rule Modification	✓	✗	✗
Real-Time Response	✓	Moderate	Moderate

3.3 Network Topology

3.3.1 Network Architecture

The proposed cybersecurity framework was deployed within a virtualized environment using Oracle VirtualBox to simulate a realistic enterprise network. The architecture consists of multiple interconnected virtual machines, each performing a dedicated security function within the overall system. Separating the security components into independent virtual machines improves modularity, simplifies management, and enables realistic testing of cyberattack scenarios while maintaining isolation between system components.

The network architecture includes four primary machines. The Ubuntu Server hosts the Apache HTTP Server, the Damn Vulnerable Web Application (DVWA), the ModSecurity Web Application Firewall, the Wazuh Agent, and the Rule-Based Expert Decision Support System. The Wazuh Server functions as the centralized SIEM platform responsible for collecting security logs, correlating events, and generating alerts. The pfSense Firewall serves as the network perimeter defense by controlling network traffic and enforcing automated firewall rules. Finally, the Parrot Security OS acts as the attacker machine used to generate different web application attacks for evaluating the effectiveness of the proposed framework.

All virtual machines communicate through the internal virtual network provided by Oracle VirtualBox, allowing security events to be exchanged without exposing the experimental environment to external networks. This isolated architecture ensures repeatable testing conditions while accurately representing the interaction between enterprise security components.

Security Architecture with Automated Threat Response

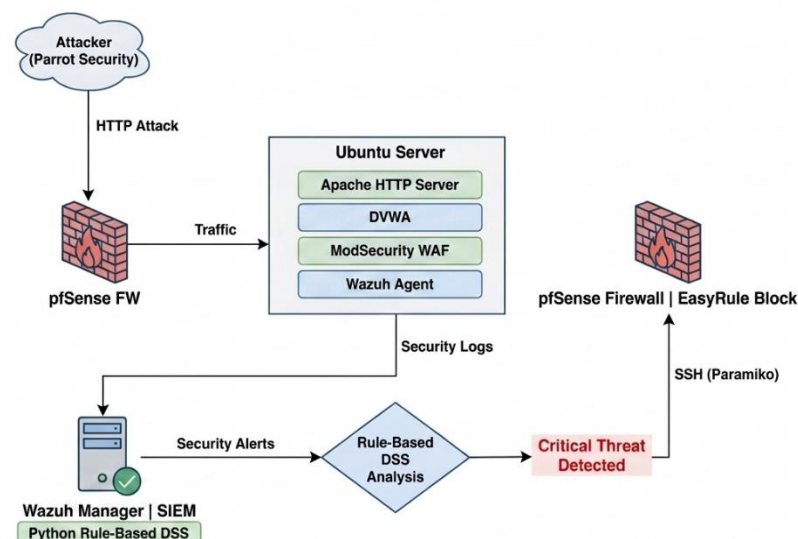


Figure 15 Overall Architecture of the Proposed Framework

3.3.2 Virtual Machines and IP Addressing

The proposed cybersecurity framework was implemented using multiple virtual machines connected through Oracle VirtualBox's virtual networking environment. Each virtual machine performs a dedicated role within the overall architecture, enabling realistic simulation of enterprise cybersecurity operations. Assigning separate operating systems and services to different virtual machines improves modularity, simplifies troubleshooting, and allows individual components to be configured and evaluated independently.

Each machine was assigned a static IP address to ensure reliable communication between the firewall, web server, SIEM platform, attacker machine, and decision

support system. Static addressing also simplifies log correlation, firewall rule creation, and automated response operations throughout the experimental environment.

Table 13 . Virtual Machines in Proposed Framework

Virtual Machine	Operating System	Main Components	IP Address
DMZ	Ubuntu Server 22.04 LTS	Apache, DVWA, ModSecurity, Wazuh agent	10.10.10.127
SIEM Server	Ubuntu Server 22.04 LTS	Manager, Indexer, Dashboard, python DSS	10.10.10.123
Pfsense Firewall	Pfsense CE	Firewall, EasyRule	20.20.20.1 (WAN) 10.10.10.1 (LAN) 192.168.56.1(OPT)
Attacker Machine	Parrot Security OS	Pen Testing Tools	20.20.20.10

Table 14 The Role of each Virtual Machine

Virtual Machine	Role in the Proposed System
DMZ	Hosts the web application, WAF, Wazuh Agent, Send Security Logs to the SIEM Server
SIEM Server	Collects logs, correlates security events, generates alerts, and forwards them for decision analysis.
Pfsense	Monitors network traffic and automatically blocks malicious IP addresses after receiving commands from the Decision Support System.
Attacker Machine	Simulates cyberattacks against the target web application for experimental evaluation.

3.3.3 Data Flow within the Proposed Framework

The proposed cybersecurity framework processes security events through a sequence of interconnected stages, beginning with attack generation and ending with automated threat mitigation. This workflow enables the integration of web application protection, centralized security monitoring, rule-based decision support, and automated firewall response within a unified security architecture.

The process begins when the attacker machine running Parrot Security OS sends malicious HTTP requests toward the target web application hosted on the Ubuntu Server. These requests may include SQL Injection, Cross-Site Scripting (XSS), Command Injection, or other web application attacks. Before the requests reach the

DVWA application, they are inspected by the ModSecurity Web Application Firewall, which compares the incoming traffic against the OWASP Core Rule Set (CRS). Whenever malicious activity is detected, ModSecurity records detailed security logs describing the detected attack.

The generated security logs are collected by the Wazuh Agent, which continuously monitors the web server and operating system logs. The agent securely forwards these events to the Wazuh Manager, where predefined detection rules and custom correlation rules are applied to identify attack patterns and generate security alerts. These alerts are subsequently processed by the Rule-Based Expert Decision Support System. The Rule-Based Expert Decision Support System continuously monitors newly generated Wazuh alerts and extracts relevant information such as the rule identifier, severity level, attack description, source IP address, and correlation groups. Based on predefined expert rules, the system classifies each event into different threat levels and determines the appropriate response action. Informational and low-risk events are recorded for monitoring purposes, while high-risk attacks are flagged for administrative review. Critical attacks, such as Command Injection or correlated multi-stage attacks, immediately trigger automated incident response.

For critical threats, the Decision Support System establishes a secure SSH connection with the pfSense Firewall using the Paramiko Python library and executes an EasyRule command to automatically create a firewall rule that blocks the attacker's IP address. At the same time, the generated decision, together with its scientific justification and recommended response, is stored in a dedicated JSON log file for auditing and forensic analysis. This automated workflow significantly reduces incident response time while maintaining transparent and explainable security decisions.

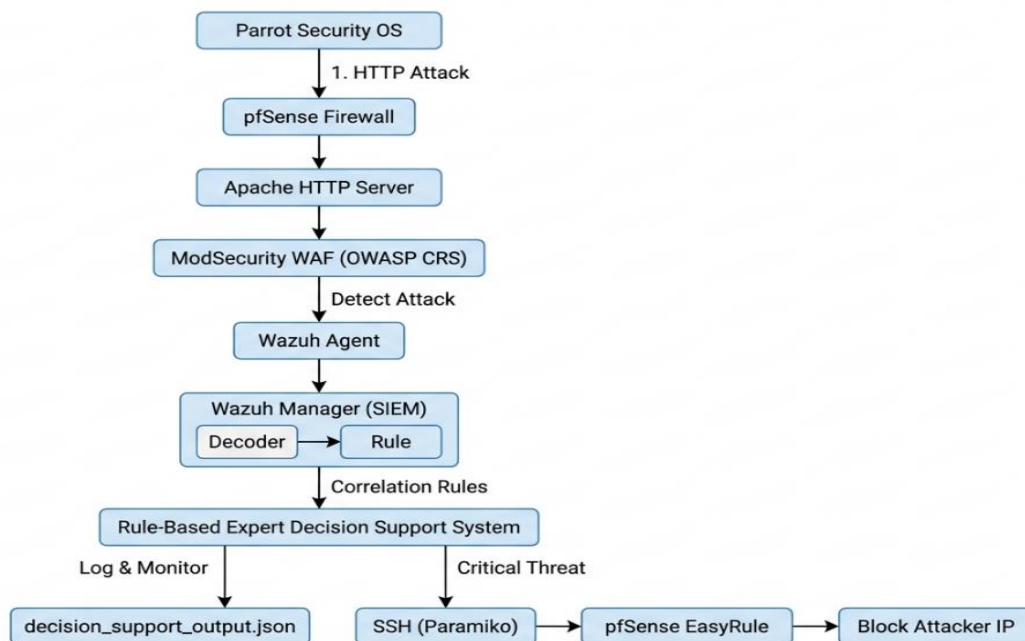


Figure 16 Data Flow of the Proposed System

3.4 System Configuration

3.4.1 Ubuntu Server Configuration

The Ubuntu Server virtual machine was configured as the primary host of the proposed cybersecurity framework. Ubuntu Server 20.04 LTS was selected due to its long-term support, stability, and compatibility with the security tools used throughout the project. Following the installation, the operating system was updated to ensure that all installed packages were compatible with the latest security updates.

A static IP address was assigned to the server to guarantee stable communication with the Wazuh Manager, pfSense firewall, and other virtual machines within the isolated virtual network. Remote administration was enabled through the OpenSSH service, allowing secure access to the server and enabling the Rule-Based Expert Decision Support System to communicate with the pfSense firewall using the Paramiko Python library. Since the pfSense firewall was responsible for perimeter protection, the Ubuntu UFW firewall remained disabled to avoid conflicts with the centralized firewall policy. The server was then prepared for hosting the cybersecurity framework by installing all required software packages, including Apache HTTP Server version 2.4.41, PHP version 7.4.3, MariaDB version 10.3.39, Python version 3.8.10, and Git. These components provided the execution environment required for the vulnerable web application, the web application firewall, the Wazuh Agent, and the Rule-Based Expert Decision Support System.

Table 15 Ubuntu Server Configuration

Configuration item	Value
Operating System	Ubuntu Server 20.04 LTS
Web Server	Apache HTTP Server 2.4.41
PHP Version	7.4.3
Database Server	MariaDB 10.3.39
Python Version	3.8.10
UFW Firewall	Inactive
Virtualization Platform	Oracle VirtualBox

3.4.2 Apache HTTP Server Configuration

Apache HTTP Server was configured as the primary web server responsible for hosting the Damn Vulnerable Web Application (DVWA) within the proposed cybersecurity framework. Apache version 2.4.41 was installed on the Ubuntu Server and configured to serve web content while supporting dynamic PHP applications. PHP version 7.4.3 was integrated with Apache to enable the execution of DVWA and provide the runtime environment required for simulating web application attacks.

Following the installation, the Apache service was configured to start automatically during system boot, ensuring continuous availability throughout the experimental evaluation. The web server document root was configured to host the DVWA

application, while the necessary file permissions were assigned to allow proper execution of PHP scripts and interaction with the MariaDB database. The successful deployment of Apache was verified by accessing the hosted web application through a web browser from both the attacker machine and other virtual machines within the isolated laboratory network.

Because Apache generates detailed access and error logs, it also serves as one of the primary data sources for security monitoring. These logs are continuously monitored by the Wazuh Agent and combined with ModSecurity alerts to support centralized event correlation and security analysis within the proposed framework.

Table 16 Apache Configuration

Configuration Item	Value
Web Server	Apache HTTP Server
Version	2.4.41
Document Root	/var/www/html
Service Startup	Automatic
Log Files	Access.log, error.log
Hosted Application	DVWA

3.4.3 DVWA Configuration

The Damn Vulnerable Web Application (DVWA) was deployed on the Ubuntu Server to provide a controlled environment for evaluating the proposed cybersecurity framework against common web application attacks. After downloading the application, it was placed within the Apache document root and configured to communicate with the MariaDB database. The database connection parameters were updated in the DVWA configuration file to establish successful communication between the application and the database server.

Following the initial configuration, the DVWA database was initialized through the application's setup interface, allowing all vulnerable modules to become operational. The application security level was configured according to the experimental requirements to enable the execution of different attack scenarios, including SQL Injection, Cross-Site Scripting (XSS), Command Injection, and Brute Force attacks. These controlled vulnerabilities generated realistic attack traffic that was subsequently inspected by ModSecurity, logged by the Wazuh Agent, analyzed by Wazuh SIEM, and evaluated by the Rule-Based Expert Decision Support System.

The deployment of DVWA provided a safe and repeatable testing platform without exposing production systems to security risks. Furthermore, its intentionally vulnerable design enabled consistent evaluation of the proposed detection, correlation, decision-making, and automated response mechanisms implemented throughout the project.

Table 17 DVWA configuration

Configuration Item	Value
Application	Damn Vulnerable Web Application
Database	MariaDB 10.3.39
Installation Directory	/var/www/html/DVWA
Security Level	Configured according to testing Scenario

3.4.4 ModSecurity Configuration

ModSecurity version 3.2.0 was configured as the Web Application Firewall (WAF) protecting the Apache HTTP Server and the DVWA application. After installation, ModSecurity was enabled within the Apache web server and configured to inspect all incoming HTTP requests and outgoing responses. The firewall was operated in blocking mode to actively detect and prevent malicious web requests before they reached the hosted application.

To improve detection accuracy, the OWASP Core Rule Set (CRS) was integrated with ModSecurity, providing a comprehensive collection of predefined security rules capable of identifying common web application attacks. These rules detect threats such as SQL Injection, Cross-Site Scripting (XSS), Command Injection, Directory Traversal, Local File Inclusion (LFI), Remote File Inclusion (RFI), and other attacks listed in the OWASP Top 10. Whenever a malicious request matched one or more CRS rules, ModSecurity generated detailed security logs containing information about the detected attack, including the rule identifier, attack description, severity level, and source IP address.

The generated ModSecurity logs were stored within the Apache logging system and continuously monitored by the Wazuh Agent. This integration enabled the Wazuh SIEM platform to receive web application security events in real time, apply custom correlation rules, and forward the generated alerts to the Rule-Based Expert Decision Support System for threat evaluation and automated incident response.

Table 18 ModSecurity Configuration

Configuration Item	Value
WAF Software	ModSecurity
Version	3.2.0
Web Server	Apache HTTP Server
Rule Set	OWASP Core Rule Set (CRS)
Operating Mode	Blocking Mode
Log Source	Apache & ModSecurity Logs
Integrated With	Wazuh Agent

3.4.5 Wazuh Manager Configuration

The Wazuh Manager was deployed as the central Security Information and Event Management (SIEM) platform responsible for collecting, processing, and correlating security events generated throughout the proposed cybersecurity framework. After installation, the manager was configured to receive security logs from the Wazuh Agent

installed on the Ubuntu Server, enabling centralized monitoring of operating system events, Apache logs, and ModSecurity alerts.

To support the objectives of the proposed framework, custom detection and correlation rules were added to the Wazuh Manager in addition to the default Wazuh rule set. These custom rules were designed to identify repetitive attack attempts, correlate multiple web application attacks originating from the same source, and assign appropriate severity levels before forwarding alerts to the Rule-Based Expert Decision Support System. This correlation process significantly reduced isolated alerts and improved the identification of coordinated attack scenarios.

The generated alerts were stored within the Wazuh alert repository in JSON format, allowing the Decision Support System to continuously monitor the alerts.json file and evaluate each security event according to predefined expert rules. The successful integration between the Wazuh Manager, Wazuh Agent, ModSecurity, and the Decision Support System established the centralized monitoring capability required for automated threat detection and response.

Table 19 Wazuh Manager Configuration

Configuration Item	Value
SEIM Platform	Wazuh Manager
Agent Version	4.7.5
Alert Format	JSON
Log Sources	Apache, ModSecurity, Ubuntu System Logs
Custom Rules	Enabled
Correlation Rules	Enabled
Output File	/var/ossec/logs/alerts/alerts.json

3.4.6 Wazuh Agent Configuration

The Wazuh Agent version 4.7.5 was installed on the Ubuntu Server to collect security events generated by the operating system, Apache HTTP Server, and ModSecurity Web Application Firewall. After installation, the agent was successfully registered with the Wazuh Manager and configured to establish secure communication for continuous log transmission. This configuration enabled centralized monitoring of the target server while ensuring reliable delivery of security events.

To support the proposed cybersecurity framework, the Wazuh Agent was configured to monitor multiple log sources, including Apache access logs, Apache error logs, ModSecurity audit logs, and operating system logs. Whenever a monitored log file was updated, the agent immediately transmitted the new events to the Wazuh Manager for analysis and correlation. This real-time monitoring capability ensured that web application attacks were detected without noticeable delay.

The successful integration between the Wazuh Agent and the Wazuh Manager established the data collection layer of the proposed framework. All generated security events became available for correlation, severity classification, and subsequent evaluation by the Rule-Based Expert Decision Support System, allowing automated response actions to be initiated whenever critical attacks were detected.

Table 20 Wazuh Agent Configuration

Configuration item	Value
Agent Version	4.7.5
Operating System	Ubuntu server 20.04 LTS
Monitored Logs	Apache Access, Apache Error, ModSecurity, System Logs
Connection	Secure Communication with Wazuh Manager
Status	Active
Log Forwarding	Real-Time

3.4.7 pfSense Firewall Configuration

The pfSense firewall was configured as the network perimeter security device responsible for controlling network traffic and enforcing automated blocking actions. A dedicated virtual machine running pfSense Community Edition was deployed within the Oracle VirtualBox environment and configured to communicate with both the Ubuntu Server and the Rule-Based Expert Decision Support System.

To enable automated incident response, SSH remote administration was configured on the pfSense firewall. This allowed the Rule-Based Expert Decision Support System to establish secure remote connections using the Paramiko Python library and execute firewall management commands after critical threats were detected. Specifically, the system executed the EasyRule command to dynamically create firewall rules that blocked malicious IP addresses without requiring manual administrator intervention. The firewall configuration was validated by executing multiple attack scenarios from the Parrot Security OS machine. Whenever the Decision Support System classified an attack as a Critical Threat, the corresponding attacker IP address was automatically blocked on the pfSense firewall. This configuration demonstrated the successful integration between attack detection, intelligent decision-making, and automated network protection within the proposed cybersecurity framework.

Table 21 Pfsense Configuration

Configuration Item	Value
Firewall Platform	Pfsense Community Edition
Deployment	Oracle VirtualBox
Remote Administration	SSH
Integration Method	Paramiko (Python SSH Library)
Automatic Blocking	EasuRule Command
Trigger Source	Rule-Based Expert Decision Support System
Response Type	Automatic IP Blockinng
Connection Type	Syslog

3.4.8 Rule-Based Expert Decision Support System Configuration

The Rule-Based Expert Decision Support System was implemented in Python 3.8.10 to automate the evaluation of security alerts generated by the Wazuh SIEM platform. The system was configured to continuously monitor the alerts.json file generated by the Wazuh Manager, allowing newly generated alerts to be processed immediately after detection. This configuration enabled real-time analysis of security events without requiring manual intervention from security administrators.

The system parses each JSON alert and extracts the information required for decision-making, including the Wazuh rule identifier, alert severity level, attack description, source IP address, and associated rule groups. Based on these attributes, the inference engine applies predefined expert rules to classify the detected event into one of four threat levels: Informational, Medium Threat, High Threat, and Critical Threat. Each classification is mapped to a predefined response strategy, including event logging, continuous monitoring, administrator approval, or immediate firewall blocking. To minimize false positive responses, a whitelist of trusted IP addresses was configured within the system. Before executing any response action, the source IP address is compared against the whitelist, ensuring that legitimate administrative devices are excluded from automated blocking. Furthermore, every generated decision is stored in a dedicated JSON file (decision_support_output.json) together with the attack information, recommended action, and scientific justification, providing a complete audit trail for security investigations and system evaluation.

Whenever a critical threat is identified, the Decision Support System establishes a secure SSH connection to the pfSense firewall using the Paramiko Python library. The system then executes the EasyRule command to dynamically create a firewall rule that blocks the attacker's IP address. This automated integration significantly reduces response time and demonstrates the effectiveness of combining centralized monitoring, rule-based decision-making, and automated firewall enforcement within the proposed cybersecurity framework.

Table 22 DSS Configuration

Configuration Item	Value
Programming Language	Python 3.8.10
Communication Library	Paramiko
Input Source	/var/ossec/logs/alerts/alerts.json
Output File	/var/ossec/logs/alerts/decision_support_output.json
Input Format	JSON
Decision Model	Rule-Based Expert System
Threat Levels	Informational, Medium, High, Critical
Automatic Response	SSH + EasyRule on Pfsense

3.5 Experimental Setup

3.5.1 Testing Environment

To evaluate the effectiveness of the proposed cybersecurity framework, a controlled experimental environment was established using Oracle VirtualBox. The environment consisted of four virtual machines interconnected through an isolated virtual network to ensure safe and repeatable testing conditions. The Ubuntu Web Server hosted Apache HTTP Server, PHP, MariaDB, DVWA, ModSecurity, and the Wazuh Agent. A separate Ubuntu virtual machine hosted the Wazuh Manager together with the Rule-Based Expert Decision Support System. This architecture separates web application hosting from centralized security monitoring and decision-making, providing a modular and realistic deployment similar to enterprise environments, while the pfSense firewall controlled network traffic and executed automated blocking actions. A fourth virtual machine running Parrot Security OS was used to simulate realistic cyberattacks against the target web application.

All experiments were performed within the isolated laboratory environment to prevent any impact on external networks. During each experiment, attack traffic generated from the attacker machine was inspected by ModSecurity, forwarded by the Wazuh Agent, analyzed by the Wazuh Manager, evaluated by the Rule-Based Expert Decision Support System, and finally mitigated through automated firewall blocking whenever the predefined decision rules classified the attack as a critical threat.

3.5.2 Attack Scenarios

Several web application attack scenarios were conducted to evaluate the effectiveness of the proposed cybersecurity framework. The selected attacks represent common threats targeting web applications and are included among the vulnerabilities addressed by the OWASP Top 10. All attacks were launched from the Parrot Security OS machine against the DVWA application hosted on the Ubuntu Server.

The experimental evaluation focused on three attack scenarios: SQL Injection, Cross-Site Scripting (XSS), and Command Injection, and a correlation between those attacks and multi-vector attacks, and how many times this attack is repeated. For each attack, ModSecurity inspected the incoming HTTP requests and generated security logs whenever malicious activity was detected. The Wazuh Agent forwarded these logs to the Wazuh Manager, where security events were correlated and transformed into alerts. Finally, the Rule-Based Expert Decision Support System analyzed each alert, determined the corresponding threat level, and executed the appropriate response according to the implemented decision rules.

Each attack scenario was executed multiple times to verify the consistency of the proposed framework and ensure reliable operation of the detection, decision-making, and automated response mechanisms.

3.5.3 Evaluation Procedure

The proposed cybersecurity framework was evaluated using a sequential workflow designed to verify the functionality of each security component and the overall integration between them. For every attack scenario, malicious requests were initiated from the Parrot Security OS machine and transmitted to the DVWA web application through the pfSense firewall. ModSecurity inspected each request and generated security logs whenever malicious activity was detected.

The Wazuh Agent continuously monitored the generated logs and securely transmitted them to the Wazuh Manager for centralized processing. The Wazuh Manager applied both predefined and custom correlation rules to generate security alerts, which were subsequently analyzed by the Rule-Based Expert Decision Support System. Based on the implemented inference rules, the system classified the detected event, selected the appropriate response strategy, and automatically executed firewall blocking for critical threats through a secure SSH connection to the pfSense firewall.

The effectiveness of the proposed framework was evaluated by observing the successful completion of each processing stage, including attack detection, log collection, alert generation, threat classification, decision logging, and automated firewall response. The collected experimental results are presented and analyzed in the following chapter.

3.6 Chapter Summary

This chapter presented the development environment and implementation details of the proposed cybersecurity framework. It described the hardware and software components used to build the system, the network topology, the configuration of each security component, and the implementation of the Rule-Based Expert Decision Support System. In addition, the chapter presented the experimental setup adopted to evaluate the proposed framework, including the testing environment, attack scenarios, and evaluation procedure. These implementation details establish the foundation for the experimental results and performance analysis presented in the next chapter.

Chapter 4

Proposed solution

4.1 Chapter Overview

This chapter presents the proposed cybersecurity solution developed in this project and explains how its components are integrated to provide intelligent threat detection, decision-making, and automated incident response. Unlike the previous chapter, which focused on the deployment and configuration of the development environment, this chapter describes the internal design of the proposed framework and the implementation of its custom-developed components.

The chapter begins by presenting the overall system architecture and the workflow followed during attack processing. It then describes the implementation of the custom Wazuh decoders and detection rules developed to improve the identification and classification of web application attacks. After that, the Rule-Based Expert Decision Support System is presented in detail, including its alert monitoring mechanism, decision-making process, and automated firewall response using the pfSense platform. Finally, the complete operational workflow of the proposed framework is illustrated to demonstrate how security events are processed from the initial attack until the execution of the corresponding response.

4.2 Overall System Architecture

The proposed cybersecurity framework integrates multiple open-source security technologies into a unified architecture capable of detecting, analyzing, and responding to web application attacks in real time. The architecture consists of four virtual machines connected through an isolated virtual network. The first virtual machine hosts the vulnerable web application environment, including Apache HTTP Server, PHP, MariaDB, DVWA, ModSecurity, and the Wazuh Agent. The second virtual machine hosts the Wazuh Manager together with the Rule-Based Expert Decision Support System, which continuously monitors security alerts and performs automated threat evaluation. The third virtual machine is the pfSense firewall, responsible for enforcing automated network protection by blocking malicious IP addresses. The fourth virtual machine runs Parrot Security OS and is used to simulate cyberattacks against the target web application.

During system operation, every incoming HTTP request is first inspected by ModSecurity before reaching the web application. When malicious activity is detected, ModSecurity generates security logs that are collected by the Wazuh Agent and forwarded to the Wazuh Manager. The generated alerts are continuously monitored by the Rule-Based Expert Decision Support System, which applies predefined expert rules to classify each security event and determine the appropriate response. For critical attacks, the system establishes a secure SSH connection to the pfSense firewall using the Paramiko Python library and executes an EasyRule command to automatically block the attacker's IP address. This integrated architecture enables rapid threat detection and significantly reduces incident response time through automated decision-making and firewall enforcement.

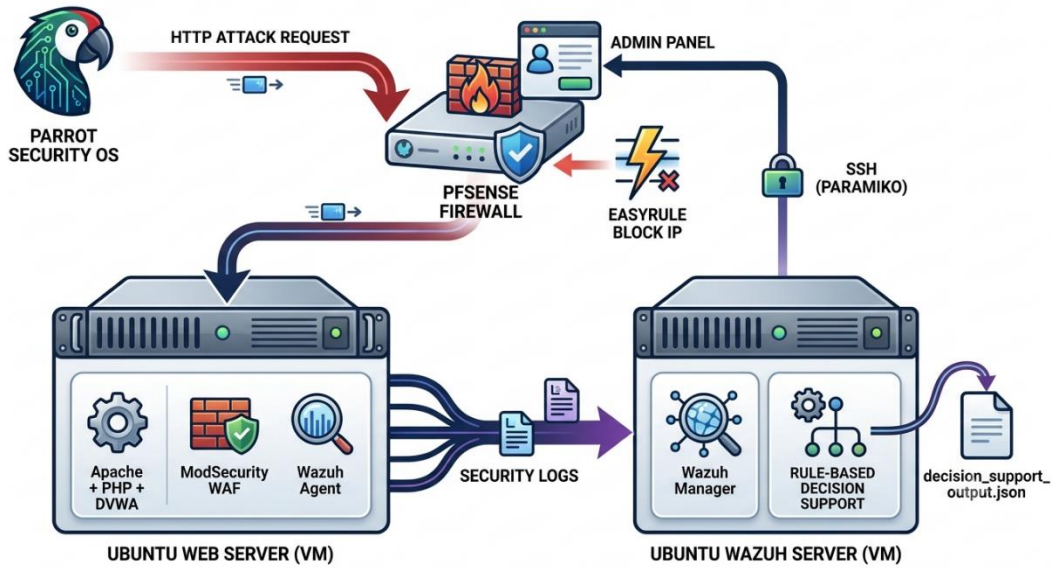


Figure 17 Overall System Architecture

4.3 Development Methodology

The proposed cybersecurity framework was developed using a modular implementation approach to ensure that each security component could be independently configured, tested, and integrated into the overall system. The development process was carried out in multiple stages, beginning with the deployment of a secure web application environment and ending with the implementation of an automated decision-making and response mechanism. This methodology simplified system integration while allowing each component to be validated before proceeding to the next development stage. The first stage focused on establishing the protected web application environment by deploying Apache HTTP Server, PHP, MariaDB, DVWA, ModSecurity, and the Wazuh Agent on the Ubuntu Web Server. In parallel, a dedicated Ubuntu virtual machine was configured to host the Wazuh Manager and the Rule-Based Expert Decision Support System. The communication between the Wazuh Agent and the Wazuh Manager was verified to ensure reliable transmission of security events generated by the web server.

After completing the infrastructure deployment, the second stage involved developing custom detection components within Wazuh. This included creating custom decoders to extract relevant information from ModSecurity logs and implementing custom detection rules to classify web application attacks according to their characteristics and severity levels. These custom components enabled the generation of structured security alerts that served as the primary input for the decision-making process. The third stage focused on implementing the Rule-Based Expert Decision Support System using Python. The system was designed to continuously monitor alerts generated by the Wazuh Manager, analyze their security attributes, classify the detected threats according to predefined expert rules, and determine the appropriate response for each security event. To complete the proposed framework, the Decision Support System was integrated with the pfSense firewall through secure SSH communication

using the Paramiko library, enabling automatic firewall rule creation whenever critical threats were detected.

Finally, all system components were integrated and validated through a series of controlled attack scenarios, including SQL Injection, Cross-Site Scripting (XSS), and Command Injection attacks. These experiments verified the correct interaction between the detection layer, the decision-making engine, and the automated response mechanism, confirming the successful implementation of the proposed cybersecurity framework.

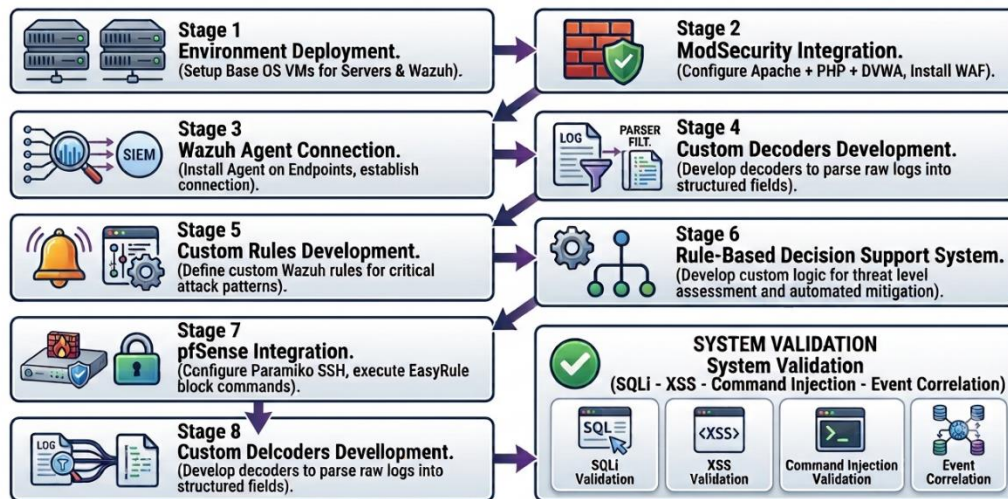


Figure 18 Development Methodology

4.4 Wazuh Detection Layer

4.4.1 Custom Decoders

The proposed framework relies on custom Wazuh decoders to process ModSecurity logs before they are evaluated by the detection rules. Since the default Wazuh decoders do not extract all the information required by the proposed Decision Support System, three custom decoders were developed to identify ModSecurity events, extract the attacker's IP address, and retrieve the relevant attack information from each log entry.


```

<decoder name="ModSec-log">
  <prematch>ModSecurity:</prematch>
</decoder>

<decoder name="ModSec-ip">
  <parent>ModSec-log</parent>
  <regex>client (\d+\.\d+\.\d+\.\d+)</regex>
  <order>srcip</order>
</decoder>

<decoder name="ModSec-fields">
  <parent>ModSec-log</parent>
  <regex>id "\d+" msg "(.+)" ".+data "(.+)"</regex>
  <order>modsec_id, modsec_msg, modsec_data</order>
</decoder>

```

Figure 19 Custom Wazuh Decoders

The first decoder, ModSec-log, serves as the parent decoder for all ModSecurity events. It identifies log entries containing the keyword "ModSecurity:" using the prematch directive. Once a matching event is identified, it is forwarded to the child decoders for additional processing.

The second decoder, ModSec-ip, extracts the attacker's IP address using a regular expression and stores it in the srcip field. This field is later used by the correlation rules and the Rule-Based Expert Decision Support System to identify repeated attacks originating from the same source.

The third decoder, ModSec-fields, extracts additional security information from the ModSecurity log, including the attack message and the corresponding attack data. These extracted fields provide the contextual information required for accurate event classification and decision-making.

Together, the three custom decoders transform raw ModSecurity log entries into structured security events that can be efficiently processed by the custom Wazuh rules and the Rule-Based Expert Decision Support System.

4.4.2 Custom Detection Rules

After processing ModSecurity logs through the custom decoders, a set of custom Wazuh detection rules was developed to identify web application attacks and classify security events according to their severity and behavioral characteristics. These rules constitute the core of the proposed detection layer by converting parsed ModSecurity events into structured Wazuh alerts that are subsequently evaluated by the Rule-Based Expert Decision Support System. The developed rules were organized into three categories: attack detection rules, correlation rules, and repeated attack rules.


```

21 group name="modsec,attacks,web_attacks"
22
23
24 <rule id="100100" level="7">
25   <decoded_as>ModSec-log</decoded_as>
26   <match>942100</match>
27   <description>SQL injection detected (ModSecurity)</description>
28   <group>,modsecurity,sqli,sqli_xss,web_attack,sqli_cmdinj,</group>
29 </rule>
30
31 <rule id="100101" level="7">
32   <decoded_as>ModSec-log</decoded_as>
33   <match>941160</match>
34   <description>cross Site Scripting -XSS- detected (ModSecurity)</-
35   <group>,modsecurity,xss,sqli_xss,web_attack,xss_cmdinj,</group>
36 </rule>
37
38 <rule id="100102" level="12">
39   <decoded_as>ModSec-log</decoded_as>
40   <match>932105</match>
41   <description>Command injection detected (ModSecurity)</description>
42   <group>,modsecurity,command_injection,sqli_cmdinj,web_attack,xss_cmdinj,</-
43   </group>
44 </rule>
45 </group>

```

Figure 20 Custom Rules for Individual Attacks

The first group of rules was designed to detect individual web application attacks generated by ModSecurity. Three custom rules were implemented to identify SQL Injection, Cross-Site Scripting (XSS), and Command Injection attacks by matching the corresponding ModSecurity rule identifiers. The SQL Injection and XSS rules generate alerts with severity level 7, reflecting attacks that require additional contextual analysis before initiating automated response actions. In contrast, the Command Injection rule generates a severity level 12 alert because this attack type represents a significantly higher risk due to its potential to execute arbitrary operating system commands on the target server. Each rule was also assigned descriptive groups to facilitate event classification and subsequent correlation within Wazuh.

These detection rules provide the initial classification of security events before they are processed by the correlation rules and the Rule-Based Expert Decision Support System. By assigning different severity levels according to the nature of each attack, the proposed framework enables more accurate threat prioritization and supports context-aware security decision-making.

```

48 <group name="modsec,correlation">
49
50
51 <rule id="100200" level="12" timeframe="30">>
52   <if_sid>100100</if_sid>
53   <if_matched_sid>100101</if_matched_sid>
54   <same_source_ip />
55   <description>Multi-Vector Attack: XSS then SQLI, check rules: 100100,
56   100101 for more details</description>
57 </rule>
58 <rule id="100201" level="12" timeframe="30">>
59   <if_sid>100101</if_sid>
60   <if_matched_sid>100100</if_matched_sid>
61   <same_source_ip />
62   <description>Multi-Vector Attack: SQI then XSS, check rules: 100100,
63   100101</description>
64 </rule>
65 <rule id="100202" level="12" timeframe="30">>
66   <if_sid>100102</if_sid>
67   <if_matched_sid>100100</if_matched_sid>
68   <same_source_ip />
69   <description>Multi-Vector Attack: SQLI then COMMAND INJECTION, check
70   rules: 100100, 100102</description>
71 </rule>

```

```

72 <rule id="100203" level="12" timeframe="30">
73   <if_sid>100100</if_sid>
74   <if_matched_sid>100102</if_matched_sid>
75   <same_source_ip />
76   <description>Multi-Vector Attack: COMMAND INJECTION then SQLI, check
rules: 100100, 100102</description>
77 </rule>
78
79 <rule id="100204" level="12" timeframe="30">
80   <if_sid>100101</if_sid>
81   <if_matched_sid>100102</if_matched_sid>
82   <same_source_ip />
83   <description>Multi-Vector Attack: COMMAND INJECTION then XSS, check
rules: 100100, 100101</description>
84 </rule>
85
86 <rule id="100205" level="12" timeframe="30">
87   <if_sid>100102</if_sid>
88   <if_matched_sid>100101</if_matched_sid>
89   <same_source_ip />
90   <description>Multi-Vector Attack: XSS then COMMAND INJECTION, check
rules: 1300100, 100101</description>
91 </rule>

```

Figure 21 Custom Wazuh Correlation Rules for Multi-Vector attack

To enhance the detection capabilities of the proposed framework, additional correlation rules were developed to identify multi-vector attack scenarios performed by the same attacker. Unlike the individual detection rules, which identify a single attack type, these rules correlate different attack techniques occurring within a 30-second timeframe from the same source IP address. This correlation is implemented using the `<if_sid>`, `<if_matched_sid>`, and `<same_source_ip>` directives provided by the Wazuh rule engine.

Six correlation rules were implemented to detect different combinations of SQL Injection, Cross-Site Scripting (XSS), and Command Injection attacks. Whenever two different attack types are observed within the specified timeframe, Wazuh generates a new correlated alert with severity level 12, indicating a complex attack sequence that represents a higher security risk than isolated attack events. These correlated alerts provide valuable contextual information for the Rule-Based Expert Decision Support System, enabling more accurate threat prioritization and automated response decisions.

```

99 <rule id="100207" level="10" frequency="2" timeframe="30">
100   <if_matched_sid>100101</if_matched_sid>
101   <same_source_ip />
102   <description>Multiple XSS attack attempts from same ip</description>
103 </rule>
104
105 <rule id="100208" level="10" frequency="2" timeframe="30">
106   <if_matched_sid>100102</if_matched_sid>
107   <same_source_ip />
108   <description>Multiple command injection attempts from same ip</-
description>
109 </rule>
110
111 </group>

```

Figure 22 Custom Rules for Repeated Attacks

In addition to detecting multi-vector attacks, the proposed framework includes custom rules for identifying repeated attack attempts originating from the same source IP address. These rules utilize the frequency and timeframe parameters to monitor the occurrence of identical attack types within a 30-second period. When the predefined attack frequency is exceeded, Wazuh generates a new alert with severity level 10, indicating persistent malicious activity that requires further attention.

Three repeated attack rules were implemented to monitor repeated SQL Injection, Cross-Site Scripting (XSS), and Command Injection attempts. This approach enables the framework to distinguish between isolated attack events and continuous attack campaigns, providing the Decision Support System with additional behavioral context before determining the appropriate security response.

Table 23 Summary of Custom Wazuh Detection Rules

Rule Category	Rule IDs	Purpose	Severity Level
Individual Attack Detection	100101-100102	Detect SQL Injection, XSS, and Command Injection	7/12
Multi-Vector Correlation	100200-100205	Detect combinations of different attacks from the same source IP	12
Repeated Attack Detection	100206-100208	Detect repeated attacks within a predefined timeframe	10

4.5 Rule-Based Expert Decision Support System

4.5.1 System Overview

The Rule-Based Expert Decision Support System (DSS) was developed to automate the analysis of security alerts generated by the Wazuh Manager and to determine the appropriate response according to predefined expert rules. Rather than relying solely on alert severity, the system evaluates multiple security attributes, including the Wazuh rule identifier, alert level, attack category, correlation groups, and source IP address, to classify detected events into different threat levels.

The Decision Support System continuously monitors the Wazuh alerts.json file, where newly generated security alerts are stored in JSON format. Each alert is parsed and evaluated using a predefined set of decision rules that determine whether the event should be logged, monitored, submitted for administrator approval, or immediately mitigated by automatically blocking the attacker's IP address through the pfSense firewall. In addition, every generated decision is recorded together with its corresponding justification to provide a complete audit trail for incident analysis and future security investigations.

The proposed Decision Support System consists of four functional modules: the Alert Monitoring Module, which continuously monitors newly generated alerts; the Decision Engine, which evaluates each alert according to the implemented expert rules; the Automated Response Module, which performs firewall blocking when necessary; and the Decision Logging Module, which stores all generated decisions for documentation and analysis purposes.

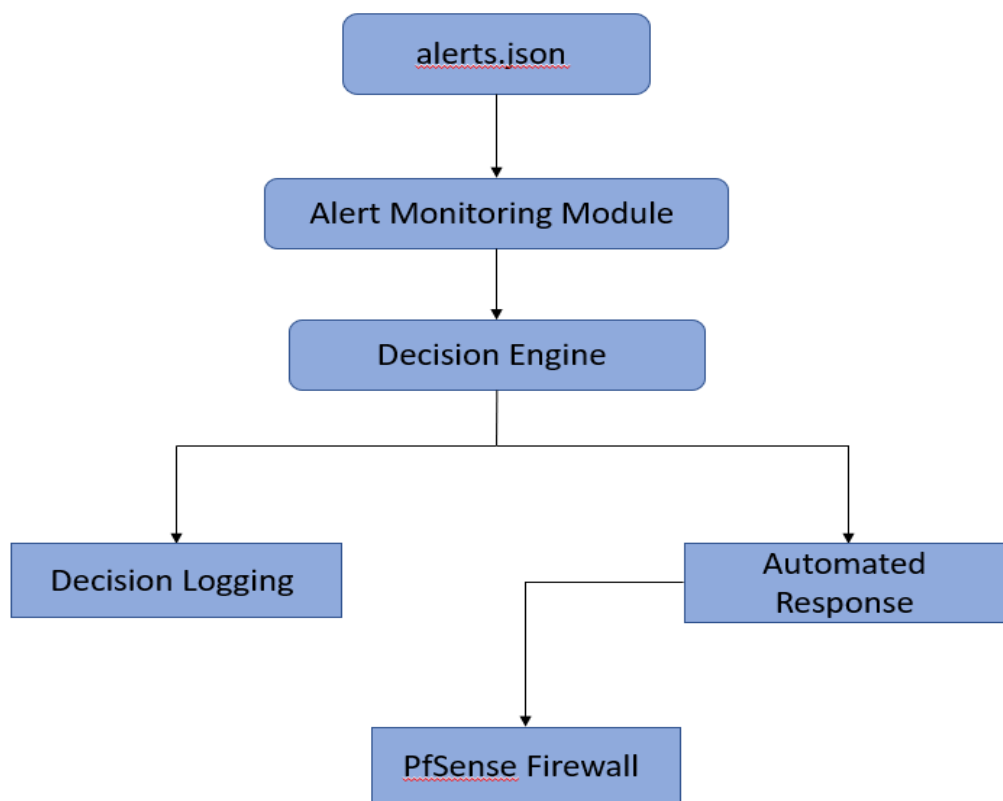


Figure 23 Rule-Based Expert DSS Architecture

4.5.2 Alert Monitoring Module

The Alert Monitoring Module serves as the entry point of the proposed Rule-Based Expert Decision Support System. Its primary responsibility is to continuously monitor the Wazuh alerts.json file, where all security alerts generated by the Wazuh Manager are stored. Instead of periodically reopening the file, the implemented module adopts a real-time monitoring approach similar to the Linux tail -f command, allowing newly generated alerts to be processed immediately after they are written to the log file.

```

133 def follow_alerts_file():
134
135     print(f"[*] Decision Support System Started. Monitoring:
136           {ALERTS_FILE_PATH}...")
137
138     while not os.path.exists(ALERTS_FILE_PATH):
139         time.sleep(1)
140
141     with open(ALERTS_FILE_PATH, "r") as f:
142         f.seek(0, os.SEEK_END)
143
144

```

Figure 24 Alert Monitoring Initialization

As shown in Figure 22, the module first verifies that the alerts.json file exists before starting the monitoring process, preventing runtime errors that may occur if the file has not yet been created by Wazuh. Once the file becomes available, the monitoring function positions the file pointer at the end of the log using `seek(0, os.SEEK_END)`, ensuring that only newly generated alerts are processed rather than previously stored events.

The monitoring process operates continuously inside an infinite loop. Whenever a new line is appended to the log file, it is immediately read and converted into a JSON object using Python's `json.loads()` function. This structured representation allows the Decision Support System to access individual alert attributes efficiently before forwarding the event to the decision engine for further analysis.

To improve system reliability, exception handling mechanisms were implemented to ignore malformed JSON entries and prevent unexpected runtime errors from interrupting the monitoring service. As a result, the module remains continuously operational and capable of processing security alerts in real time without requiring manual intervention.

4.5.3 Decision Engine

The Decision Engine represents the core component of the proposed Rule-Based Expert Decision Support System. Its primary function is to analyze every security alert received from the Alert Monitoring Module and determine the appropriate response based on a predefined set of expert security rules. Unlike traditional alert processing mechanisms that rely solely on alert severity, the proposed Decision Engine evaluates multiple security attributes to provide context-aware threat assessment and response selection.

```

62 def evaluate_alert_decision(alert_json):
63
64     rule_info = alert_json.get("rule", {})
65     rule_id = str(rule_info.get("id"))
66     level = int(rule_info.get("level", 0))
67     groups = rule_info.get("groups", [])
68     description = rule_info.get("description", "").lower()
69     src_ip = alert_json.get("data", {}).get("srcip", "unknown")
70

```

Figure 25 Extraction of Security Attributes

As illustrated in Figure 23, the Decision Engine begins by extracting the most relevant security information from each Wazuh alert. The extracted attributes include the Wazuh rule identifier, alert severity level, attack description, attack groups, and the attacker's source IP address. These parameters represent the primary input of the decision-making process and provide the contextual information required to accurately evaluate the detected security event.

The extracted security attributes are subsequently used to determine whether the event corresponds to a trusted source, an isolated web attack, a repeated attack, or a correlated multi-vector attack. This structured evaluation enables the Decision Support System to apply different response strategies according to both the attack characteristics and its contextual risk level.


```

74     src_ip = alert_json.get("data", {}).get("srcip", "unknown")
75
76     if src_ip in WHITELIST_IPS:
77         return {
78             "decision": "LOW / BYPASS",
79             "action": "IGNORE_EVENT",
80             "reason": "Traffic from whitelisted administrative IP.
Automated blocking bypassed to preserve business continuity."
81         }
82
83     if "command" in description or rule_id in ["100102", "100208"]:
84         return {
85             "decision": "CRITICAL THREAT",
86             "action": "IMMEDIATE_AUTO_BLOCK",
87             "reason": f"Remote Code Execution (RCE) via Command Injection
detected (Rule {rule_id}). "
88             f"CVSS v3.1 rates this vector as CRITICAL (Score:
9.8) because it allows complete OS take-over. "
89             f"Immediate firewall containment executed."
90         }
91
92     ZZZZ
93     elif level == 12 and "correlation" in groups:
94         return {
95             "decision": "CRITICAL THREAT",
96             "action": "IMMEDIATE_AUTO_BLOCK",
97             "reason": f"Multi-Vector Attack chain detected (Rule
{rule_id}: '{rule_info.get('description')}'). "
98             f"Per Cyber Kill Chain methodology, shifting between
different exploit methodologies (XSS/SQLi) "
99             f"confirms a high-confidence targeted penetration
attempt. Machine-speed mitigation required."
100         }
101
102     elif level == 10 and "correlation" in groups:
103         return {
104             "decision": "HIGH THREAT",
105             "action": "PENDING_ADMIN_APPROVAL",
106             "reason": f"Repetitive web scanning/exploitation attempts
detected (Rule {rule_id}). "
107             f"OWASP Risk Methodology dictates high likelihood of
success due to frequency. "
108             f"Deferred to Dashboard for human validation to rule
out potential NAT/Proxy false positives."
109         }
110
111     elif level == 7:
112         return {
113             "decision": "MEDIUM THREAT",
114             "action": "LOG_AND_MONITOR",
115             "reason": f"Isolated single event web attack (Rule {rule_id}).
"
116             f"NIST SP 800-61 r2 triage guidelines advise logging
single vectors "
117             f"to minimize business disruption while awaiting
further contextual correlation."
118         }
119
120     else:
121         return {
122             "decision": "INFORMATIONAL",
123             "action": "LOG_EVENT",
124             "reason": f"Standard system alert with level {level}. No
tactical action required."
125         }

```

Figure 26 Rule-Based Decision Logic

After extracting the required security attributes, the Decision Engine evaluates each alert using a predefined set of expert decision rules. The evaluation process follows a sequential rule-based approach, where every alert is compared against a series of security conditions until the appropriate threat category and response action are identified. This approach ensures that alerts are processed consistently while allowing different response strategies to be applied according to the attack context.

The first decision rule verifies whether the source IP address belongs to the predefined whitelist. Trusted administrative IP addresses are excluded from automated mitigation to prevent unnecessary service interruptions and reduce the possibility of false positive responses. If the source IP matches an entry in the whitelist, the alert is classified as a LOW / BYPASS event, and no defensive action is performed.

For non-whitelisted events, the Decision Engine evaluates the attack severity according to the generated Wazuh alert. Command Injection attacks and multi-vector correlated attacks are classified as Critical Threats because they represent highly dangerous attack scenarios that require immediate containment. In these cases, the system automatically initiates the firewall blocking procedure without requiring human intervention. Alerts generated by the repeated attack correlation rules are classified as High Threats.

Instead of immediately blocking the attacker, the system forwards these events for administrator validation before any mitigation action is executed. This decision strategy minimizes the possibility of blocking legitimate users who may share the same public IP address through NAT or proxy services.

Finally, isolated SQL Injection and Cross-Site Scripting alerts are classified as Medium Threats. These events are recorded and continuously monitored without initiating an immediate response, allowing the system to collect additional contextual information before escalating the threat level. Any remaining alerts that do not satisfy the predefined decision rules are classified as Informational events and are logged for auditing purposes only.

Table 24 Decision Logic in the DSS

Detection Condition	Threat Level	Recommended Action
Source IP in Whitelist	LOW/BYPASS	Ignore Event
Command Injection	CRITICAL THREAT	Immediate Auto Block
Multi-Vector Correlation	CRITICAL THREAT	Immediate Auto Block
Repeated Attack	HIGH THREAT	Pending Administrator Approval
Individual Attack	MEDIUM THREAT	Log and Monitor
Other Alerts	INFORMATIONAL	Log Event

4.5.4 Automated Firewall Response

The Automated Firewall Response Module is responsible for enforcing the security decisions generated by the Rule-Based Expert Decision Support System. Once the Decision Engine classifies an event as a Critical Threat and recommends an IMMEDIATE_AUTO_BLOCK action, the module automatically initiates the firewall mitigation process without requiring administrator intervention. This approach significantly reduces the response time between threat detection and containment, thereby minimizing the attacker's opportunity to continue exploiting the target system.

```

13 def execute_active_response_block(ip):
14     PFSense_IP = "10.10.10.1"
15     PFSense_USER = "admin"
16     PFSense_PASS = "pfsense"
17     WAN_INTERFACE = "wan"
18     try:
19         print(f"[*] Attempting to block IP {ip} on pfSense via SSH...")
20         ssh = paramiko.SSHClient()
21         ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())
22         ssh.connect(PFSense_IP, username=PFSense_USER,
23 password=PFSense_PASS, timeout=5)
24         command = f"easyrule block {WAN_INTERFACE} {ip}"
25         stdin, stdout, stderr = ssh.exec_command(command)
26         output = stdout.read().decode().strip()
27         error = stderr.read().decode().strip()
28         ssh.close()
29         if error:
30             print(f"[-] pfSense Error: {error}")
31         else:
32             print(f"[+] pfSense Response: {output}")
33             print(f"[{datetime.now()}] [ACTIVE RESPONSE] IP {ip} blocked
34 successfully on pfSense firewall.")
35     except Exception as e:
36         print(f"[-] Failed to execute block on pfsense for IP {ip}: {e}")

```

Figure 27 Automated Firewall Response Using Paramiko and pfSense

As illustrated in Figure 25, the module establishes a secure SSH connection to the pfSense firewall using the Paramiko Python library. After successful authentication, the system dynamically constructs an easyrule block command containing the attacker's IP address and executes it remotely on the firewall. This command automatically creates a new firewall rule that blocks all subsequent traffic originating from the malicious source IP.

The response procedure is executed only when the Decision Engine recommends an immediate blocking action, ensuring that firewall rules are created exclusively for verified critical threats. In addition, the module verifies the execution result and reports whether the blocking operation was completed successfully or whether an execution error occurred. This verification mechanism improves the reliability of the automated response process and provides immediate feedback regarding the mitigation status. The integration between the Decision Support System and the pfSense firewall enables machine-speed incident response, eliminating the delays associated with manual firewall administration while ensuring that critical attacks are contained immediately after detection.

4.5.5 Decision Logging

The Decision Logging Module was implemented to maintain a complete record of every security decision generated by the proposed Rule-Based Expert Decision Support System. Instead of storing only the original Wazuh alert, the module creates a new structured record that includes both the detection information and the decision generated by the expert system. This provides a comprehensive audit trail that can be used for incident investigation, system evaluation, and future security analysis.

```

145         decision_output = {
146             "timestamp": datetime.now().isoformat(),
147             "attacker_ip": alert_json.get("data", {}).get("srcip",
"unknown"),
148             "wazuh_rule_id": alert_json.get("rule", {}).get("id"),
149             "attack_type": alert_json.get("rule",
{}).get("description"),
150             "decision_level": analysis["decision"],
151             "recommended_action": analysis["action"],
152             "scientific_justification": analysis["reason"]
153         }
154
155
156         print(f"[[{decision_output['decision_level']}] IP:
{decision_output['attacker_ip']} -> Action:
{decision_output['recommended_action']}")
157
158         log_decision_result(decision_output)

```

Figure 28 Generation and Logging of Decision Records

For every processed alert, the module creates a structured JSON object containing the processing timestamp, attacker IP address, Wazuh rule identifier, detected attack type, assigned threat level, recommended response, and the scientific justification associated with the generated decision. These records are stored in a dedicated log file named `decision_support_output.json`, providing a persistent history of all decisions produced by the system.

Maintaining a dedicated decision log offers several practical advantages. It enables security administrators to review previously generated decisions, verify the correctness of automated responses, and analyze the overall behavior of the Decision Support System during security incidents. Furthermore, separating decision records from the original Wazuh alerts simplifies forensic investigations by clearly documenting both the detected event and the reasoning behind the selected response.

4.6 Implementation and Attack Scenarios

The proposed framework was experimentally validated using multiple web application attack scenarios to verify the effectiveness of the detection, correlation, decision-making, and automated response mechanisms. Each attack was launched against the intentionally vulnerable DVWA application hosted on the Apache web server. During each experiment, the generated attack traffic passed through ModSecurity, was analyzed by the custom Wazuh detection rules, evaluated by the Rule-Based Expert Decision Support System, and, when necessary, mitigated automatically through the pfSense firewall.

The following subsections present the implementation of the tested attack scenarios and illustrate how the proposed framework processes different types of attacks from their initial detection to the final security decision. The presented scenarios demonstrate the interaction between all system components and validate the practical operation of the proposed cybersecurity framework.

4.6.1 SQL Injection Attack Scenario

The first experimental scenario evaluates the capability of the proposed framework to detect and analyze an isolated SQL Injection attack against the DVWA web application. The purpose of this experiment is to verify the complete processing pipeline, starting from attack execution, through Wazuh detection, and ending with the security decision generated by the Rule-Based Expert Decision Support System.

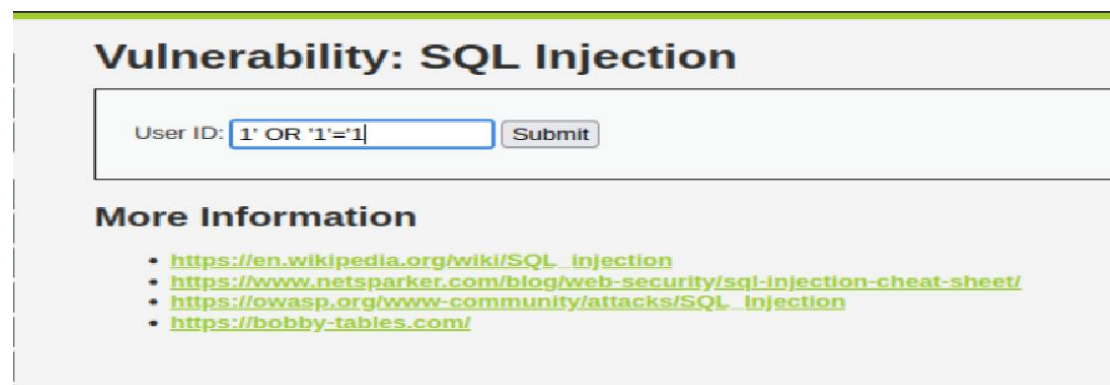


Figure 29 SQL Injection Executed Against DVWA

As shown in Figure 27, a SQL Injection payload was submitted to the vulnerable DVWA application to simulate an unauthorized database access attempt. The malicious request was intercepted by ModSecurity, which analyzed the HTTP request using the OWASP Core Rule Set and identified it as a SQL Injection attack. The generated security event was then forwarded to the Wazuh Manager for further processing.

```

> Jul 24, 2026 @ 15:03:25.322 input.type: log agent.ip: 10.10.10.127 agent.name: WAF agent.id: 003
manager.name: yamen data.srcip: 20.20.20.10 rule.firedtimes: 2 rule.mail: false
rule.level: 7 rule.description: SQL injection detected (ModSecurity)
rule.groups: modsec, attacks, web_attacks, modsecurity, sqli, sqli_xss, web_attack,
sqli_cmdinj rule.id: 100100 location: /var/log/apache2/error.log

```

Figure 30 SQL Injection Alert by Wazuh

After receiving the ModSecurity event, the custom Wazuh decoder extracted the relevant attack information, including the source IP address and attack details. Subsequently, the custom detection rule (Rule ID 100100) classified the event as a SQL Injection attack and generated a security alert with severity level 7. This alert was then forwarded to the Rule-Based Expert Decision Support System for threat evaluation and response selection.

```

52 {"timestamp": "2026-07-24T15:03:25.813578", "attacker_ip": "20.20.20.10",
"wazuh_rule_id": "100100", "attack_type": "SQL injection detected
(ModSecurity)", "decision_level": "MEDIUM THREAT", "recommended_action":
"LOG_AND_MONITOR", "scientific_justification": "Isolated single event web
attack (Rule 100100). NIST SP 800-61 r2 triage guidelines advise logging
single vectors to minimize business disruption while awaiting further
contextual correlation."}

```

Figure 31 Decision Generated by DSS for SQL Injection

As illustrated in Figure 29, the Decision Support System analyzed the received Wazuh alert according to the implemented decision rules. Since the detected event represented an isolated SQL Injection attack without additional correlated or repeated malicious activity, the system classified it as a Medium Threat. Based on this classification, the recommended response was Log and Monitor, meaning that the event was recorded for continuous monitoring without triggering an automatic firewall response. This behavior minimizes unnecessary service disruption while allowing the system to collect additional evidence before applying stronger mitigation actions.

Table 25 Summary of SQL Injection Scenario

Processing Stage	Result
Alert Type	SQL Injection
Detection Component	ModSecurity
Wazuh Rule ID	100100
Alert Severity	Level 7
DSS Classification	Medium Threat
Recommended Action	Log and Monitor
Automatic Firewall Block	NO

4.6.2 Cross-Site Scripting (XSS) Attack Scenario

The second experimental scenario evaluates the proposed framework against a Cross-Site Scripting (XSS) attack targeting the DVWA web application. The objective of this experiment is to verify the framework's ability to detect client-side code injection attempts, generate the appropriate Wazuh alert, and classify the attack according to the implemented decision policy.

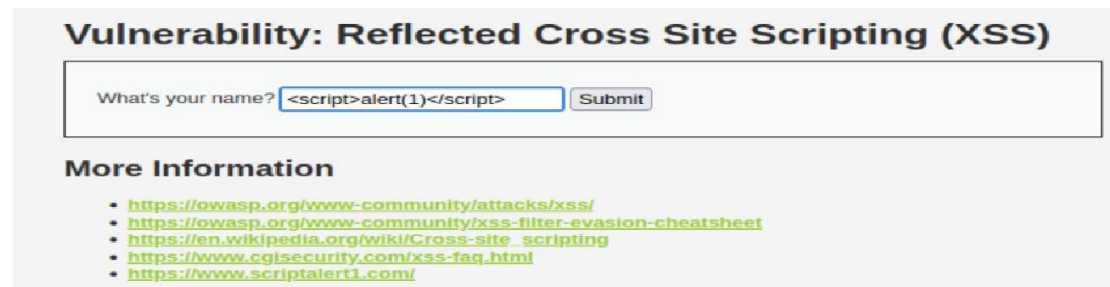


Figure 32 XSS Executed Against DVWA

As illustrated in Figure 30, a malicious JavaScript payload was submitted through the vulnerable web application to simulate a reflected Cross-Site Scripting attack. The HTTP request was inspected by ModSecurity using the OWASP Core Rule Set, which successfully identified the malicious payload and generated a security event for further analysis.



Figure 33 . XSS Alert by Wazuh

Following the detection process, the generated ModSecurity event was forwarded to the Wazuh Manager, where the custom decoder extracted the relevant attack information. The custom detection rule (Rule ID 100101) classified the event as a Cross-Site Scripting attack and generated a security alert with severity level 7. The alert was subsequently delivered to the Rule-Based Expert Decision Support System for evaluation.

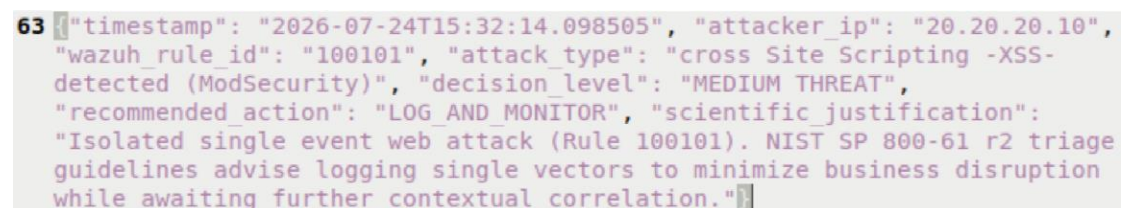


Figure 34 Decision Generated by DSS for XSS

After evaluating the received alert, the Decision Support System classified the event as a Medium Threat because the attack represented an isolated XSS attempt without any additional evidence indicating repeated or correlated malicious activity. Consequently, the system selected the LOG_AND_MONITOR action, allowing the event to be documented and continuously monitored without initiating an automated firewall response. This decision follows the implemented expert policy, which prioritizes contextual evidence before applying active mitigation measures.

Table 26 Summary of XSS Scenario

Processing Stage	Result
Alert Type	Cross-Site Scripting (XSS)
Detection Component	ModSecurity
Wazuh Rule ID	100101
Alert Severity	Level 7
DSS Classification	Medium Threat
Recommended Action	Log and Monitor
Automatic Firewall Block	NO

4.6.3 Command Injection Attack Scenario

The third experimental scenario evaluates the framework using a Command Injection attack. Unlike the previous scenarios, this experiment demonstrates the complete automated response workflow, where the detected attack is classified as a critical threat and immediately mitigated through the pfSense firewall.

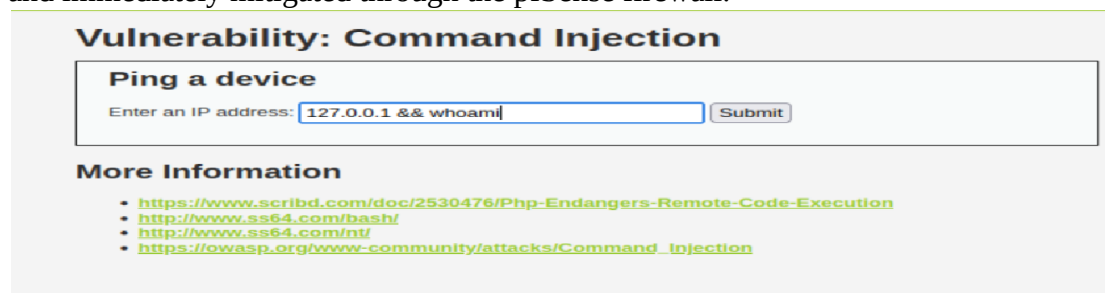


Figure35 Command Injection Executed Against DVWA

As shown in Figure 33, a malicious operating system command was injected through the vulnerable application to simulate an attempt to execute unauthorized commands on the target server. The malicious request was intercepted by ModSecurity, which successfully detected the attack and generated a corresponding security event.

```
> Jul 24, 2026 @ 14:32:03.951 input.type: log agent.ip: 10.10.10.127 agent.name: WAF agent.id: 003
manager.name: yamen data.srcip: 20.20.20.10 rule.firedtimes: 3 rule.mail: true
rule.level: 12 rule.description: Command injection detected (ModSecurity)
rule.groups: modsec, attacks, web_attacks, modsecurity, command_injection, sqli_cmdinj,
web_attack, xss_cmdinj rule.id: 100102 location: /var/log/apache2/error.log
```

Figure 36 Command Injection Alert by Wazuh

After processing the ModSecurity log, the custom decoder extracted the attack information, and the custom detection rule (Rule ID 100102) generated a security alert with severity level 12. The alert was then forwarded to the Rule-Based Expert Decision Support System for threat assessment.

```
2 {"timestamp": "2026-07-08T03:36:01.361437", "attacker_ip": "20.20.20.10",
"wazuh_rule_id": "100102", "attack_type": "Command injection detected
(ModSecurity)", "decision_level": "CRITICAL THREAT", "recommended_action":
"IMMEDIATE_AUTO_BLOCK", "scientific_justification": "Remote Code Execution
(RCE) via Command Injection detected (Rule 100102). CVSS v3.1 rates this
vector as CRITICAL (Score: 9.8) because it allows complete OS take-over.
Immediate firewall containment executed."}
```

Figure 37 Decision Generated by DSS for Command Injection

Based on the implemented expert decision rules, the detected Command Injection attack was classified as a Critical Threat due to its high potential impact on the target system. Consequently, the Decision Support System immediately selected the IMMEDIATE_AUTO_BLOCK action, initiating the automated mitigation procedure without requiring administrator intervention.

Rules (Drag to Change Order)											
☐	States	Protocol	Source	Port	Destination	Port	Gateway	Queue	Schedule	Description	Actions
☒	0/45 KiB	IPv4 *	EasyRuleBlockHostsWAN	*	*	*	*	none		Blocked via EasyRule	   

Figure 38 Automatically Generated Rule for Blocking the Attacker IP

Upon receiving the automated blocking request, the Decision Support System established a secure SSH connection with the pfSense firewall using the Paramiko library. The system then executed the easyrule block command, which dynamically created a firewall rule to deny all subsequent network traffic originating from the attacker's IP address. This experiment demonstrates the successful integration between Wazuh, the Decision Support System, and pfSense, enabling immediate containment of high-risk attacks.

Table 27 Summary of Command Injection Scenario

Processing Stage	Result
Alert Type	Command Injection
Detection Component	ModSecurity
Wazuh Rule ID	100102
Alert Severity	Level 12
DSS Classification	Critical Threat
Recommended Action	Immediate Auto Block
Automatic Firewall Block	YES

4.6.4 Multi-Vector Attack Scenario

The fourth experimental scenario demonstrates the framework's capability to identify and respond to a multi-vector web attack. Unlike isolated attack attempts, this scenario simulates an attacker performing multiple attack techniques from the same source IP address within a short time interval. The objective is to verify the effectiveness of the implemented correlation rules and the automated response mechanism.

Vulnerability: SQL Injection

User ID:

More Information

- https://en.wikipedia.org/wiki/SQL_injection
- <https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/>
- https://owasp.org/www-community/attacks/SQL_injection
- <https://bobby-tables.com/>

Vulnerability: Reflected Cross Site Scripting (XSS)

What's your name?

More Information

- <https://owasp.org/www-community/attacks/xss/>
- <https://owasp.org/www-community/xss-filter-evasion-cheatsheet>
- https://en.wikipedia.org/wiki/Cross-site_scripting
- <https://www.cgisecurity.com/xss-faq.html>
- <https://www.scriptalert1.com/>

Figure 39 Multi-Vector Attack Executed Against DVWA

As shown in Figure 37, two different web attacks were launched sequentially from the same source IP address within the configured correlation timeframe. The first attack generated an individual Wazuh alert, followed shortly by the second attack. Since both attacks originated from the same source, the implemented Wazuh correlation rules recognized the attack sequence as a coordinated multi-vector attack rather than two isolated security events.


```
> Jul 24, 2026 @ 16:07:19.313 input.type: log agent.ip: 10.10.10.127 agent.name: WAF agent.id: 003
manager.name: yamen data.srcip: 20.20.20.10 rule.firedtimes: 1 rule.mail: true
rule.level: 12 rule.description: Multi-Vector Attack: SQI then XSS, check rules:
100100, 100101 rule.groups: modsec, correlation rule.id: 100201 rule.frequency: 2
location: /var/log/apache2/error.log decoder.name: ModSec-log id: 1784898439.52101
```

Figure 40 Multi-Vector Attack Alert by Wazuh

After the correlation rule was triggered, Wazuh generated a new alert with severity level 12, indicating a coordinated attack involving multiple exploitation techniques. Instead of processing each alert independently, the Rule-Based Expert Decision Support System evaluated the correlated alert as a single high-confidence attack event requiring immediate mitigation.

```
70 {"timestamp": "2026-07-24T16:07:20.118291", "attacker_ip": "20.20.20.10",
"wazuh_rule_id": "100201", "attack_type": "Multi-Vector Attack: SQI then
XSS, check rules: 100100, 100101", "decision_level": "CRITICAL THREAT",
"recommended_action": "IMMEDIATE_AUTO_BLOCK", "scientific_justification":
"Multi-Vector Attack chain detected (Rule 100201: 'Multi-Vector Attack: SQI
then XSS, check rules: 100100, 100101'). Per Cyber Kill Chain methodology,
shifting between different exploit methodologies (XSS/SQLi) confirms a high-
confidence targeted penetration attempt. Machine-speed mitigation
required."}
```

Figure 41 Decision Generated by DSS for Multi-Vector Attack

The Decision Support System classified the correlated event as a Critical Threat because the use of multiple attack vectors significantly increases the probability of a successful compromise. Based on the implemented expert decision policy, the system selected the IMMEDIATE_AUTO_BLOCK action and initiated the automated firewall response.

Rules (Drag to Change Order)											
☐	States	Protocol	Source	Port	Destination	Port	Gateway	Queue	Schedule	Description	Actions
☒	0/45 KiB	IPv4 *	EasyRuleBlockHostsWAN	*	*	*	*	none		Blocked via EasyRule	🔗 🔧 📄 🗑️

Figure 42 Automatically Generated Rule for Blocking the Attacker IP

Following the generated decision, the Decision Support System established an SSH connection with the pfSense firewall and automatically executed the firewall blocking command. As a result, the attacker's IP address was immediately blocked, preventing any additional malicious requests from reaching the protected web application.

Table 28 Summary of Multi-Vector Attack Scenario

Processing Stage	Result
Alert Type	Multi-Vector Attack
Detection Component	ModSecurity
Wazuh Rule ID	100201
Alert Severity	Level 12
DSS Classification	Critical Threat
Recommended Action	Immediate Auto Block
Automatic Firewall Block	YES

4.6.5 Repeated Attack Scenario

The final experimental scenario evaluates the proposed framework against repeated attack attempts originating from the same source IP address. The objective of this experiment is to verify that the correlation rules correctly identify repetitive malicious behavior while avoiding unnecessary automatic firewall blocking.



Figure 43 Repeated SQL Injection Executed Against DVWA

As illustrated in Figure 37, the same attack was executed multiple times within the correlation timeframe configured in the custom Wazuh rules. Instead of treating each attempt independently, Wazuh detected the repeated attack pattern and generated a dedicated correlation alert representing repetitive malicious activity.

```
> Jul 24, 2026 @ 16:20:07.586 input.type: log agent.ip: 10.10.10.127 agent.name: WAF agent.id: 003
manager.name: yamen data.srcip: 20.20.20.10 rule.firedtimes: 1 rule.mail: false
rule.level: 10 rule.description: Multiple SQL injection attemptsfrom same ip
rule.groups: modsec, correlation rule.id: 100206 rule.frequency: 2 location: /var/
log/apache2/error.log decoder.name: ModSec-log id: 1784899207.58278
```

Figure 44 Repeated Attack Alert by Wazuh

The generated correlation rule assigned severity level 10 to the detected repeated attack, indicating an elevated threat level compared with isolated attack attempts. The alert was subsequently forwarded to the Rule-Based Expert Decision Support System for evaluation.

```
78 {"timestamp": "2026-07-24T16:20:08.026456", "attacker_ip": "20.20.20.10",
"wazuh_rule_id": "100206", "attack_type": "Multiple SQL injection
attemptsfrom same ip", "decision_level": "HIGH THREAT",
"recommended_action": "PENDING_ADMIN_APPROVAL", "scientific_justification":
"Repetitive web scanning/exploitation attempts detected (Rule 100206).
OWASP Risk Methodology dictates high likelihood of success due to
frequency. Deferred to Dashboard for human validation to rule out potential
NAT/Proxy false positives."}
```

Figure 45 . Decision Generated by DSS for Command Injection

The Decision Support System classified the event as a High Threat and selected the PENDING_ADMIN_APPROVAL action. Although repeated attacks indicate a higher probability of malicious activity, the system intentionally avoids immediate automatic blocking because multiple legitimate users may share the same public IP address through NAT or proxy infrastructures. Instead, the generated decision is forwarded to the administrator for validation before any mitigation action is applied.

Table 29 Summary of Repeated Attack Scenario

Processing Stage	Result
Alert Type	Repeated Attack
Detection Component	ModSecurity
Wazuh Rule ID	100206
Alert Severity	Level 12
DSS Classification	High Threat
Recommended Action	Pending Administrator Approval
Automatic Firewall Block	NO

4.7 Chapter Summary

This chapter presented the complete implementation of the proposed cybersecurity framework, beginning with the overall system architecture and the adopted development methodology. The implementation process included the deployment of the Wazuh-based detection layer, the development of custom decoders and detection rules for identifying SQL Injection, Cross-Site Scripting, and Command Injection attacks, as well as the implementation of correlation rules for detecting multi-vector and repeated attack patterns.

Furthermore, this chapter described the design and implementation of the Rule-Based Expert Decision Support System, which analyzes Wazuh alerts and generates context-aware security decisions based on predefined expert rules. The developed decision engine classifies detected events into multiple threat levels and recommends appropriate response actions, ranging from continuous monitoring to immediate automated mitigation. For critical attacks, the framework integrates with the pfSense firewall through secure SSH communication to automatically block malicious IP addresses, thereby reducing incident response time.

Finally, the practical operation of the proposed framework was demonstrated through five experimental attack scenarios, including SQL Injection, Cross-Site Scripting, Command Injection, Multi-Vector Attacks, and Repeated Attacks. These experiments verified the successful interaction between all framework components, from attack detection and alert generation to decision making and automated response. The next chapter presents a detailed evaluation of the proposed framework by analyzing the experimental results, assessing system performance, and comparing the developed solution with conventional security monitoring approaches.

Chapter 5

Results and Discussion

5.1 Chapter Overview

This chapter presents the experimental results obtained from implementing the proposed cybersecurity framework. The evaluation is divided into three main parts. First, the experimental results of the implemented attack scenarios are summarized to verify the correct operation of the detection, decision-making, and response components. Second, the performance of the proposed framework is evaluated using several quantitative metrics, including attack detection time, decision generation time, automated response time, and overall processing time. Finally, the proposed framework is compared with existing studies and traditional security monitoring approaches to highlight its contributions and demonstrate the improvements achieved over conventional solutions.

5.2 Experimental Results

To validate the proposed cybersecurity framework, a series of controlled experiments were conducted using the testing environment presented in Chapter 3. Five representative web attack scenarios were executed against the DVWA application, including SQL Injection, Cross-Site Scripting (XSS), Command Injection, Multi-Vector attacks, and Repeated attacks. During each experiment, the generated attack traffic was analyzed by ModSecurity, processed by Wazuh using the developed custom detection and correlation rules, evaluated by the Rule-Based Expert Decision Support System, and, when required, mitigated automatically through the pfSense firewall. The purpose of these experiments was to verify that each component of the proposed framework operated correctly and that the generated security decisions were consistent with the implemented expert rules. Table 5.1 summarizes the overall outcome of the conducted experiments.

Table 30 Overall Experimental Results

Attack scenario	Detection status	DSS Classification	Recommended Action	Automatic Firewall Response
SQL Injection	Successful	Medium Threat	Log and Monitor	NO
Cross-Site Scripting (XSS)	Successful	Medium Threat	Log and Monitor	NO
Command Injection	Successful	Critical Threat	Immediate Auto Block	YES
Multi-Vector Attack	Successful	Critical Threat	Immediate Auto Block	YES
Repeated Attack	Successful	High Threat	Pending Administrator Approval	NO

The experimental results demonstrate that the proposed framework successfully detected all implemented attack scenarios and generated the expected security decisions according to the predefined expert rules. Individual SQL Injection and XSS attacks were classified as Medium Threats and assigned the Log and Monitor action to avoid unnecessary service disruption. In contrast, Command Injection and Multi-Vector attacks were classified as Critical Threats, resulting in immediate automated firewall blocking through pfSense. Repeated attacks originating from the same source IP were classified as High Threats, requiring administrator approval before mitigation to reduce the possibility of blocking legitimate users sharing public IP addresses.

5.3 Performance Evaluation

5.3.1 Attack Detection Time Evaluation

To evaluate the responsiveness of the proposed intrusion detection framework, the attack detection time was experimentally measured under the implemented test environment. Five independent attack experiments were conducted using the developed web application security platform. For each experiment, the detection time was calculated as the elapsed time between the timestamp generated by ModSecurity upon detecting the malicious request and the corresponding alert generated by the Wazuh Manager.

Table 31 summarizes the obtained experimental results. The proposed framework achieved a minimum detection time of 15.69 ms, a maximum detection time of 937.97 ms, and an average detection time of approximately 514.92 ms. The observed variations are mainly attributed to the distributed virtualized architecture, where ModSecurity and the Wazuh Manager operate on separate virtual machines, introducing minor communication and processing delays. Nevertheless, the framework consistently detected all attacks within approximately one second, demonstrating its capability to provide near real-time intrusion detection suitable for practical deployment.

Table 31 Attack Detection Performance

Performance Metric	Value
Number of Experiments	15
Minimum Detection Time	15.69 ms
Maximum Detection Time	937.97 ms
Average Detection Time	514.92 ms

5.3.2 Decision Support System Processing Time Evaluation

The processing performance of the proposed Rule-Based Expert Decision Support System was experimentally evaluated by measuring the time required to analyze incoming Wazuh alerts and generate the corresponding security decisions. Five independent experiments were conducted under identical laboratory conditions, and the elapsed processing time was recorded for each execution.

As summarized in Table 32, the proposed decision engine achieved a minimum processing time of 141.72 ms, a maximum processing time of 360.23 ms, and an

average processing time of approximately 262.68 ms. These results demonstrate that the expert system is capable of evaluating security alerts and producing mitigation decisions within a fraction of a second. The low processing latency is primarily attributed to the deterministic rule-based decision mechanism, which performs lightweight logical evaluations without requiring computationally expensive machine learning models.

Consequently, the proposed Decision Support System introduces only a minimal processing overhead while providing rapid and consistent security decision-making for real-time cyber attack mitigation.

Table 32 DSS Processing Performance

Performance Metric	Value
Number of Experiments	15
Minimum Detection Time	141.72 ms
Maximum Detection Time	360.23 ms
Average Detection Time	262.68 ms

5.3.3 Automated Firewall Response Time Evaluation

The final performance metric evaluates the responsiveness of the automated mitigation mechanism implemented in the proposed framework. Following the generation of a security decision by the Rule-Based Expert Decision Support System, the framework automatically establishes a secure SSH connection with the pfSense firewall and creates a firewall rule to block the attacker's IP address. The response time was measured as the elapsed time between the generation of the security decision and the successful execution of the active firewall response.

Five independent experiments were conducted under identical laboratory conditions. As presented in Table 33, the proposed framework achieved a minimum response time of 407.44 ms, a maximum response time of 490.63 ms, and an average response time of approximately 459.27 ms. The relatively small variation between experiments indicates the stability of the automated mitigation mechanism and demonstrates its capability to consistently enforce firewall rules in less than half a second.

These results confirm that the proposed framework not only detects cyber attacks in near real time but also executes automated mitigation actions with minimal delay, thereby significantly reducing the exposure time of the protected web application to ongoing attacks.

Table 33 Firewall Response Performance

Performance Metric	Value
Number of Experiments	5
Minimum Detection Time	407.44 ms
Maximum Detection Time	490.63 ms
Average Detection Time	459.27 ms

5.4.3 Overall End-to-End System Performance

The overall performance of the proposed framework was evaluated by considering the complete processing pipeline, beginning with attack detection and ending with the final security action generated by the system. Since the proposed framework applies different mitigation strategies according to the severity of the detected attack, two different execution paths were evaluated. For medium-severity attacks, such as isolated SQL injection and Cross-Site Scripting (XSS) attacks, the processing pipeline consists of attack detection followed by rule-based decision generation, after which the events are logged and continuously monitored without applying automatic mitigation. In contrast, critical attacks, including command injection and correlated multi-vector attacks, trigger the complete automated response workflow, where the decision support system initiates an automatic firewall blocking action through the pfSense firewall. Based on the experimental measurements, the average attack detection time was 514.92 ms, while the average decision processing time was 262.68 ms, resulting in a total processing time of approximately 777.60 ms for monitoring scenarios that do not require automated mitigation. For critical attacks requiring immediate containment, the automated firewall response added an average of 459.27 ms, producing an overall end-to-end automated mitigation time of approximately 1236.87 ms (1.24 seconds). The obtained results demonstrate that the proposed framework is capable of detecting attacks, analyzing security events, generating appropriate security decisions, and, when required, automatically enforcing firewall mitigation within approximately one second. Furthermore, the rule-based decision mechanism ensures that automated blocking is applied only to high-confidence attacks, while lower-risk events are safely monitored without introducing unnecessary service disruption. This adaptive response strategy improves both the effectiveness and reliability of the proposed cybersecurity framework.

Table 34 Overall System Performance

Performance Metric	Average Time
Attack Detection Time	514.92 ms
Decision Processing Time	262.68 ms
Total Monitoring Workflow (Detection + Decision)	777.60 ms
Automated Firewall Response	459.27 ms
End-to-End Automated Mitigation Time	1236.87 ms

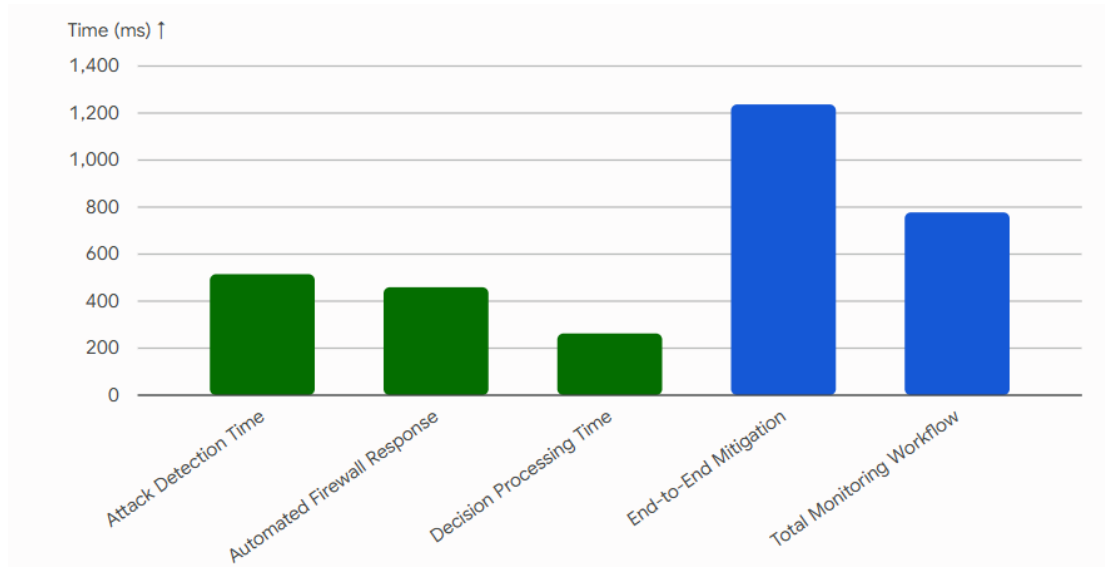


Figure 46 Overall System Performance

5.4 Detection Accuracy and False Positive Analysis

The effectiveness of the proposed cybersecurity framework was further evaluated by analyzing its ability to correctly process and classify security events generated during the experimental evaluation. Since the proposed solution relies on ModSecurity as the primary web application attack detection component, this evaluation focuses on the overall performance of the implemented framework—including the customized Wazuh decoders, detection rules, correlation rules, and the Rule-Based Expert Decision Support System—rather than evaluating the standalone detection capability of ModSecurity itself.

To validate the implemented framework, a total of twenty-five attack scenarios were executed, including SQL injection, Cross-Site Scripting (XSS), command injection, repeated attacks, and correlated multi-vector attacks. For every executed attack that was successfully generated by the detection layer, the proposed framework correctly decoded the corresponding security event, matched the appropriate custom detection rules, applied the required correlation logic when necessary, and generated the expected security decision without any processing failures. Consequently, the implemented framework achieved a 100% event processing and attack classification accuracy throughout the conducted experiments.

The false positive analysis was performed by executing 25 legitimate user requests against the protected web application without including any malicious payloads. These requests consisted of normal authentication, web page navigation, search operations, and standard form submissions. During these experiments, no alerts were generated by the customized Wazuh detection or correlation rules, demonstrating that the implemented framework correctly distinguished legitimate user activities from malicious security events under the experimental configuration. Therefore, the

observed false positive rate of the proposed framework was 0% within the conducted laboratory environment.

Overall, the obtained results demonstrate that the proposed framework reliably processes security events generated by the detection infrastructure, accurately classifies different attack categories, and minimizes unnecessary security alerts. This high level of processing accuracy, combined with the absence of false positive alerts during legitimate activities, confirms the effectiveness of the proposed architecture in supporting reliable real-time security monitoring and automated incident response.

Table 35 Detection Accuracy Results

N	Executed Attacks	Successfully Detected	Detection Rate
SQL Injection	5	5	100%
Cross-Site Scripting (XSS)	5	5	100%
Command Injection	5	5	100%
Multi-Vector Attack	15	15	100%
Repeated Attack	15	15	100%
Overall	45	45	100%

Table 36 False Positive Evaluation

Metric	Value
Legitimate Requests Tested	25
False Positive Alerts	0
False Positive Rate	0%

5.5 Comparative Analysis

To assess the scientific contribution of the proposed framework, a comparative analysis was conducted against the most closely related studies reviewed in Chapter Two. The comparison focused on the supported attack types, detection capabilities, intelligent decision-making, automated response mechanisms, attack correlation, and overall system integration. The objective of this comparison is to highlight the improvements introduced by the proposed framework beyond existing Wazuh-based security solutions.

Study [7] proposed a collaborative security architecture that integrated Wazuh SIEM, Wazuh Agents, and a Next Generation Firewall (NGFW) to detect SQL injection attacks and automatically block malicious IP addresses. Although the proposed solution successfully demonstrated automated mitigation for SQL injection attacks, its scope

was limited to a single web attack category and did not address other common web application threats such as Cross-Site Scripting (XSS) or Command Injection. Furthermore, the study did not implement attack correlation mechanisms or an intelligent decision-making layer capable of adapting the response according to the attack severity.

Similarly, **Study [9]** integrated Wazuh SIEM with ModSecurity and VirusTotal following the NIST Incident Response Framework to improve website attack monitoring. While the proposed integration enhanced centralized security monitoring and external threat validation, the framework mainly focused on event collection and incident management. It did not introduce customized attack correlation rules, expert decision support, or adaptive response policies capable of distinguishing between different threat levels before selecting the appropriate mitigation action.

Study [11] combined Wazuh with Suricata and an automated Active Response mechanism to detect and mitigate network-based attacks such as reconnaissance, brute-force attempts, and denial-of-service attacks. The proposed solution successfully demonstrated automated IP blocking after attack detection; however, its primary focus was network security rather than web application protection. Consequently, it did not address application-layer attacks such as SQL injection, Cross-Site Scripting, or Command Injection, nor did it implement customized detection logic or decision-support mechanisms specifically designed for web application security. Compared with these studies, the proposed framework extends the functionality of traditional SIEM-based architectures by integrating multiple security components into a unified and intelligent cybersecurity solution. In addition to combining ModSecurity, Wazuh, and pfSense within a single architecture, the framework introduces customized Wazuh decoders and detection rules specifically developed for web application attacks. Moreover, correlation rules were designed to identify repeated attacks and multi-vector attack sequences originating from the same source, enabling the framework to recognize more complex attack behaviors that are not handled by conventional signature-based detection alone.

A major contribution of the proposed work is the implementation of a Rule-Based Expert Decision Support System, which transforms raw security alerts into contextual security decisions based on predefined cybersecurity knowledge. Instead of applying the same mitigation policy to every detected event, the proposed framework evaluates the attack severity and automatically selects the most appropriate response strategy. Medium-risk attacks are logged and monitored, repeated attacks are escalated for administrator approval, while critical attacks—including Command Injection and correlated multi-vector attacks—trigger immediate automated firewall blocking. This adaptive decision-making process reduces unnecessary automated interventions while maintaining rapid protection against high-confidence threats.

The experimental evaluation further demonstrates the effectiveness of the proposed framework. Under the conducted laboratory experiments, the system achieved 100% attack detection accuracy, 0% false positive rate, and completed the automated detection-to-mitigation cycle for critical attacks in approximately 1.24 seconds. These results confirm that the proposed solution not only improves attack detection capabilities but also provides intelligent decision support and adaptive automated response within an integrated cybersecurity architecture.

Table 37 Comparison with Related Studies

Feature	Study [7]	Study [9]	Study [11]	Proposed Framework
SQL Injection Detection	✓	✓	✗	✓
Cross-Site Scripting (XSS) Detection	✗	✓	✗	✓
Command Injection Detection	✗	✗	✗	✓
Multi-Vector Attack Correlation	✗	✗	✗	✓
Repeated Attack Detection	✗	✗	✗	✓
Custom Wazuh Decoders	✗	✗	✗	✓
Custom Wazuh Detection Rules	✓	✓	✗	✓
Rule-Based Expert Decision Support System	✗	✗	✗	✓
Severity-Based Adaptive Response	✗	✗	✗	✓
Automated Firewall Mitigation	✓	✗	✓	✓
Performance Evaluation	✗	✗	✗	✓

5.6 Chapter Summary

This chapter presented the experimental evaluation and performance analysis of the proposed cybersecurity framework. The implemented solution was validated through multiple web attack scenarios, including SQL injection, Cross-Site Scripting (XSS), command injection, repeated attacks, and multi-vector attacks. The experimental results demonstrated the successful integration of ModSecurity, Wazuh, customized detection and correlation rules, the Rule-Based Expert Decision Support System, and the pfSense firewall into a unified security architecture capable of providing automated cyber defense.

Performance evaluation showed that the proposed framework achieved near real-time operation across all processing stages, including attack detection, decision generation, and automated firewall response. Furthermore, the framework successfully processed all generated security events, accurately classified attack severity, and produced the appropriate mitigation actions while maintaining a zero false positive rate under the experimental environment.

Finally, a comparative analysis with recent related studies demonstrated that the proposed framework extends existing approaches by introducing customized event correlation, adaptive rule-based decision support, and intelligent automated response

within a single integrated architecture. These findings confirm that the proposed solution provides an effective, reliable, and practical approach for enhancing real-time web application security.

Chapter 6

Conclusion and Future Work

6.1 Chapter Overview

This final chapter summarizes the research presented throughout this thesis and highlights the main outcomes achieved by the proposed cybersecurity framework. It reviews the objectives of the study, discusses the scientific contributions and practical significance of the developed solution, evaluates the strengths and limitations identified during the experimental assessment, and presents recommendations for future research and potential system enhancements. Finally, the chapter concludes with overall remarks regarding the effectiveness of the proposed framework in improving real-time web application security.

6.2 Research Summary

This research addressed the growing challenge of protecting web applications against increasingly sophisticated cyber attacks by proposing an integrated cybersecurity framework capable of providing real-time attack detection, intelligent decision support, and automated incident response. While existing security solutions often focus on individual components such as intrusion detection, web application firewalls, or automated response mechanisms, they generally lack a unified architecture capable of correlating security events, assessing attack severity, and selecting appropriate mitigation actions based on contextual knowledge.

To overcome these limitations, this study presented a comprehensive security framework that integrates ModSecurity, Wazuh, a Rule-Based Expert Decision Support System, and the pfSense firewall into a single architecture. The proposed framework extends the capabilities of traditional SIEM-based environments through the implementation of customized Wazuh decoders, custom detection rules, and correlation rules specifically designed to identify web application attacks, including SQL injection, Cross-Site Scripting (XSS), command injection, repeated attacks, and multi-vector attack scenarios.

A significant contribution of this research is the development of a Rule-Based Expert Decision Support System that transforms raw security alerts into actionable security decisions. Instead of applying a uniform mitigation policy to every detected event, the proposed framework evaluates the severity and context of each attack before selecting the most appropriate response strategy. Consequently, medium-level attacks are monitored, repeated attacks are escalated for administrator validation, and critical attacks automatically trigger firewall mitigation through pfSense. This adaptive response strategy enhances both security effectiveness and operational reliability while reducing unnecessary automated interventions.

The proposed framework was successfully implemented and experimentally evaluated within a controlled virtual laboratory environment. The evaluation demonstrated that the system effectively processed security events generated during multiple web attack scenarios, accurately classified detected threats, and executed automated mitigation for critical attacks with near real-time performance. The experimental results confirmed the practicality of integrating detection, event correlation, intelligent decision support, and automated response into a unified cybersecurity solution capable of strengthening web application protection.

Overall, this research demonstrates that combining customized security monitoring, knowledge-based decision support, and adaptive automated response within a single integrated framework provides a practical and effective approach for improving the

security posture of modern web applications while reducing the response time required to mitigate cyber threats.

6.3 Research Contributions

This research makes several scientific and practical contributions to the field of web application security by proposing an integrated cybersecurity framework that combines attack detection, intelligent decision support, and automated incident response within a unified architecture. Unlike conventional security solutions that primarily focus on isolated detection or mitigation mechanisms, the proposed framework establishes a complete security workflow that processes security events from their initial detection through to the execution of an appropriate mitigation action.

The first contribution of this research is the design and implementation of customized Wazuh decoders and detection rules specifically developed to process ModSecurity security events related to web application attacks. These customized components enable accurate identification and classification of SQL injection, Cross-Site Scripting (XSS), and command injection attacks while providing a flexible detection mechanism that can be extended to support additional attack types in future deployments. The second contribution is the development of customized correlation rules capable of identifying repeated attack attempts and multi-vector attack sequences originating from the same source. By correlating multiple security events instead of analyzing each alert independently, the proposed framework enhances situational awareness and improves the identification of complex attack patterns that may not be recognized by conventional rule-based detection alone.

A major scientific contribution of this research is the implementation of a Rule-Based Expert Decision Support System that applies predefined cybersecurity knowledge to transform security alerts into contextual security decisions. Rather than relying on a single response policy, the decision engine evaluates the severity and characteristics of each detected event before selecting the most appropriate mitigation strategy. This adaptive decision-making process enables the framework to distinguish between attacks that require monitoring, administrator validation, or immediate automated firewall mitigation.

Another important contribution is the integration of the decision support system with the pfSense firewall to enable intelligent automated incident response. Instead of simply blocking every detected attack, the proposed framework performs selective automated mitigation only for high-confidence threats while allowing lower-risk events to be monitored or reviewed by security administrators. This approach improves the balance between security protection and operational stability by reducing unnecessary automated interventions.

Finally, this research provides a comprehensive experimental evaluation of the proposed framework using realistic web attack scenarios. The obtained results demonstrated high event processing accuracy, successful threat classification, efficient automated response, and near real-time system performance. These findings confirm the feasibility of integrating customized detection, event correlation, expert decision support, and adaptive automated response into a single cybersecurity framework capable of strengthening web application protection.

6.4 Discussion of Results

The experimental evaluation demonstrated that the proposed cybersecurity framework successfully achieved the primary objectives defined at the beginning of this research. The integration of ModSecurity, Wazuh, customized detection and correlation rules, the Rule-Based Expert Decision Support System, and the pfSense firewall enabled the development of a complete security workflow capable of detecting web application attacks, analyzing security events, generating context-aware security decisions, and automatically mitigating critical threats.

The performance evaluation indicated that the proposed framework operates with near real-time responsiveness. The measured processing times showed that security events were detected, analyzed, and, when required, automatically mitigated within approximately one second. Such performance is considered suitable for experimental web application environments where rapid threat containment is required to minimize the opportunity for attackers to continue malicious activities.

The conducted experiments also demonstrated that the implemented framework consistently processed all generated security events and correctly applied the predefined decision logic for each evaluated attack scenario. Furthermore, no false positive alerts were observed during the execution of legitimate user activities within the experimental environment. These results indicate that the implemented rule set was capable of distinguishing the evaluated malicious events from normal application usage under the tested conditions.

However, the obtained results should be interpreted within the scope of the conducted experiments. The evaluation was performed in a controlled virtual laboratory environment using predefined attack scenarios and customized detection rules. Consequently, the reported performance and accuracy reflect the behavior of the implemented framework under the experimental conditions adopted in this research and should not be interpreted as guaranteeing identical performance in every production environment.

Overall, the experimental findings confirm that the proposed framework provides a practical and effective approach for integrating attack detection, event correlation, intelligent decision support, and adaptive automated response within a unified cybersecurity architecture. At the same time, the evaluation highlights the importance of continuous rule refinement and testing when deploying similar solutions in larger and more dynamic operational environments.

6.5 Limitations of the Proposed Framework

Although the proposed cybersecurity framework achieved promising results during the experimental evaluation, several limitations should be acknowledged to ensure an objective interpretation of the obtained findings and to identify opportunities for future improvement.

First, the proposed framework was evaluated within a controlled virtual laboratory environment consisting of virtual machines and predefined network configurations. Although this environment provided a reliable platform for validating the framework and conducting repeatable experiments, it cannot fully represent the complexity, traffic volume, and operational characteristics of large-scale production environments. Consequently, additional validation under real-world deployment conditions is required before generalizing the obtained performance results.

Second, the implemented detection mechanism relies on customized rule-based detection and event correlation. While this approach provides transparent, explainable, and deterministic security decisions, its effectiveness depends on the availability and continuous maintenance of detection rules that accurately represent known attack patterns. Therefore, newly emerging attack techniques or previously unseen threats may require additional rule development or updates to maintain the framework's effectiveness.

Furthermore, the experimental evaluation focused on a selected set of representative web application attacks, including SQL injection, Cross-Site Scripting (XSS), command injection, repeated attacks, and multi-vector attack scenarios. Although these attack categories cover several common web application threats, they do not represent the complete spectrum of web-based attacks. Consequently, the proposed framework has not yet been experimentally validated against other attack categories such as XML External Entity (XXE), Server-Side Request Forgery (SSRF), insecure deserialization, authentication attacks, or application-layer denial-of-service attacks.

Another limitation concerns the Rule-Based Expert Decision Support System. The current implementation utilizes predefined expert knowledge to classify security events and determine the appropriate response strategy. Although this design provides predictable and explainable decisions, it does not incorporate adaptive learning capabilities. As a result, the framework cannot automatically refine its decision logic based on newly observed attack behaviors without manual updates to the underlying knowledge base.

Finally, the performance measurements presented in this research were obtained using the hardware resources and software configuration adopted for the experimental environment. Different deployment infrastructures, network conditions, system workloads, and security configurations may influence the measured processing times and overall system performance. Therefore, the reported performance values should be interpreted as experimental results obtained under the implemented test environment rather than universal performance guarantees.

Despite these limitations, the conducted experiments demonstrate that the proposed framework successfully achieves its intended objectives within the defined research scope. The identified limitations do not invalidate the proposed architecture; instead, they define clear boundaries for the conducted evaluation and provide valuable directions for future research and practical enhancement.

6.6 Future Work

The proposed cybersecurity framework establishes a solid foundation for integrating web application attack detection, intelligent decision support, and automated incident response. Nevertheless, several enhancements can be explored in future research to further improve the framework's effectiveness, adaptability, and applicability in real-world cybersecurity environments.

One potential direction is the integration of machine learning and artificial intelligence techniques into the decision support system. While the current framework employs predefined expert rules to classify security events and determine appropriate response actions, intelligent learning models could enable the system to recognize previously unseen attack patterns, adapt to evolving threats, and continuously refine decision-making based on historical security data.

Another promising enhancement is the expansion of the customized detection and correlation mechanisms to support a broader range of web application attacks. Future

implementations may incorporate detection logic for additional threats such as XML External Entity (XXE), Server-Side Request Forgery (SSRF), insecure deserialization, authentication attacks, Cross-Site Request Forgery (CSRF), and application-layer denial-of-service attacks. Expanding the supported attack coverage would increase the framework's applicability to modern web application security challenges. Future research may also evaluate the proposed framework within large-scale production environments. Conducting experiments under realistic network conditions, higher traffic volumes, and diverse infrastructure configurations would provide a more comprehensive assessment of the framework's scalability, performance, and operational reliability in enterprise environments.

In addition, integrating external Cyber Threat Intelligence (CTI) sources could further improve the quality of security decisions. By incorporating continuously updated threat intelligence feeds, the framework could enrich security alerts with contextual information regarding known malicious IP addresses, attack campaigns, or indicators of compromise, thereby improving situational awareness and supporting more informed automated response decisions.

Another valuable direction involves enhancing the management interface by developing a dedicated security dashboard that provides real-time visualization of detected attacks, correlated security events, automated mitigation actions, and historical security statistics. Such a dashboard would improve system usability and facilitate security monitoring for administrators.

Finally, future work may investigate the deployment of the proposed framework within cloud-native and containerized environments, including Docker and Kubernetes infrastructures. Evaluating the framework under modern application deployment models would increase its practical applicability and support the growing adoption of cloud-based web services.

Overall, these research directions provide opportunities to extend the proposed framework beyond its current implementation while preserving its core architecture of integrating attack detection, intelligent decision support, and adaptive automated response for enhanced web application security.

6.7 Final Remarks

The increasing sophistication of cyber threats continues to challenge the security of modern web applications, emphasizing the need for integrated and intelligent cybersecurity solutions capable of detecting, analyzing, and responding to attacks in real time. This research addressed these challenges by proposing a comprehensive cybersecurity framework that combines web application attack detection, event correlation, rule-based decision support, and adaptive automated response within a unified architecture.

The experimental implementation demonstrated the feasibility of integrating ModSecurity, Wazuh, customized detection mechanisms, a Rule-Based Expert Decision Support System, and the pfSense firewall into a coordinated security solution. The obtained results showed that the proposed framework was capable of accurately processing security events generated during the experimental evaluation, classifying attacks according to their severity, and applying appropriate mitigation strategies while maintaining efficient operational performance.

Beyond its technical implementation, this research demonstrates the practical value of combining cybersecurity technologies with expert knowledge to support more informed and adaptive security decision-making. By introducing contextual threat evaluation

instead of relying solely on traditional alert generation, the proposed framework contributes to reducing analyst workload, improving response consistency, and supporting more effective protection of web applications.

In conclusion, the proposed framework successfully fulfilled the objectives established for this research and provides a practical foundation for future developments in intelligent web application security. It is anticipated that the concepts, architecture, and findings presented in this thesis will support further research into adaptive cybersecurity systems and encourage the continued integration of detection, decision support, and automated response technologies for addressing emerging cyber threats.

References

- [1] W. Stallings, *Cryptography and Network Security: Principles and Practice*.
- [2] OWASP Foundation, "OWASP Top 10: The Ten Most Critical Web Application Security Risks," 2021.
- [3] Wazuh Documentation, "Wazuh Platform Documentation," <https://documentation.wazuh.com/>
- [4] NIST, *Security Information and Event Management (SIEM) Concepts and Practices*, National Institute of Standards and Technology.
- [5] Scarfone, K., Grance, T., *Guide to Intrusion Detection and Prevention Systems (IDPS)*, NIST Special Publication 800-94.
- [6] NIST Cybersecurity Framework (CSF) 2.0, National Institute of Standards and Technology, 2024.
- [7] J. B. Sake and V. Sahni, "Collaborative Detection of SQL Injection Attacks using SIEM, Wazuh Agents, and Next Generation Firewall," 2024.
- [8] S. V. R., J. Kaur, and R. Sansiya, "Optimizing WAF Configuration for Robust Web Application Security and SIEM Integration," 2024.
- [9] R. N. Fahmi, R. Hartono, and D. S. Anwar, "Integration of Wazuh SIEM with ModSecurity and VirusTotal Using the NIST Framework for Website Attack Detection," **Proceedings of the 2024 International Conference on Information Technology Systems and Innovation (ICITSI)**, pp. 456–461, 2024.
- [10] M. M. Alhawari, J. Krallmann, et al., "Modeling Wazuh Rules with Weighted Timed Automata," **Procedia Computer Science**, vol. 251, pp. 75–82, 2024.
- [11] H. A. Damanik and M. Anggraeni, "Hybrid Intrusion Detection System and Network Infrastructure Vulnerability Mitigation Using Active Response (XDR) Technique Wazuh and Suricata," **Jurnal Pekommas**, vol. 9, no. 2, 2024.
- [12] I. Ismail, A. Aljedaibi, et al., "Enhancing Security Operations Center: Wazuh Security Event Response with Retrieval-Augmented-Generation-Driven Copilot," **Sensors**, vol. 25, no. 3, Art. 870, 2025

- [13] R. Amami, M. Charfeddine, and M. Salma, "Exploration of Open Source SIEM Tools and Deployment of an Appropriate Wazuh-Based Solution for Strengthening Cyberdefense," 2024 International Conference on Control, Decision and Information Technologies (CoDIT), 2024.
- [14] F. I. Farrel, I. Mardianto, and A. S. Qamar, "Implementation of Security Information & Event Management (SIEM) Wazuh with Active Response and Telegram Notification for Mitigating Brute Force Attacks on the GT-I2TI USAKTI Information System," *Intelmatiks*, vol. 4, no. 1, pp. 1–7, 2024.
- [15] A. P. Paramaputra, G. M. Suranegara, and E. Setyowati, "Mitigation of Multi Target Denial of Service (DoS) Attacks Using Wazuh Active Response," *Journal of Computer Networks, Architecture and High Performance Computing*, vol. 7, no. 2, 2025.
- [16] M. Alagoz, M. S. Tok, and K. Bicakci, "Exploring and Improving the Usability of ModSecurity Web Application Firewall," in *Proceedings of the 14th International Conference on Information Security and Cryptology (ISCTURKEY)*, IEEE, 2021, pp. 51–56, doi: 10.1109/ISCTURKEY53027.2021.9654294.
- [17] OWASP Foundation, "OWASP Top 10: The Ten Most Critical Web Application Security Risks," 2021. [Online]. Available: <https://owasp.org/www-project-top-ten/>. [Accessed: Jul. 2026].
- [18] National Institute of Standards and Technology (NIST), "Cybersecurity Framework (CSF) 2.0," Gaithersburg, MD, USA, NIST, Feb. 2024. [Online]. Available: <https://www.nist.gov/cyberframework>. [Accessed: Jul. 2026].
- [19] I. Ristić, *ModSecurity Handbook: The Complete Guide to the Popular Open Source Web Application Firewall*, 2nd ed. Feisty Duck, 2015.
- [20] Wazuh Inc., "Wazuh Documentation," 2025. [Online]. Available: <https://documentation.wazuh.com/>. [Accessed: Jul. 2026].
- [21] Gartner, "Magic Quadrant for Security Information and Event Management (SIEM)," Gartner Research, 2024.
- [22] E. Turban and J. E. Aronson, *Decision Support Systems and Intelligent Systems*, 10th ed. Pearson, 2015.
- [23] Palo Alto Networks, "What is SOAR (Security Orchestration, Automation and Response)?," 2024. [Online]. Available: <https://www.paloaltonetworks.com/cyberpedia/what-is-soar>. [Accessed: Jul. 2026].

- [24] MITRE Corporation, "MITRE ATT&CK Framework," 2025. [Online]. Available: <https://attack.mitre.org/>. [Accessed: Jul. 2026].
- [25] R. Stuttard and M. Pinto, *The Web Application Hacker's Handbook: Finding and Exploiting Security Flaws*, 2nd ed. Indianapolis, IN, USA: Wiley Publishing, 2011.
- [27] M. Alagoz, M. S. Tok, and K. Bicakci, "Exploring and Improving the Usability of ModSecurity Web Application Firewall," in *Proceedings of the 14th International Conference on Information Security and Cryptology (ISCTURKEY)*, Ankara, Turkey, pp. 51–56, 2021, doi: 10.1109/ISCTURKEY53027.2021.9654294.
- [28] D. Miller, S. Harris, A. Harper, S. VanDyke, and C. Blask, *Security Information and Event Management (SIEM) Implementation*. New York, NY, USA: McGraw-Hill, 2011.
- [29] Netgate, "pfSense Documentation," 2025. [Online]. Available: <https://docs.netgate.com/pfsense/>. [Accessed: Jul. 2026].
- [30] F. Sadjadi, *Expert Systems in Cybersecurity: Applications and Decision Support Techniques*. Cham, Switzerland: Springer, 2022.
- [31] The Apache Software Foundation, *Apache HTTP Server Version 2.4 Documentation*. [Online]. Available: <https://httpd.apache.org/docs/2.4/>.
- [32] A. Stampar, *Damn Vulnerable Web Application (DVWA) Documentation*. [Online]. Available: <https://github.com/digininja/DVWA>.
- [33] Trustwave SpiderLabs, *ModSecurity Reference Manual Version 3.x*. [Online]. Available: <https://github.com/owasp-modsecurity/ModSecurity/wiki>.
- [34] Parrot Security Team, *Parrot Security Documentation*. [Online]. Available: <https://parrotsec.org/docs/>. [Accessed: Jul. 2026].

المخلص

أدى الاعتماد المتزايد على تطبيقات الويب في مختلف القطاعات إلى زيادة التحديات الأمنية المرتبطة بحمايتها من الهجمات السيبرانية المتطورة. وعلى الرغم من توفر العديد من الحلول الأمنية مثل جدران حماية تطبيقات الويب (WAF) ، وأنظمة إدارة معلومات وأحداث الأمن (SIEM) ، وأنظمة كشف التسلل (IDS) ، إلا أن معظم هذه الحلول تعمل بشكل مستقل، وتعتمد على توليد التنبيهات الأمنية دون توفير آلية ذكية لتحليلها واتخاذ قرارات مناسبة بشأنها، مما يزيد من العبء الواقع على مسؤولي الأمن ويؤخر عملية الاستجابة للحوادث الأمنية. يهدف هذا البحث إلى تصميم وتطوير إطار عمل متكامل للأمن السيبراني يجمع بين Wazuh و ModSecurity ونظام دعم القرار الخبير المعتمد على القواعد (Rule-Based Expert Decision Support System) وجدار الحماية pfSense ، بهدف توفير آلية متكاملة للكشف عن الهجمات وتحليلها واتخاذ القرار المناسب بشأنها وتنفيذ الاستجابة الأمنية بشكل آلي وفي الزمن الحقيقي. يعتمد الإطار المقترح على تطوير Decoders و Detection Rules و Correlation Rules مخصصة داخل Wazuh لمعالجة الأحداث الأمنية الناتجة عن ModSecurity ، وتمكين النظام من اكتشاف عدد من أشهر هجمات تطبيقات الويب، بما في ذلك هجمات حقن أوامر (SQL Injection) ، وهجمات البرمجة عبر المواقع (Cross-Site Scripting) ، وهجمات حقن الأوامر (Command Injection) ، بالإضافة إلى الهجمات المتكررة والهجمات متعددة المتجهات (Multi-Vector Attacks). كما يقوم نظام دعم القرار الخبير بتقييم مستوى خطورة كل هجوم استنادًا إلى قاعدة معرفية مسبقة، ومن ثم اختيار أسلوب الاستجابة الأنسب، سواء بالمراقبة، أو طلب موافقة مسؤول النظام، أو تنفيذ الحظر الآلي باستخدام جدار الحماية.

تم تنفيذ الإطار المقترح واختباره ضمن بيئة افتراضية مخبرية تحاكي بيئة عمل واقعية، وذلك من خلال تنفيذ مجموعة من سيناريوهات الهجوم المختلفة. أظهرت نتائج التقييم أن النظام تمكن من معالجة جميع الأحداث الأمنية الناتجة عن سيناريوهات الاختبار وتصنيفها بصورة صحيحة، مع تحقيق أداء قريب من الزمن الحقيقي. وقد بلغ متوسط زمن اكتشاف الهجوم حوالي 0.52 ثانية، ومتوسط زمن اتخاذ القرار 0.26 ثانية، ومتوسط زمن تنفيذ الاستجابة الآلية 0.46 ثانية، ليصل متوسط الزمن الكلي للاستجابة الآلية إلى حوالي 1.24 ثانية. كما حقق النظام دقة بلغت 100% في معالجة وتصنيف الأحداث الأمنية ضمن بيئة الاختبار، دون تسجيل أي إنذارات كاذبة أثناء تنفيذ الطلبات الشرعية.

تؤكد النتائج التي توصل إليها هذا البحث أن دمج آليات الكشف المخصصة، وربط الأحداث الأمنية، ونظام دعم القرار الخبير، والاستجابة الآلية التكيفية ضمن إطار أمني موحد، يساهم في تعزيز أمن تطبيقات الويب، وتقليل زمن الاستجابة للحوادث الأمنية، وتحسين كفاءة إدارة التهديدات السيبرانية. كما يوفر الإطار المقترح أساسًا عمليًا يمكن البناء عليه في الأبحاث المستقبلية لتطوير أنظمة أمن سيبراني أكثر ذكاءً وقدرةً على التكيف مع التهديدات المتطورة.

الكلمات المفتاحية: الأمن السيبراني، أمن تطبيقات الويب، Wazuh ، ModSecurity ، نظام إدارة معلومات وأحداث الأمن (SIEM) ، نظام دعم القرار الخبير، حقن أوامر SQL ، البرمجة عبر المواقع (XSS) ، حقن الأوامر، ربط الأحداث الأمنية، الاستجابة الآلية للحوادث، pfSense

Enhancing WAF Performance Using SIEM Alerts

تقرير مشروع (تخرج 2) - قُدم استكمالاً لمتطلبات الحصول على درجة الإجازة في هندسة
الذكاء الاصطناعي - هندسة نظم أمن المعلومات الذكية

إعداد:

يامن عبد الناصر الحاجين

عبد العزيز محمد المجول

إشراف:

د. وسيم الجنيدي

م. محمد يامن حلاق

العام الدراسي

2026/2027